SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Kody: Gamified Programming Learning Platform and Course Management System (K:GPLPCMS)

V 1.3

Prepared By:

De Guzman, Asher Nathaniel E.

Dela Cruz, Rupert C.

Macaraeg, Mark Cyrus P.

Date:

May, 2026

Developers	Reviewers
 Dela Cruz, Rupert C. Project Manager/Developer	Erna-Kristi Martinez Reviewer
 De Guzman, Asher Nathaniel E. UI Designer / Developer	Cabusao, Anthony Carl Reviewer
 Macaraeg, Mark Cyrus P. UI Designer / Developer	Jose, Colleen Margarette Reviewer
	Zapanta, Zeus Levi Reviewer

Revision History

Name	Date	Reason For Changes / Comments	Version
K:GPLPCMS	2026/02/27	• Complete first draft of SRS until section 3.6	V 1.0
K:GPLPCMS	2026/03/26	• Revised Sections 1 to 3.6 • Added completed draft of Sections 4 to 4.2	V 1.1
K:GPLPCMS	2026/04/24	• Revised Sections 1 to 4.2 • Added completed draft of Section 5	V 1.2
K:GPLPCMS	2026/05/09	• Final revision and addition of Appendices	V 1.3

Table of Contents

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of Kody: Gamified Programming Learning Platform with Course Management System (K:GPLPCMS). It specifies system behavior, operational constraints, and role-based interactions across all modules, ensuring traceability between requirements, use cases, and system components.

The document serves as a formal reference for system design, implementation, and validation, establishing a shared understanding among stakeholders regarding system functionality and boundaries. It supports verification activities, architectural decisions, and academic evaluation of system completeness and coherence (IEEE, 2018; Sommerville, 2016).

Intended Audience:

- System Developers
- Project Managers
- Quality Assurance Personnel
- Academic Reviewers
- Learners (Primary Users)
- Instructors and Contributors (Secondary Users)
- Design and Implementation Teams

1.2 Scope

Kody(K:GPLPCMS) is a web-based software product designed to deliver structured programming education integrated with interactive coding practice and deterministic gamification mechanisms.

The system:

- Provides a unified platform combining course management, coding challenge execution, and performance tracking within a single environment
- Supports role-based access control for Learners, Instructors, Contributors, Moderators, and Administrators, governing access to system functions and workflows
- Enables creation, management, and delivery of programming courses, modules, and coding challenges using structured content formats
- Integrates an in-browser coding execution environment with automated validation using predefined test cases for supported programming languages (Python, Java, C++)

- Implements a deterministic gamification system including experience points (XP), rank progression, reward tokens (KodeBits), and leaderboard rankings based on validated user activity
- Supports both free and premium content access, where premium content is unlocked through token usage or system-defined conditions
- Provides a unified user dashboard displaying progress, coding performance, and earned rewards

The system does not:

- Replace full-scale institutional Learning Management Systems (LMS) such as Google Classroom or Canvas
- Provide enterprise-level academic administration functions such as enrollment systems, grading registries, or institutional reporting
- Support non-programming subject domains in Version 1
- Implement probabilistic or chance-based reward mechanisms

Kody is intended to reduce fragmentation between structured learning systems and coding practice platforms by integrating validation, progression tracking, and engagement mechanisms into a single cohesive system.

1.3 Definitions, Acronyms, and Abbreviations

Definitions

- **KodeBits** - A virtual, system-controlled digital unit used as the primary medium for accessing premium content and rewarding validated user activity. KodeBits may be earned through system-defined actions or acquired via external payment, and are non-transferable between users.
- **Freemium** - A service model where basic features and selected content are available for free, while advanced or restricted content requires KodeBits or payment for access.
- **Token Economy** - The system-controlled structure governing the generation, distribution, usage, and limitation of KodeBits to maintain fairness and prevent abuse or inflation.
- **Standalone Module** - A learning module that can be accessed independently without being part of a structured course.
- **Deterministic Gamification** - A reward system in which experience points (XP), rankings, and KodeBits are granted based on predefined rules and measurable outcomes, without random or chance-based elements.

Acronyms

- **API** – Application Programming Interface; a set of protocols for integrating external services.
- **KB** – KodeBits; the system's virtual currency unit.
- **LMS** – Learning Management System; software used for structured course delivery and management.
- **RBAC** – Role-Based Access Control; a system model restricting access based on assigned user roles.
- **XP** – Experience Points; numerical values representing user progression within the gamification system.
- **OAuth** – Open Authorization; a standard protocol used for secure authentication via third-party providers.
- **TLS** – Transport Layer Security; a protocol used to secure data transmission over networks.

1.4 References

Amazon Web Services. (n.d.). Amazon EC2 Pricing. https://aws.amazon.com/ec2/pricing/

Amazon Web Services. (n.d.). Amazon RDS Pricing. https://aws.amazon.com/rds/pricing/

Amazon Web Services. (n.d.). Amazon S3 Pricing. https://aws.amazon.com/s3/pricing/

Dahlstrom, E., Brooks, D. C., & Bichsel, J. (2014). The current ecosystem of learning management systems in higher education. EDUCAUSE Center for Analysis and Research.

Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining gamification. Proceedings of the 15th International Academic MindTrek Conference, 9–15.

Hamari, J., Koivisto, J., & Sarsa, H. (2014). Does gamification work? A literature review of empirical studies on gamification. 47th Hawaii International Conference on System Sciences, 3025–3034.

IEEE. (2018). ISO/IEC/IEEE 29148:2018 Systems and software engineering—Life cycle processes—Requirements engineering.

Judge0. (n.d.). Jude0 API Pricing. https://judge0.com/#pricing

OAuth 2.0 Authorization Framework. (n.d.). OAuth.net. https://oauth.net/2/

OWASP Foundation. (2021). OWASP Top Ten. https://owasp.org

National Institute of Standards and Technology (NIST). (2020). Digital Identity Guidelines (SP 800-63B). https://pages.nist.gov/800-63-3/

Twilio Inc. (n.d.). Twilio SendGrid Email API. https://www.twilio.com/en-us/products/email-api

Xendit. (n.d.). Xendit Payment Gateway (Philippines). https://www.xendit.co/en-ph/

1.5 Overview

This Software Requirements Specification (SRS) is organized to present Kody from high-level context to detailed system behavior. Section 2 describes the business environment, objectives, and stakeholder context, providing a conceptual understanding of the system and its intended use.

Section 3 presents the overall system description, including system architecture, user characteristics, constraints, and operating environment, which are essential for design and implementation. Section 4 defines the system's functional requirements through structured use cases, forming the basis for development, validation, and testing activities.

This document is intended to support different stakeholder needs. End users and evaluators may focus on Section 2 to understand system purpose and value, while developers and technical teams should refer to Sections 3 and 4 for implementation details and system behavior specifications.

2 Business Description

2.1 Business Description

Kody operates within the education technology (EdTech) sector, specifically in the domain of online programming education and skill development platforms. This domain is characterized by the integration of digital learning delivery systems, automated assessment technologies, and user engagement mechanisms, typically deployed in web-based environments accessible to a broad learner base.

Programming education platforms are commonly divided into two functional categories: structured learning systems and interactive coding practice systems that provide automated validation. These systems often operate independently, resulting in fragmented workflows, inconsistent progress tracking, and reduced learner engagement across platforms.

Kody is positioned as an integrated EdTech solution that unifies structured course delivery, coding challenge validation, and deterministic gamification within a single platform. It operates in a hybrid educational model, supporting both self-paced learners and content creators (instructors and contributors), while incorporating governance mechanisms such as moderation and role-based access control to ensure content quality and system integrity.

The platform follows a freemium digital service model, offering both free and premium content, with access regulated through a token-based system and platform-defined conditions. This positions Kody at the intersection of education, digital content platforms, and microtransaction-based service ecosystems, influencing both learning outcomes and platform sustainability.

2.2 Business Objectives

The business objectives of Kody define measurable outcomes aligned with improving engagement, system performance, and platform sustainability within the education technology domain:

- **Increase Learner Engagement** – Achieve at least a 70% course completion rate within 3 months of user enrollment through structured progression and engagement mechanisms.
- **Ensure Automated Coding Validation** – Evaluate 100% of coding submissions using automated test cases, with feedback returned within 3-5 seconds per submission during normal system operation.
- **Establish Monetization Structure** – Implement a freemium content model in Version 1, with at least 30% of available courses designated as premium, accessible through token-based unlocking or direct purchase.
- **Support Scalable Content Creation** – Enable the creation and publication of 20–50 courses or coding challenges per month, depending on contributor activity, using structured templates and moderated workflows.
- **Unify Performance Tracking** – Provide a centralized dashboard integrating user progress, coding results, and reward metrics, with data updates reflected within 2 seconds of validated activity.
- **Ensure System Accessibility** – Maintain compatibility across modern web browsers, ensuring at least 95% of core system features are accessible on both desktop and mobile platforms.

2.3 Stakeholder Profile

Representative	Martinez, Erna-Kristi
Description	Reviewer
Type	Project Reviewer
Responsibilities	Review SRS alignment with academic standards, validate assessment structures, ensure clarity of documentation
Involvement	Requirements review and approval, provides academic oversight and validates

	pedagogical relevance of system requirements.
Deliverables	K:GPLPCMS SRS, System Profile, Project Documentation
Comments / Issues	

Representative	Cabusao, Anthony Carl Jose, Colleen Margarette Zapanta, Zeus Levi
Description	Reviewer
Type	Reviewer, Learner User
Responsibilities	Provide feedback on usability expectations, validate clarity of system functionality descriptions
Involvement	Requirements review
Deliverables	Feedbacks
Comments / Issues	

Representative	Dela Cruz, Rupert C.
Description	Project Manager
Type	Requirement Analyst, System Developer
Responsibilities	Define project scope and deliverables,, coordinate development tasks, guides team members, ensures team punctuality
Involvement	Oversees planning, scope management, coordination, and validation of system requirements
Deliverables	K:GPLPCMS SRS, System Profile, Project Documentation, Functional system, Test reports
Comments / Issues	

Representative	De Guzman, Asher Nathaniel E. Macaraeg, Mark Cyrus P.
Description	Project Team Member
Type	Requirement Analyst, System Developer
Responsibilities	Develop backend and frontend components, implement coding execution engine, implement RBAC and dashboard systems, conduct integration testing
Involvement	Design, implementation, debugging, documentation

Deliverables	K:GPLPCMS SRS, System Profile, Project Documentation, Functional system, Test reports
Comments / Issues	

3 The Overall Description

3.1 Product Perspective

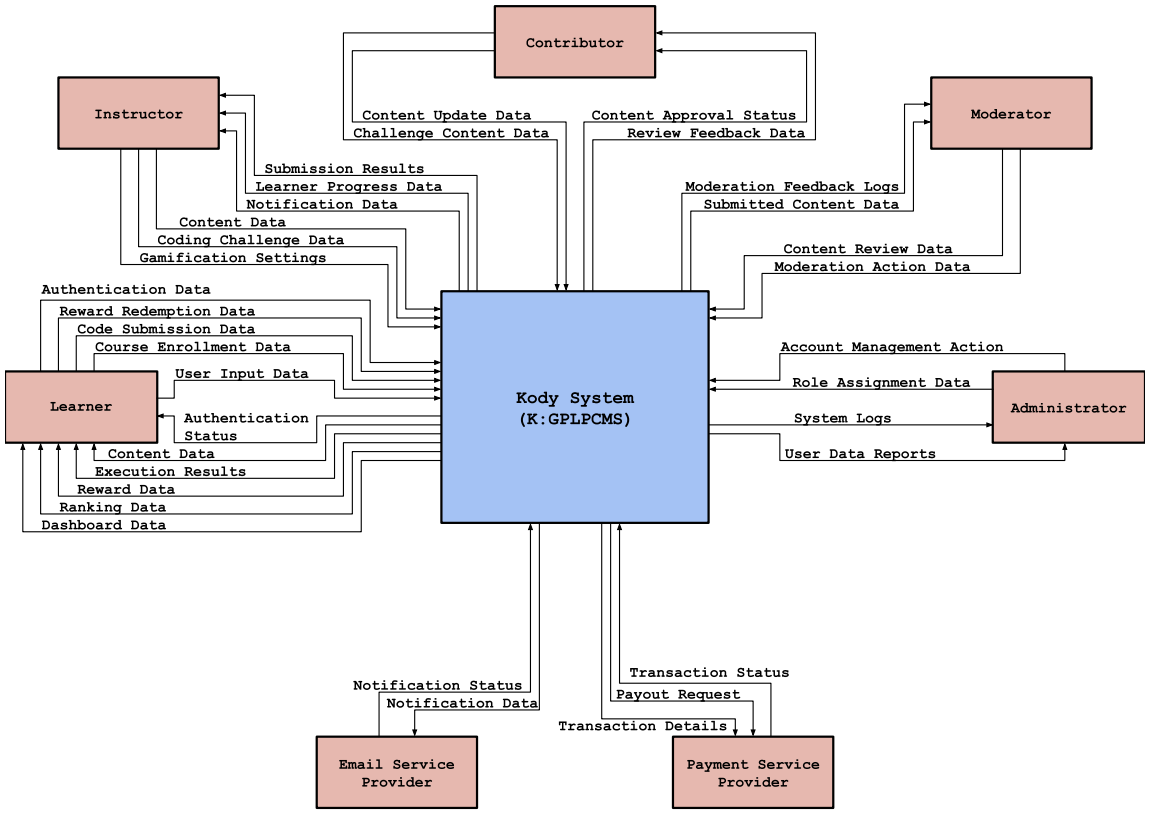


Diagram 1.0 K:GPLPCMS - Context Diagram

Kody is a web-based gamified programming learning platform deployed in a cloud-hosted environment and accessible through standard web browsers. It operates as a standalone system that integrates structured learning, coding practice, and user progression within a unified platform.

The system is designed as a modular application composed of four primary internal components:

1. **User Management Module** – Manages user registration, authentication, and role-based access control for Learners, Instructors, Contributors, Moderators, and Administrators

2. **Content Management Module** – Supports the creation, organization, and delivery of courses, modules, and coding challenges using structured content formats
3. **Coding Execution Module** – Processes user-submitted code and validates outputs against predefined test cases for supported programming languages (Python, Java, C++)
4. **Gamification Module** – Manages experience points (XP), rank progression, reward tokens, and leaderboard rankings based on deterministic activity outcomes

The system interacts with external services through defined interfaces:

- Authentication Service (OAuth provider) for secure user login.
- Third-Party Code Execution APIs for controlled runtime validation.
- Email Services for notifications and system alerts.
- Cloud Hosting Infrastructure for deployment and scalability.
- Payment Service provider to handle monetized content revenue.

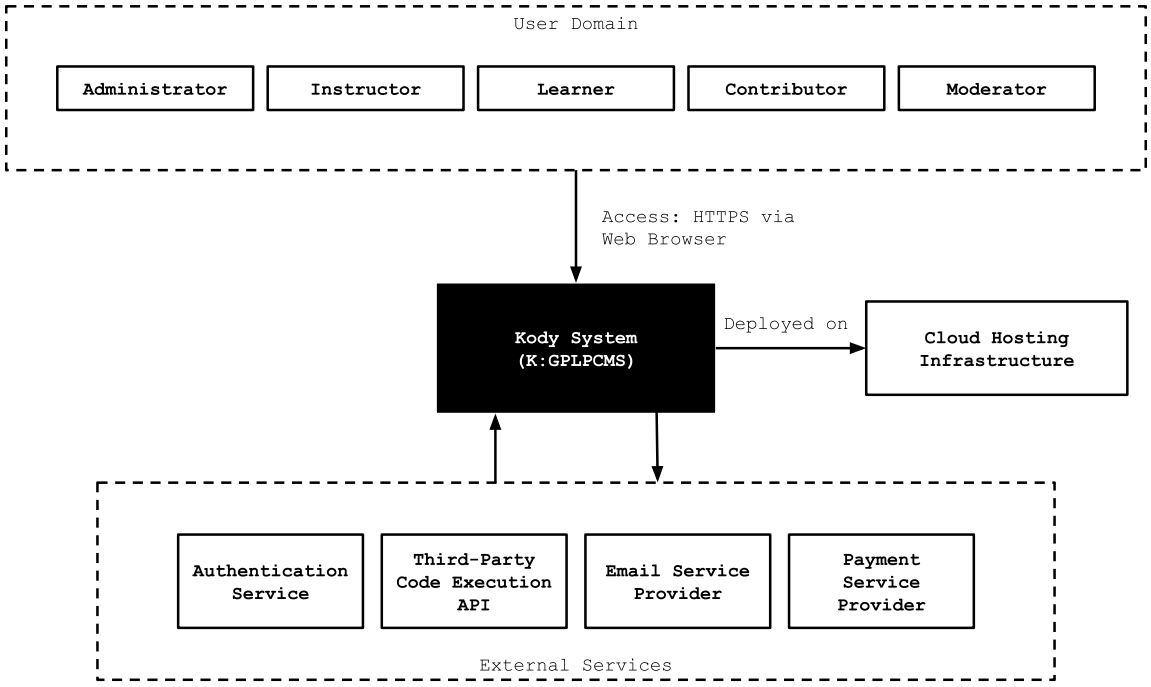


Diagram 1.1 K:GPLPCMS – Block Diagram

Kody operates as a standalone platform and does not replace institutional Learning Management Systems. It provides a unified interface that integrates coding practice, structured content, and gamified engagement within a single environment.

3.1.a System Interfaces

Kody will interface with the following external systems:

Authentication Service

The system shall integrate with **Google Identity Platform (OAuth 2.0)** for user authentication.

- Protocol: OAuth 2.0 / OpenID Connect
- Data retrieved: user email, display name, unique identifier
- Authentication handled via Google Sign-In API (free tier)
- Role assignment is managed internally within Kody

Code Execution API

The system shall integrate with **Judge0 API** (hosted via RapidAPI) for secure code execution.

Supported languages: Python, Java, C++, JavaScript, PHP

Inputs:

- Source code
- Language ID
- Test cases

Outputs:

- Execution output
- Status (success, compilation error, runtime error)
- Execution time and memory usage

For non-executable languages (e.g., HTML, SQL), validation shall be handled internally using predefined rules.

Email Service Provider

The system shall integrate with **SendGrid Email API** (Free to Essentials tier) for transactional email delivery.

Use cases:

- Account verification
- Role approval notifications
- System alerts and announcements

Protocol: REST API over HTTPS

Payment Service Provider

The system shall integrate with **Xendit API** (standard merchant account) for handling token (KodeBit) purchases and monetized content transactions.

Supported methods:

- E-wallets (GCash, Maya)
- Bank transfers
- Virtual accounts

Inputs:

- Payment request data (amount, transaction ID, user reference)

Outputs:

- Transaction status (pending, success, failed)
- Payment confirmation

Integration:

- REST API over HTTPS
- Webhook support for asynchronous payment status updates

Environment:

- Sandbox mode for development and testing
- Production mode for deployment

Cloud Hosting Environment

The system shall be deployed on **Amazon Web Services (AWS)** using the following configuration:

Compute: AWS EC2 (t3.small — 2 vCPU, 2GB RAM)

Database: AWS RDS (db.t3.micro, MySQL/PostgreSQL)

Storage: AWS S3 for static assets

Scalability: Vertical scaling (initial), optional horizontal scaling

3.1.b User Interfaces

Kody provides a web-based graphical user interface accessible via modern desktop and mobile browsers.

The interface shall:

- Provide role-specific dashboards for all user roles
- Support structured navigation for courses, modules, and challenges
- Display separate views for academic progress and gamification metrics
- Provide a coding editor with syntax highlighting and controlled submission
- Support responsive design for cross-device compatibility

The interface shall comply with WCAG 2.1 Level AA accessibility standards.

The system shall not provide a command-line interface.

3.1.c Hardware Interfaces

The system has no direct hardware interface requirements. Users access Kody through standard computing devices with internet connectivity.

3.1.d Software Interfaces

The system operates within modern web browsers supporting: HTML5, CSS3, JavaScript (ES6+)

Supported platforms:

- Desktop browsers (Chrome, Edge, Firefox)
- Mobile browsers (Android/iOS)

3.1.e Communications Interfaces

All communication between client and server shall occur over HTTPS using TLS 1.3 encryption.
The system shall use:

- RESTful APIs for interaction with authentication, code execution, email, and payment services
- SMTP/HTTPS-based email transmission via the email service provider

For near real-time updates (such as leaderboard updates), the system may use:

- WebSocket connections or
- Polling mechanisms

3.1.f Memory Constraints

The system shall operate within the constraints of the selected cloud infrastructure:

- Application Server: AWS EC2 t3.small (2GB RAM)
- Database Instance: AWS RDS db.t3.micro (1GB RAM)

Memory usage shall be optimized to:

- Support concurrent user sessions without degradation
- Ensure code execution requests do not exceed allocated resource limits

The system shall rely on external execution services (e.g., Judge0) to offload computational load from the main server.

3.1.g Operations

The system shall operate as a continuously available web platform with a target uptime of at least 99%, excluding scheduled maintenance.
Maintenance:

- Frequency: ≤ once per week
- Duration: ≤ 2 hours
- Users shall be notified in advance

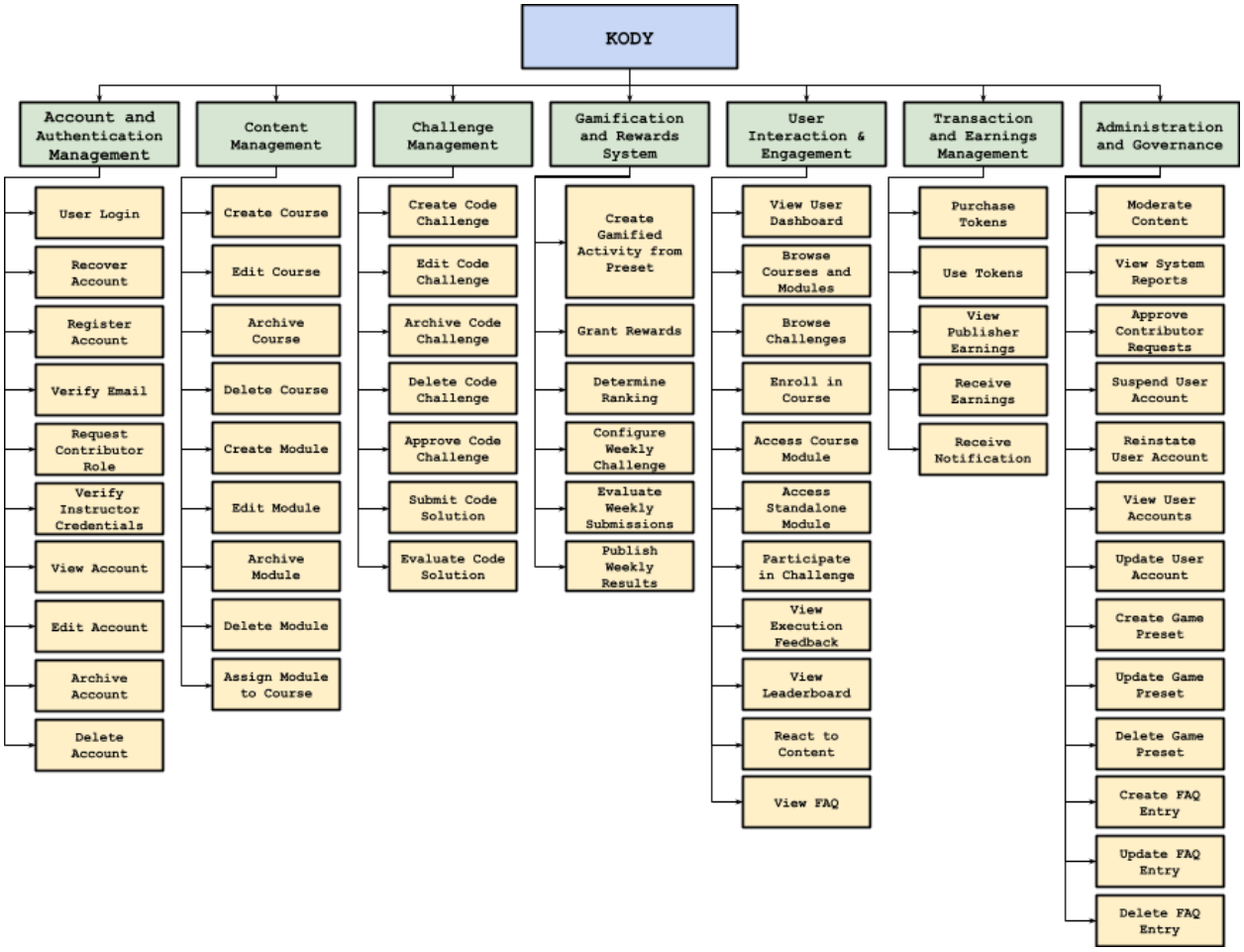
The system shall support concurrent users under normal conditions without degradation of core functionalities.

3.1.h Site Adaptation Requirements

The system requires no physical site adaptation or on-premise installation, as it is fully cloud-hosted.

3.2 Product Functions

Functional Decomposition Diagram



Functionalities Table

The following table summarizes the major system functions that define the operational capabilities of Kody and serve as the basis for the functional requirements specified in Section 4

Functionality	Description
Account and Authentication Management	
User Login	Authenticates users using valid credentials with a maximum of three failed attempts before temporary account lockout.
Recover Account	Allows users to reset their password using a one-time verification code sent via email with retry and cooldown limits.
Register Account	Registers new users as Learner or Instructor with validation of required fields and acceptance of Terms and Conditions.
Verify Email	Validates user email through a verification link before granting system access.

Request Contributor Role	Allows eligible Learners to request Contributor role based on predefined criteria and credential submission.
Verify Instructor Credentials	Enables Moderators to review and approve or reject Instructor applications based on submitted credentials.
View Account	Displays user account information including profile details and account status.
Edit Account	Allows users to update account information, requiring verification for sensitive changes such as email and password.
Archive Account	Enables users to temporarily deactivate their profiles while retaining their uploaded content and data.
Delete Account	Permanently removes a user's data from the system, while safely preserving platform integrity for content they may have published.
Content Management	
Create Course	Allows Instructors to establish structured courses by defining titles, descriptions, and visibility settings.
Edit Course	Enables Instructors to modify course details or remove associated learning modules.
Archive Course	Permits Instructors to hide a course from new enrollees while keeping it accessible to currently enrolled learners.
Delete Course	Permanently removes a course from the platform.
Create Module	Allows Instructors to build individual learning units containing study materials, exercises, and difficulty settings.
Edit Module	Enables Instructors to update or correct module contents and metadata.
Archive Module	Hides a learning module from public view while preserving access for active participants.
Delete Module	Permanently removes a learning module from the platform.
Assign Module to Course	Allows Instructors to link a standalone learning module into a structured course curriculum.
Challenge Management	
Create Code Challenge	Allows Contributors and Instructors to publish coding problems, defining constraints and test cases.

Edit Code Challenge	Enables creators to modify the parameters, descriptions, or hidden test cases of an existing challenge.
Archive Code Challenge	Removes a coding challenge from public access, preventing new submissions.
Delete Code Challenge	Permanently removes a coding challenge from the system.
Review Code Challenge	Enables Moderators to evaluate user-submitted challenges for quality, safety, and accuracy.
Approve Code Challenge	Allows Moderators to publish a reviewed challenge, making it visible to all platform learners.
Submit Code Solution	Allows learners to input and submit their programmed solutions for a specific challenge.
Evaluate Code Solution	Automatically executes submitted code in a secure sandbox, validating it against predefined test cases.
Gamification and Rewards System	
Create Gamified Activity from Preset	Enables content creators to generate interactive learning games using system-defined templates.
Grant Rewards	Automatically distributes Experience Points (XP) and Tokens to users upon successful completion of modules or challenges.
Determine Ranking	Calculates and updates user performance metrics based on execution efficiency, correctness, and speed.
Configure Weekly Challenge	Allows configuration of time-bound challenges with defined rules and reward structures.
Evaluate Weekly Submissions	Processes and validates weekly challenge entries with submission limits and moderation review.
Publish Weekly Results	Releases final rankings and distributes rewards to top-performing users.
User Interaction and Engagement	
View User Dashboard	Serves as the central hub displaying a user's active courses, performance metrics, and relevant notifications.
Browse Courses and Modules	Allows users to search, filter, and view the catalog of available educational content.
Browse Challenges	Enables users to search for coding challenges based on programming language, difficulty, and popularity.
Enroll in Course	Processes a user's entry into a structured course, granting access to its sequential modules.
Access Course Module	Allows enrolled users to open and interact with specific lessons within a course.

Access Standalone Module	Permits users to take independent learning modules not tied to a specific structured course.
Participate in Challenge	Allows users to enter a specific coding problem, potentially deducting a token entry fee if required.
View Execution Feedback	Displays runtime statistics, syntax errors, and validation results generated after a code submission.
View Leaderboard	Shows competitive rankings of users based on their performance in specific challenges or modules.
Rate Content	Enables users who have completed a course or challenge to leave qualitative feedback or ratings.
View FAQ	Provides a searchable repository of frequently asked questions to assist users with platform navigation.
Transaction and Earnings Management	
Purchase Tokens	Allows users to acquire tokens (KB) through an external payment service with transaction validation (₱100 for 105 KB, ₱300 for 330 KB).
Use Tokens	Deducts KodeBits from a user's wallet to grant access to paid modules (50-200 KB), paid courses (400-1500KB) or premium challenges (5-25 KB).
View Publisher Earnings	Displays analytics and accumulated token revenue for Instructors and Contributors.
Receive Earnings	Processes periodic payouts to content creators based on a 65/35 revenue split between creator and platform.
Receive Notifications	Delivers system alerts, purchase receipts, and account update notices via email.
Administration and Governance	
Moderate Content	Allows Administrators and Moderators to review, flag, or take down inappropriate platform material.
Review User Reports	Enables moderation staff to investigate complaints submitted by users regarding content or behavior.
Approve Contributor Requests	Allows Moderators to grant elevated publishing privileges to qualified Learners based on their activity.
Suspend User Account	Temporarily revokes a user's access to the platform due to terms of service violations.
Reinstate User Account	Restores platform access for a previously suspended account.
View User Accounts	Allows administrators to monitor user details, activity logs, and account statuses across the platform.

Edit User Accounts	Enables administrative modification of user profiles and platform permissions.
View System Reports	Generates high-level analytics on platform health, financial transactions, and overall user engagement.
Configure Game Presets	Allows Administrators to define the logic, rules, and UI templates for gamified activities.
Create FAQ Entry	Enables Administrators to add new instructional guides to the platform's help center.
Edit FAQ Entry	Allows Administrators to update existing help documentation.
Delete FAQ Entry	Removes outdated or irrelevant documentation from the platform's help center.

3.3 User Characteristics

User	Description	Functions
Learner	Primary users who access structured learning content and coding challenges; they represent the baseline role from which higher roles inherit permissions.	User Login, Recover Account, Register Account, Verify Email, Request Contributor Role, View Account, Edit Account, Archive Account, Delete Account, Evaluate Code Solution, Grant Rewards, Determine Ranking, View User Dashboard, Browse Courses and Modules, Browse Challenges, Enroll in Course, Access Course Module, Access Standalone Module, Participate in Challenge, View Execution Feedback, View Leaderboard, React to Content, View FAQ, Submit Code Solution, Purchase Tokens, Use Tokens, Receive Notifications
Contributor	Learners with extended privileges who can create and submit coding challenges subject to moderation and approval workflows.	**Inherits all Learner functions;** Create Code Challenge, Edit Code Challenge, Archive Code Challenge, Delete Code Challenge, View Publisher Earnings, Receive Earnings
Instructor	Users with authority to create and manage structured learning content, combining Learner and Contributor capabilities with additional course and module management privileges.	**Inherits all Contributor functions;** Create Course, Edit Course, Archive Course, Delete Course, Create Module, Edit Module, Archive Module, Delete Module, Assign Module to Course, Create Gamified Activity from Preset

Moderator	Users responsible for content validation, role approval, and enforcement of platform rules while also retaining standard user capabilities.	**Inherits all Learner functions;** Verify Instructor Credentials, Configure Weekly Challenge, Evaluate Weekly Submissions, Publish Weekly Results, Approve Code Challenge, Moderate Content, Approve Contributor Requests, Suspend User Account, Reinstate User Account, View User Accounts
Administrator	Users with full system control responsible for governance, reporting, user management, and configuration of system-wide features.	**Inherits all Moderator functions;** View System Reports, Update User Account, Create Game Preset, Update Game Preset, Delete Game Preset, Create FAQ Entry, Update FAQ Entry, Delete FAQ Entry

3.4 Constraints

Regulatory and Data Protection Constraints

- User data shall be handled in accordance with applicable data protection principles (RA 10173 and GDPR)..
- Personally identifiable information (PII) shall be encrypted at rest using AES-256.
- All data in transit shall be secured using TLS 1.3.
- Role-based access control (RBAC) shall restrict access to authorized users only.

Security Constraints

- All system interactions shall occur over HTTPS.
- External service integrations shall use encrypted communication channels.
- Code execution shall be performed in a sandboxed environment.
- Users shall not access or modify other users' data or submissions.
- The system shall enforce request rate limiting to prevent abuse and unauthorized access attempts.
- User sessions shall expire after a defined period of inactivity.

Content and Academic Integrity Constraints

- All user-generated content shall require Moderator approval before publication.
- Content must be original and comply with platform standards.
- Code evaluation shall be isolated per submission to ensure result integrity.

Token (KodeBits) Economy Constraints

- KodeBits rewards shall be granted only after successful validation of user activity.
- KodeBits rewards shall be proportional to task difficulty and performance.
- KodeBits rewards shall be limited to a controlled proportion (≤5%) of total token circulation to prevent inflation and ensure system sustainability.
- Repeated completion of identical content shall not yield additional rewards.
- Free content shall not generate KodeBits rewards.

Payment Integrity Constraints

- KodeBit balances shall be updated only after verified payment confirmation via webhook.
- Pending or failed transactions shall not grant tokens.
- All payment transactions shall be logged for audit and reconciliation.

Reliability Constraints

- The system shall maintain ≥99% uptime, excluding scheduled maintenance.
- Scheduled maintenance shall not exceed 2 hours per occurrence.
- User data and progress shall be stored persistently without loss.
- Execution results shall be consistent under identical inputs and conditions.

Performance Constraints

- Code execution results shall be returned within 3–5 seconds under normal load.
- Dashboard response time shall not exceed 3 seconds.
- Gamification updates shall be reflected within 2 seconds after validation.

Technical Environment Constraints

- The system shall operate as a web-based application.
- The system shall support modern web browsers.
- Supported programming languages in Version 1 are limited to Python, Java, and C++.

Development Constraints

- Development is limited by academic timelines and available resources.

- Version 1 scope shall be restricted to core features (content management, coding validation, gamification, and token-based access).

User Constraints

- Users must have stable internet connectivity.
- Users must use a modern web browser.
- Users are expected to have basic programming knowledge.

Project Constraints

- The system shall not implement full Learning Management System (LMS) functionalities in Version 1.
- Monetization shall be limited to token-based access and basic payment integration.

3.5 Assumptions and Dependencies

Technical Assumptions

- Users access the system through modern web browsers supporting HTML5, CSS3, and JavaScript (ES6+).
- Stable internet connectivity is available for real-time features such as code execution and dashboard updates.
- External services (Google OAuth, Judge0 API, SendGrid, Xendit) are available and provide stable API responses under normal conditions.
- Cloud infrastructure (AWS EC2 and RDS) provides sufficient capacity to handle expected concurrent users in Version 1.

User Assumptions

- Users possess basic programming knowledge (beginner to intermediate level).
- Learners engage with content either independently or through instructor-guided activities.
- Instructors and Contributors are capable of creating valid and structured programming content.

Operational Dependencies

- Authentication depends on successful integration with Google OAuth services.
- Code validation depends on availability and correct configuration of the Judge0 execution service and test cases.
- Email notifications depend on SendGrid service availability and configuration.
- Payment processing and KodeBits allocation depend on Xendit API and webhook confirmation.

- System behavior relies on correct configuration of gamification rules (XP, KodeBits, rank thresholds).

3.6 Apportioning of Requirements.

The implementation of Kody shall be executed in phased development to ensure controlled delivery and alignment with system modules.

Phase 1: Requirements and Governance Definition

1. Finalize system requirements, use cases, and constraints
2. Define role-based access control (RBAC) structure
3. Establish content moderation and validation policies
4. Define KodeBits economy rules (earning, usage, limits)

Phase 2: Interface and Interaction Design

1. Design user interface layouts and navigation flows
2. Define dashboard structure and data presentation
3. Model user interactions for content access, submissions, and rewards
4. Validate usability for target user levels

Phase 3: Core System Development

1. Implement authentication and user management module
2. Implement content and challenge management modules
3. Integrate code execution and validation service (Judge0)
4. Implement gamification system (XP, KodeBits, ranking)
5. Develop user dashboard and interaction features

Phase 4: Integration and Deployment Setup

1. Configure cloud environment (AWS EC2, RDS, S3)
2. Integrate external services (OAuth, execution API, email, payment)
3. Configure administrative controls and system initialization
4. Deploy system for controlled access.

Phase 5: Testing and Validation

1. Validate code execution and evaluation accuracy
2. Verify KodeBits allocation and ranking logic
3. Test role-based access and moderation workflows
4. Conduct usability and performance testing
5. Perform security and access control validation

Phase 6: Maintenance and Enhancement

1. Optimize system performance and scalability
2. Resolve defects and improve system stability
3. Introduce incremental feature enhancements
4. Expand supported programming languages as feasible

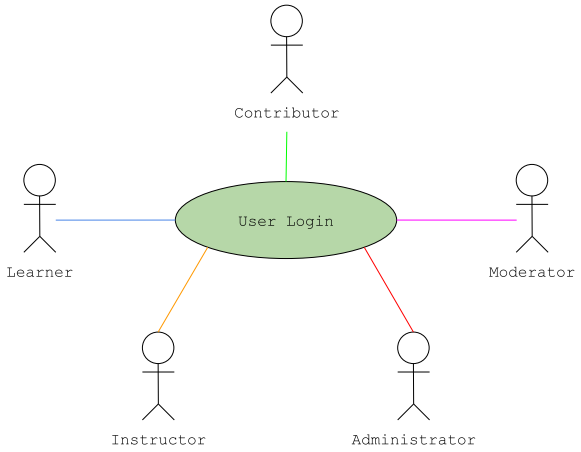
Future enhancements may include more monetization features (e.g. advertisements), institutional integrations, and advanced analytics.

4 Specific Requirements

4.1 Functional Requirements

A. Account and Authentication Management

4.1.a01 User Login

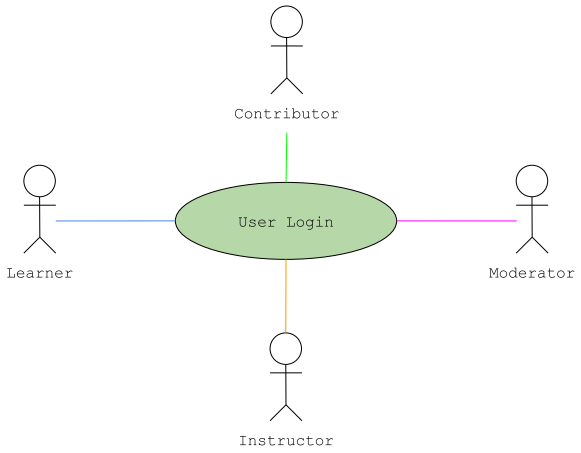


K:GPLPCMS - UC A01	User Login	
Description	This use case allows a registered and verified user to authenticate into the system using valid credentials.	
Actor	Learner, Contributor, Instructor, Moderator, Administrator	
Trigger	User selects "Login".	
Complexity	Easy	
Pre-Condition	• User account exists and is Active (Verified) • User is not currently authenticated	
Post-Condition	• User is authenticated • User is redirected to dashboard	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Enters email and password **Step 3** Submits login form	**Step 1** Displays login form **Step 4** Validates input format **Step 5** Retrieves account by email, verifies password, and checks account status **Step 6** Creates authenticated session and redirects user to dashboard
Alternate Flow	**Invalid Input Format**	

	Step 4a.3 Returns to Step 2	**Step 4a.1** Detect validation error **Step 4a.2** Display field errors
	Incorrect Credentials	
	Step 4b.4 Returns to Step 2 or Recover Account (UC A02)	**Step 4b.1** Detect mismatch **Step 4b.2** Increment failed attempt counter **Step 4b.3** Display "Invalid email or password"
	Account Not Verified	
	Step 5a.3 Returns to Step 2 or or Register Account (A01)	**Step 5a.1** Detect missing account **Step 5a.2** Display "Invalid email or password"
	Email Not Found	
	Step 5b.4 Returns to Step 2 or Recover Account (A02)	**Step 5b.1** Detect unverified status **Step 5b.2** Deny login and display verification message
	Account Suspended or Archived	
	Step 5c.4 Returns to Step 2 or Recover Account (A02)	**Step 5c.1** Detect restricted status **Step 5c.2** Deny login **Step 5c.3** Display access restriction message
	Account Lockout	
	Step 5d.4 May attempt to Recover Account (UC A02)	**Step 5d.1** Detects failed attempts has exceeded threshold **Step 5d.2** Lock account after threshold **Step 5d.3** Display lockout message
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No authentication attempt If interruption occurs during processing → No session created unless authentication completes Upon return → User remains unauthenticated **Invalid Attempts (Incorrect password)** A progressive lockout is initiated upon: 3rd invalid attempt, 15 minutes lockout 6th invalid attempt, 1 hour lockout 9th invalid attempt, 24 hours lockout	
Includes		
Extends		
Assumptions	• The user already has an account	

Business Rules	• Only verified accounts can authenticate • Enforce an initial 15-minute lockout after three failed attempts, with the lockout duration increasing for each subsequent failed attempt.
Related Models / Diagrams	KODY - USE CASE DIAGRAM KODY-ACC-AUTH
Field Validations	KODY DDT-User-Account-Information
Author	dela Cruz, Rupert C.
Date	March 21, 2026 - Created April 17, 2026 - Revised

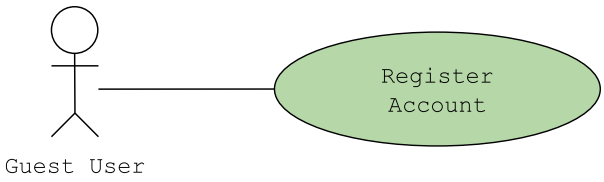
4.1.a02 Recover Account



K:GPLPCMS - UC A02	Recover Account	
Description	This use case allows users to re-enable their account and/or reset their password through a secure, token-based email recovery process.	
Actor	Learner, Contributor, Instructor, Moderator	
Trigger	User selects "Recover Account"	
Complexity	Difficult	
Pre-Condition	• Email service is available • Token generation and validation mechanisms are active	
Post-Condition	• Account is reactivated and/or password is changed • Reset token is invalidated • All active sessions are terminated	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Enter email **Step 3** Submit request **Step 7** Access reset link **Step 10** Enter new password **Step 12** Submit new	**Step 2** Capture input **Step 4** Validate email **Step 5** Generate reset token **Step 6** Send reset email **Step 8** Validate token **Step 9** Display reset form **Step 11** Capture input **Step 13** Validate password

	password	**Step 14** Update password **Step 15** Invalidate token, terminate sessions, and display success message
Alternate Flow	**Invalid Input Format**	
	Step 4a.3 Returns to Step 1	**Step 4a.1** Detect validation error **Step 4a.2** Display error message
	Email Not Found	
	Step 4b.4 Return to Step 1	**Step 4b.1** Can not verify email from registry **Step 4b.2** Display error message
	Invalid or Expired Token	
	Step 8a.2 Return to Step 3	**Step 8a.1** Detect invalid/expired token **Step 8a.2** Display error message
	Invalid Password	
	Step 13a.4 Returns to Step 10	**Step 13a.1** Detect validation error **Step 13a.2** Display requirements
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No reset initiated If interruption occurs during processing → Password is updated only if process completes Upon return → System reflects last completed state	
Includes	UC A04 - Verify Email	
Extends		
Assumptions	• Token is received by the user	
Business Rules	• Reset tokens must be single-use and time-limited • System must not disclose whether an email exists • Reactivating an account requires password change • Password must meet security requirements	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-AUTH	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 21, 2026 - Created April 17, 2026 - Revised	

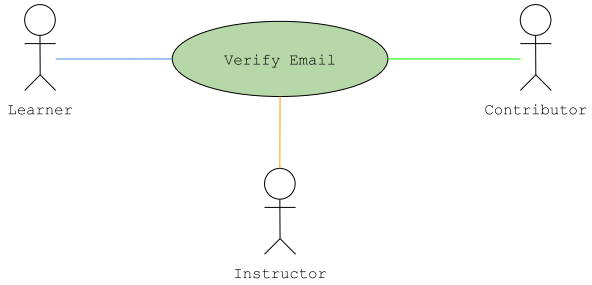
4.1.a03 Register Account



K:GPLPCMS - UC A03	Register Account	
Description	This use case allows a guest user to create a new account by providing required details.	
Actor	Guest User	
Trigger	User selects "Register Here"	
Complexity	Average	
Pre-Condition	• User is not authenticated • Registration interface is accessible	
Post-Condition	• Account is created with Unverified status • Verification email is sent	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Enter registration details **Step 3** Submit registration	**Step 2** Capture input **Step 4** Validate fields **Step 5** Check email uniqueness **Step 6** Create account (Unverified) and generate verification token **Step 8** Send verification email and display confirmation message
Alternate Flow	**Invalid Input**	
	Step 4a.3 Returns to Step 1	**Step 4a.1** Detect validation error **Step 4a.2** Display field errors
	Email Already Registered	
	Step 5a.3 Returns to Step 1 or proceed to User Login (A01)	**Step 5a.1** Detect duplicate email **Step 5a.2** Display error message and prompts user to Login instead
	Instructor Registration Path	
	Step 5b.1 Selected Instructor role **Step 5b.3** Select Instructor role	**Step 5b.2** Display additional fields **Step 5b.4** Store credentials and saves request for approval (G03) **Step 5b.5** Proceed to Step 6
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)**	

	If interruption occurs before submission → No account created If interruption occurs during processing → Account is created only if process completes Upon return → System reflects last completed state
Includes	UC A04 - Verify Email
Extends	UC A06 — Verify Instructor Credentials
Assumptions	• User has an Email and Email service is available
Business Rules	• Email must be unique • Password must meet security requirements (12-32 characters; mix of uppercase/lowercase letters, numbers, and symbols) • Account remains Unverified until email confirmation • Instructor/Contributor roles require additional approval but they can login as Learner users • Registration must follow secure authentication standards
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-AUTH
Field Validations	KODY DDT-User-Account-Information
Author	dela Cruz, Rupert C.
Date	March 21, 2026 - Created April 17, 2026 - Revised

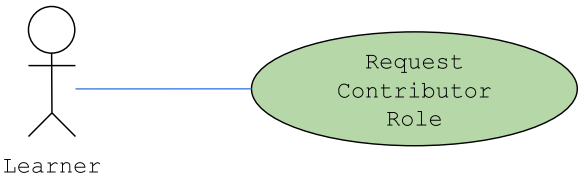
4.1.a04 Verify Email



K:GPLPCMS - UC A04	Verify Email	
Description	This use case allows a user to verify their email upon account registration or account recovery.	
Actor	Learner, Contributor, Instructor (Unverified)	
Trigger	User accessed the verification link	
Complexity	Average	
Pre-Condition	• User account exists with status of Unverified • Registration interface is accessible	
Post-Condition	• Account status is set to Active • Verification token is invalidated • User can now log in	
Basic Flow	**Actor Action**	**System Response**

	Step 1 Step 1 Access verification link	**Step 2** Receive token **Step 3** Validate token **Step 4** Retrieve account **Step 5** Activate account **Step 6** Invalidate token **Step 7** Display success message
Alternate Flow	**Invalid or Expired Token**	
	Step 1a.1 Access invalid/expired link **Step 1a.4** Requests new access link	**Step 1a.2** Detect invalid token **Step 1a.3** Display error message
	Token Already Used	
	Step 1b.1 Access already used token **Step 1b.3** Requests new access link	**Step 1b.2** Detect token reuse **Step 1b.3** Display status message
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before completion → No status change If interruption occurs during processing → Account is activated only if process completes Upon return → System reflects last completed state	
Includes		
Extends		
Assumptions	• Token is securely generated and stored	
Business Rules	• Authentication tokens must include a defined expiration time and automatically become invalid after the expiration period. • Account remains Unverified until successful validation	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-AUTH	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 21, 2026 - Created April 17, 2026 - Revised	

4.1.a05 Request Contributor Role



K:GPLPCMS - UC A05	**Request Contributor Role**
Description	This use case allows an authenticated user to

	request elevation to a contributor role by submitting required information and credentials for administrative review.	
Actor	Learner	
Trigger	User selects "Apply for to be a Contributor"	
Complexity	Average	
Pre-Condition	• User is authenticated (A01) • Account status is Active • User is not Contributor or higher • No existing pending request	
Post-Condition	• Contributor request is created (Pending) • Request is available for review (G03) • User retains current role	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Enter details and upload credentials **Step 4** Submit request	**Step 1** Displays form **Step 3** Capture input **Step 5** Validate submission **Step 6** Create request record and sets status to Pending **Step 7** Notify admins/moderators **Step 8** Display confirmation
Alternate Flow	**File Upload Failure**	
	Step 3a.3 Returns to Step 2	**Step 3a.1** Detects upload failure **Step 3a.2** Display error
	Invalid Submission	
	Step 5a.3 Returns to Step 2	**Step 5a.1** Detects validation error **Step 5a.2** Display field errors
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No request created If interruption occurs during processing → Request is created only if process completes Upon return → System reflects last completed state	
Includes		
Extends		
Assumptions	• Moderation workflow is active	
Business Rules	• Only eligible users can request contributor role (Account is at least 30 days old; completed at least 25 modules / 50 challenges) • Only one pending request per user is allowed	

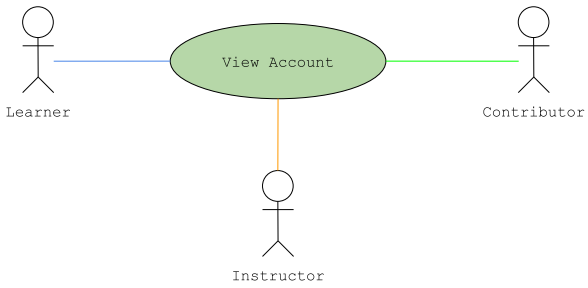
	• Requests require administrative approval
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-ROLE
Field Validations	DDT-Contributor-Request-Information
Author	dela Cruz, Rupert C.
Date	March 21, 2026 - Created April 17, 2026 - Revised

4.1.a06 Verify Instructor Credentials

K:GPLPCMS - UC A06	Verify Instructor Credentials	
Description	This use case processes submitted instructor credentials through automated validation and prepares them for administrative review.	
Actor	Moderator, Administrator	
Trigger	Guest User selects Instructor and finishes Registration (A03) or a Learner edits their profile to Instructor Role (A08)	
Complexity	Average	
Pre-Condition	• User account exists • Credential data has been submitted (A03 or A08)	
Post-Condition	• Credential record is created and stored • Status is set (Pending Review / Rejected / Accepted) • User may be elevated to Instructor Role	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Receive credential data **Step 2** Validate submission and perform automated checks **Step 3** Create credential record, set status and link to user account **Step 4** Present relevant credentials and files for review	
	Step 5 Verifies credentials	
	Step 6 Approve Instructor Role elevation	**Step 7** Applies role elevation to user account and notifies them
Alternate Flow	**Automated Validation Failure**	
		Step 2a.1 Detects invalid data **Step 2a.2** Set account status to Invalid and notifies user

		Step 2a.3 End of use case
	Instructor Role Elevation Rejected	
	Step 6a.1 Rejects role elevation and states the reason	**Step 6a.2** Notifies user **Step 6a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No data processed If interruption occurs during processing → Record is created only if process completes Upon return → System reflects last completed state	
Includes		
Extends		
Assumptions	• Instructor credentials are valid	
Business Rules	• Automated validation (email domain check) does not grant instructor role • Manual validation requires Moderator/Administrator to follow a set of criteria and credibility verification process	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-ROLE	
Field Validations	DDT-Instructor-Verification-Information	
Author	dela Cruz, Rupert C.	
Date	March 21, 2026 - Created April 17, 2026 - Revised	

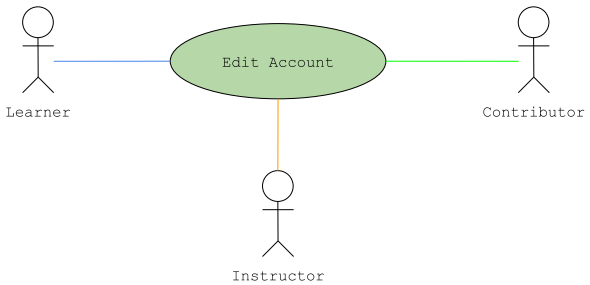
4.1.a07 View Account



K:GPLPCMS - UC A07	View Account
Description	This use case allows an authenticated user to view their account profile.
Actor	Learner, Contributor, Instructor
Trigger	User access account/profile page
Complexity	Easy
Pre-Condition	• User is authenticated (A01) • User account exists
Post-Condition	• Account information is displayed

	• No data is modified	
Basic Flow	**Actor Action**	**System Response**
		Step 1 Verify session **Step 2** Retrieve account data **Step 3** Filter sensitive fields **Step 4** Format data **Step 5** Display profile
Alternate Flow	**Partial Data Available**	
		Step 2a.1 Detects missing fields **Step 2a.2** Display available data **Step 2a.3** Mark missing fields
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before display → No data shown If interruption occurs during display → Partial rendering possible Upon return → Data is reloaded	
Includes		
Extends	UC A08 — Edit Account	
Assumptions	• Data is stored and accessible	
Business Rules	• Users may only view their own account • Sensitive data must not be displayed • Email is partially masked	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-MNGT	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 21, 2026 - Created April 17, 2026 - Revised	

4.1.a08 Edit Account

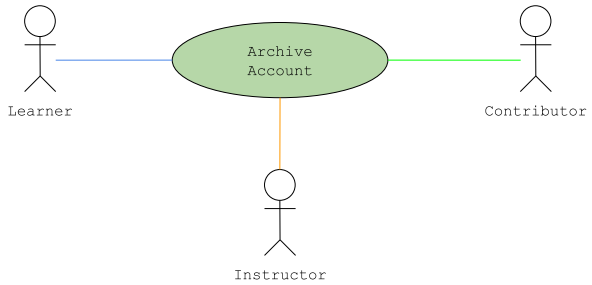


K:GPLPCMS - UC A08	Edit Account
Description	This use case allows an authenticated user to update account information.

Actor	Learner, Contributor, Instructor	
Trigger	User selects "Edit Account".	
Complexity	Average	
Pre-Condition	• User is authenticated (A01) • User account exists • User is accessing own account (A07 context)	
Post-Condition	• Account data is updated and stored • Audit record is created	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Modify fields **Step 3** Submit changes	**Step 1** Retrieve account data and displays editable fields **Step 4** Capture input **Step 5** Validate input and check constraints **Step 7** Apply security checks **Step 8** Update account data record audit **Step 9** Display success message
Alternate Flow	**Sensitive Change (Re-authentication Required)**	
	Step 2a.1 Modify sensitive field **Step 2a.3** Inputs password **Step 2a.6** Proceeds to Step 3	**Step 2a.2** Prompt for password **Step 2a.4** Verify password **Step 2a.5** Allows sensitive change
	Email Change (Re-verification Required)	
	Step 2b.1 Change email **Step 2b.4** User proceeds to UC04	**Step 2b.2** Mark account as Unverified **Step 2b.3** Generate verification token and sends verification email
	Instructor Role Elevation	
	Step 2c.1 Updated role to Instructor **Step 2c.3** Inputs credentials	**Step 2c.2** Prompts use to fill fields required for verification **Step 2c.4** Store credentials and saves request for approval (A06) **Step 2c.5** Proceed to Step 9
	Invalid Input	
	Step 5a.1 Detect sensitive field **Step 5a.3** Returns to Step 2	**Step 5a.2** Display field errors
	Email Already in Use	
		Step 5b.1 Detect duplicate

	Step 5b.3 Returns to Step 2	email in registry **Step 5b.2** Display error message
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No changes applied If interruption occurs during processing → Updates applied only if process completes Upon return → System reflects last completed state	
Includes		
Extends	UC A07 - View Account	
Assumptions	• Data storage supports updates and logging	
Business Rules	• Users may edit only their own account • Sensitive changes require re-authentication • Updates must pass validation before persistence	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-MNGT	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 21, 2026 - Created April 17, 2026 - Revised	

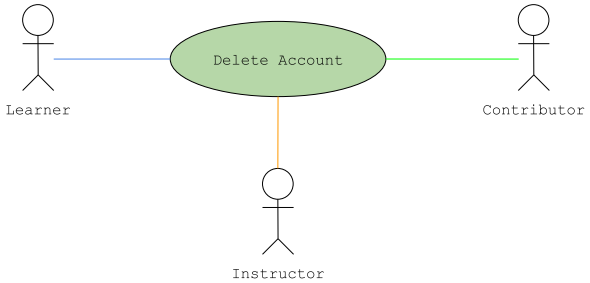
4.1.a09 Archive Account



K:GPLPCMS - UC A09	Archive Account	
Description	This use case allows an authenticated user to archive their account.	
Actor	Learner, Contributor, Instructor	
Trigger	User selects "Archive Account"	
Complexity	Average	
Pre-Condition	• User is authenticated (A01) • Account status is Active	
Post-Condition	• Account status is set to Archived • Sessions are terminated	
Basic Flow	**Actor Action**	**System Response**
		Step 1 Display confirmation

	Step 2 Confirm action **Step 4** Enter password	**Step 3** Prompt for password **Step 5** Verify password **Step 6** Update status to Archived and terminates session **Step 7** Display success message
Alternate Flow	**Cancel Action**	
	Step 2a.1 Cancel request	**Step 2a.2** Abort operation **Step 2a.3** Return to account view
	Invalid Password	
	Step 5a.3 Returns to Step 4	**Step 5a.1** Detect password mismatch **Step 5a.2** Display error
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No changes applied If interruption occurs during processing → Account archived only if process completes Upon return → System reflects last completed state	
Includes		
Extends	UC A07 - View Account	
Assumptions		
Business Rules	• Archived accounts cannot log in (A01) • Archived accounts may be reactivated through Recover Account (A02) • User data must remain intact	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-MNGT	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 21, 2026 - Created April 17, 2026 - Revised	

4.1.a10. Delete Account



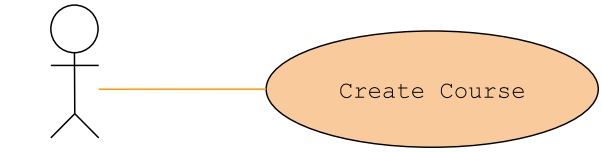
K:GPLPCMS - UC A10	**Delete Account**
Description	This use case allows an authenticated user to

	permanently delete or anonymize their account and associated data.	
Actor	Learner, Contributor, Instructor	
Trigger	User selects "Delete Account"	
Complexity	Difficult	
Pre-Condition	• User is authenticated (A01) • Account status is Active or Archived	
Post-Condition	• Account is deleted or anonymized • All sessions are terminated • User cannot access or recover account	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Enter confirmation phrase **Step 3** Confirm action **Step 6** Enter password	**Step 1** Display warning **Step 4** Validate confirmation **Step 5** Prompt for password **Step 7** Verify password **Step 8** Execute deletion/anonymization and terminate sessions **Step 9** Display success message
Alternate Flow	**Cancel Deletion**	
	Step 1a.1 Cancel action	**Step 1a.2** Abort operation **Step 1a.3** Return to account view
	Invalid Password	
	Step 7a.3 Return to Step 6	**Step 7a.1** Detect mismatch **Step 7a.2** Display error
	Invalid Confirmation Phrase	
	Step 4a.3 Return to Step 2	**Step 4a.1** Detects invalid confirmation **Step 4a.2** Reject confirmation and display error
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No changes applied If interruption occurs during processing → Deletion applied only if process completes Upon return → System reflects final state	
Includes		
Extends	UC A07 - View Account	
Assumptions	• System supports deletion or anonymization	
Business Rules	• Requires explicit confirmation and re-authentication	

	• All personal data must be removed or anonymized • System may retain non-identifiable data
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-MNGT
Field Validations	KODY DDT-User-Account-Information
Author	dela Cruz, Rupert C.
Date	March 21 2026 - Created April 17, 2026 - Revised

B. Learning Content Management

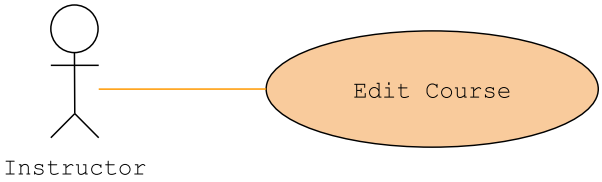
4.1.b01. Create Course



K:GPLPCMS - UC B01	Create Course	
Description	This use case allows an instructor to create a new course by defining its metadata and configuration.	
Actor	Instructor	
Trigger	User selects "Create Course"	
Complexity	Average	
Pre-Condition	• User is authenticated • User has Instructor role (G03)	
Post-Condition	• Course is created with status set to Draft • Course is available for module assignment (B09)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Enter course details **Step 3** Define configuration **Step 5** Submit course	**Step 2** Capture input **Step 4** Capture configuration **Step 6** Validate fields and configuration **Step 7** Create course (Draft) and store course data **Step 8** Display confirmation
Alternate Flow	**Missing Required Fields / Invalid Configuration**	
	Step 6a.3 Returns to Step 1	**Step 6a.1** Detects missing fields or inconsistency **Step 6a.2** Display field errors and validation message
	Cancel Creation	
	Step 5a.1 Cancel creation	**Step 5a.2** Discard input

		Step 5a.3 End of Use Case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No course created If interruption occurs during processing → Course created only if process completes Upon return → System reflects last completed state	
Includes		
Extends		
Assumptions		
Business Rules	• Only approved Instructors can create courses • Course must include required metadata • Course status must default to Draft and may be published through Edit Course (B02)	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT	
Field Validations	DDT-Course-Information	
Author	Macaraeg, Mark Cyrus P.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

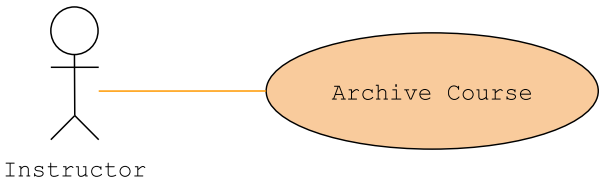
4.1.b02. Edit Course



K:GPLPCMS - UC B02	Edit Course	
Description	This use case allows an instructor to modify course metadata and configuration while preserving course integrity and enrolled user conditions.	
Actor	Instructor	
Trigger	User selects "Edit Course"	
Complexity	Difficult	
Pre-Condition	• User has Instructor role • Course exists and is not Deleted	
Post-Condition	• Course is updated and is published (if selected)	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Modify course **Step 4** Submit updates	**Step 1** Retrieve course data and displays editable fields **Step 3** Capture changes **Step 5** Validate fields **Step 6** Validate configuration and

		structure **Step 7** Save updates, record modification **Step 8** display confirmation
Alternate Flow	**Concurrent Edit Conflict**	
	Step 2a.3 Confirms reload or invoke Cancel Edit	**Step 2a.1** Detects multiple accounts editing **Step 2a.2** Prevent overwrite and prompts reload or merge **Step 2a.4** Return to Step 1
	Cancel Edit	
	Step 4a.1 Cancels edit	**Step 4a.2** Discards changes **Step 4a.3** End of use case
	Validation Failure / Invalid Configuration	
	Step 5a.3 Return to Step 2	**Step 5a.1** Detect validation error or inconsistency **Step 5a.2** Display field errors or conflict message
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No changes applied If interruption occurs during processing → Updates applied only if process completes Upon return → System reflects last completed state	
Includes		
Extends	UC B01 - Create Course	
Assumptions		
Business Rules	• Changes must not invalidate enrolled users' access • Structural integrity with assigned modules must be preserved • All modifications must be logged	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT	
Field Validations	DDT-Course-Information	
Author	Macaraeg, Mark Cyrus P.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

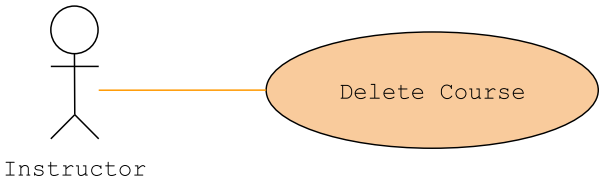
4.1.b03. Archive Course



K:GPLPCMS - UC B03	Archive Course	
Description	This use case allows an instructor to archive a course, making it unavailable for new enrollment while preserving its structure and data.	
Actor	Contributor, Instructor	
Trigger	User selects on "Archive Course"	
Complexity	Easy	
Pre-Condition	• User has Contributor or Instructor role • Course exists and status is Active • User has permission to manage the course	
Post-Condition	• Course status is Archived • Course is removed from public listings and enrollment • Course data and relationships are preserved	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm action	**Step 1** Display confirmation **Step 3** Update status to Archived **Step 4** Remove from listings and enrollment **Step 5** Records action **Step 6** Display confirmation
Alternate Flow	**Cancel Action**	
	Step 2a.1 Cancel archive	**Step 2a.2** Abort operation **Step 2a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation: → No changes applied If interruption occurs during processing: → Status updated only if process completes Upon return: → System reflects last completed state	
Includes		
Extends	UC B01 - Create Course	
Assumptions		
Business Rules	• Only authorized users can archive courses • Only Active courses can be archived • Archived courses must not be visible in public browsing (E02), and enrollment (E04) • Archived courses must not delete modules,learner progress, and analytics data • Access policy must be defined: **Policy A — Retain Access:** enrolled users retain	

	access and progress continues **Policy B — Lock Access:** enrolled users cannot access content • Archiving actions must be logged (actor, timestamp)
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT
Field Validations	DDT-Course-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 22, 2026 - Created April 18, 2026 - Revised

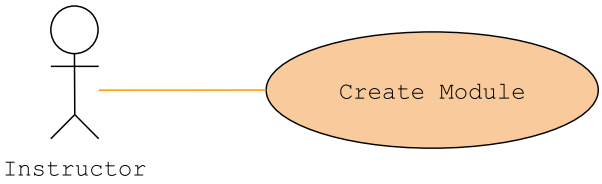
4.1.b04. Delete Course



K:GPLPCMS - UC B04	Delete Course	
Description	This use case allows an instructor to permanently delete a course when no active dependencies exist.	
Actor	Instructor	
Trigger	User selects "Deletes Course"	
Complexity	Easy	
Pre-Condition	• User has Instructor role • Course exists and is not Deleted • Course has no active dependencies: enrolled users (E04), learner progress records, rewards/leaderboard data (D02, D03)	
Post-Condition	• Course is permanently deleted • Deletion is recorded	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm action	**Step 1** Displays confirmation warning **Step 3** Check dependencies **Step 4** Delete course data **Step 5** Record deletion **Step 6** Display confirmation
Alternate Flow	**Active Dependencies**	
	Step 3a.4 Cancel action or Archive (B03)	**Step 3a.1** Detects dependency/ies **Step 3a.2** Blocks deletion **Step 3a.3** Display reason and suggest archive (B03)
	Cancel Deletion	

	Step 2a.1 Cancel action	**Step 2a.2** Abort operation **Step 2a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No changes applied If interruption occurs during processing → Deletion applied only if process completes Upon return → System reflects final state	
Includes		
Extends	UC B01 - Create Course	
Assumptions		
Business Rules	• Deletion is allowed only when no dependencies exist • Deletion must remove all course-level data • Deletion must be logged • Users must receive explicit irreversible warning	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT	
Field Validations	DDT-Course-Information	
Author	Macaraeg, Mark Cyrus P.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

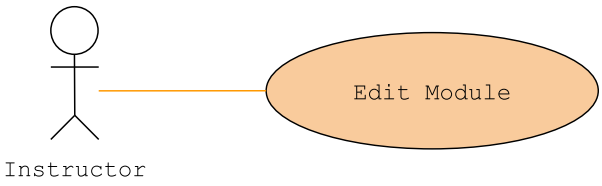
4.1.b05. Create Module



K:GPLPCMS - UC B05	Create Module	
Description	This use case allows an instructor to create a new learning module by defining its content, structure, and metadata.	
Actor	Instructor	
Trigger	User selects "Create Module"	
Complexity	Average	
Pre-Condition	• User is authenticated • User has Contributor or Instructor role (G03)	
Post-Condition	• Module is created with status set to Draft • Module is available for editing and assignment (B09)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Enter module details **Step 3** Define	**Step 2** Capture input **Step 4** Capture structure

	structure/content **Step 5** Submit module	**Step 6** Validate fields and format **Step 7** Create module (Draft) and store module data **Step 9** Display confirmation
Alternate Flow	**Missing Required Fields / Invalid Content Format**	
	Step 6a.3 Return to Step 1	**Step 6a.1** Detect missing fields or invalid format **Step 6a.2** Display field errors
	Cancel Creation	
	Step 5a.1 Cancel creation	**Step 5a.2** Discard input **Step 5a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No module created If interruption occurs during processing → Module created only if process completes Upon return → System reflects last completed state	
Includes		
Extends		
Assumptions	• Editor supports text, code, and media	
Business Rules	• Module must include required content • Module is not accessible until assigned (B09) / published (B06)	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT	
Field Validations	DDT-Module-Information	
Author	Macaraeg, Mark Cyrus P.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

4.1.b06. Edit Module

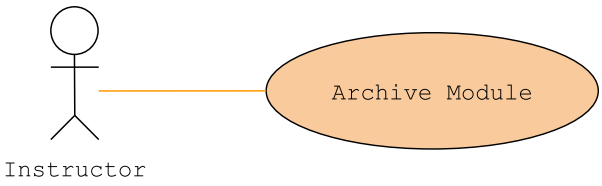


K:GPLPCMS - UC B06	Edit Module
Description	This use case allows an instructor to modify an existing module's content, structure, and metadata while preserving data integrity.
Actor	Instructor

Trigger	User selects "Edit Module"	
Complexity	Difficult	
Pre-Condition	• User has Instructor role • Module exists and is not Deleted	
Post-Condition	• Module is published (if selected) or updated • Modification record is created	
Basic Flow	**Actor Action**	**System Response**
	Step 3 Modify module **Step 4** Submit updates	**Step 1** Retrieve module data **Step 2** Display editable fields **Step 5** Capture changes **Step 6** Validate fields **Step 7** Save updates and records modification **Step 8** Display confirmation
Alternate Flow	**Validation Failure**	
	Step 6a.3 Return to Step 3	**Step 6a.1** Detect validation error **Step 6a.2** Display field errors
	Concurrent Edit Conflict	
	Step 2a.4 Confirm reload or invoke Cancel Edit	**Step 2a.1** Detect multiple account editing **Step 2a.2** Prevent overwrite **Step 2a.3** Prompt reload or merge **Step 2a.5** Return to Step 1
	Cancel Edit	
	Step 4a.1 Cancel edit	**Step 4a.2** Discard changes **Step 4a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No changes applied If interruption occurs during processing → Updates applied only if process completes Upon return → System reflects last completed state	
Includes		
Extends	UC B05 - Create Module	
Assumptions	• Module may be linked to courses (B09)	
Business Rules	• System must record modification (timestamp, editor) • Updates must not break course integrity if module is in use	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT	

Field Validations	DDT-Module-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 22, 2026 - Created April 18, 2026 - Revised

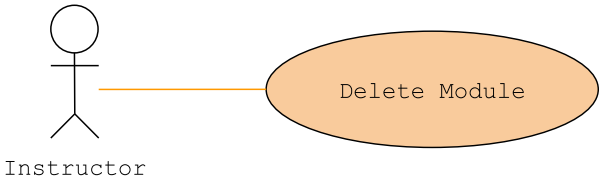
4.1.b07. Archive Module



K:GPLPCMS - UC B07	Archive Module	
Description	This use case allows an instructor to archive a module.	
Actor	Instructor	
Trigger	User selects "Archive Module"	
Complexity	Easy	
Pre-Condition	• User has Instructor role • Module exists and status is Active • User has permission to manage the module	
Post-Condition	• Module status is set to Archived • Module is not accessible to learners • Module cannot be assigned to courses (B09) • Existing references are preserved	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm action	**Step 1** Display confirmation **Step 3** Update status to Archived **Step 4** Remove from active listings **Step 5** Record action **Step 6** Display confirmation
Alternate Flow	**Cancel Action**	
	Step 2a.1 Cancel archive	**Step 2a.2** Abort operation **Step 2a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No changes applied If interruption occurs during processing → Status updated only if process completes Upon return → System reflects last completed state	
Includes		
Extends	UC B05 - Create Module	
Assumptions		

Business Rules	• Only Active modules can be archived • Archived modules must not be visible to learners • Action must be logged (actor, timestamp)
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT
Field Validations	DDT-Module-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 22, 2026 - Created April 18, 2026 - Revised

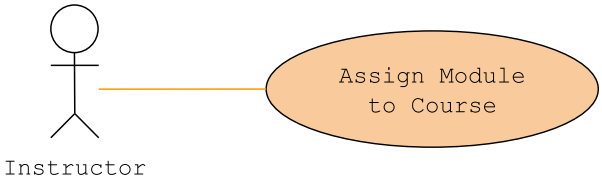
4.1.b08. Delete Module



K:GPLPCMS - UC B08	Delete Module	
Description	This use case allows an instructor to permanently delete a module when no active dependencies exist.	
Actor	Instructor	
Trigger	Confirm deletion action	
Complexity	Easy	
Pre-Condition	• User has Instructor role • Module exists and is not Deleted • Module has no active dependencies (course, learner progress, challenge)	
Post-Condition	• Module and associated data is deleted • Deletion is recorded	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm action	**Step 1** Display warning **Step 3** Check dependencies **Step 4** Delete module and data **Step 5** Record deletion **Step 6** Display confirmation
Alternate Flow	**Dependency Exists**	
	Step 3a.4 Cancels action or Archive (B07)	**Step 3a.1** Detect dependency/ies **Step 3a.2** Block deletion **Step 3a.3** Display reason and suggest archive (B07)
	Cancel Deletion	
	Step 2a.1 Cancel action	**Step 2a.2** Abort operation **Step 2a.3** End of use case

Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No changes applied If interruption occurs during processing → Deletion applied only if process completes Upon return → System reflects final state
Includes	
Extends	UC B05 - Create Module
Assumptions	
Business Rules	• Deletion allowed only when no dependencies exist • System must check course, learner progress, and challenges • Deletion must be logged (actor, timestamp, module ID) • Users must receive explicit warning before deletion
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT
Field Validations	DDT-Module-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 22, 2026 - Created April 18, 2026 - Revised

4.1.b09. Assign Module to Course

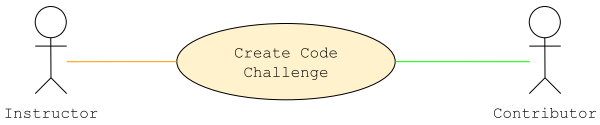


K:GPLPCMS - UC B09	**Assign Module to Course**	
Description	This use case allows an instructor to assign modules to a course and define their sequence, establishing the course structure.	
Actor	Instructor	
Trigger	User selects "Assign Module to Course"	
Complexity	Average	
Pre-Condition	• User has Instructor role • Course exists and is not Deleted	
Post-Condition	• Module(s) are linked to the course • Changes are reflected in access logic (E05)	
Basic Flow	**Actor Action**	**System Response**

	Step 2 Define sequence **Step 4** Submit assignment	**Step 1** Retrieve available modules **Step 3** Capture ordering **Step 5** Validate modules and structure **Step 6** Link modules to course and update course structure **Step 7** Display confirmation
Alternate Flow	**Invalid Module State / Invalid Sequence**	
	Step 4a.1 Selects invalid module or defines conflicting order **Step 4a.5** Return to Step 2	**Step 4a.2** Detect invalid status or conflict **Step 4a.3** Rejects selection and prevents saving **Step 4a.4** Prompt for correction
	Cancel Assignment	
	Step 4a.1 Cancel operation	**Step 4a.2** Abort change **Step 4a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No changes applied If interruption occurs during processing → Changes applied only if process completes Upon return → System reflects last completed state	
Includes		
Extends		
Assumptions		
Business Rules	• Duplicate assignments are not allowed • Each module must have a unique position within a course • Changes must reflect in learner access logic (E05) • Assignment must not break existing learner progress	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-MODULE-MNGT	
Field Validations	DDT-Module-Information	
Author	Macaraeg, Mark Cyrus P.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

C. Code Challenge Management

4.1.c01. Create Code Challenge



K:GPLPCMS - UC C01	Create Code Challenge	
Description	This use case allows a contributor or instructor to create a code challenge by defining its problem, test cases, and execution parameters.	
Actor	Contributor, Instructor	
Trigger	User selects "Create Code Challenge"	
Complexity	Average	
Pre-Condition	• User has Contributor or Instructor role (G03) • Challenge management interface is accessible • Test case system is available	
Post-Condition	• Challenge is created with status of Draft • Challenge is available for editing (C02) and approval (C06)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Enter problem details **Step 3** Define I/O and constraints **Step 5** Add test cases **Step 7** Configure execution settings **Step 9** Submit challenge	**Step 2** Capture input **Step 4** Capture specifications **Step 6** Validate test cases **Step 8** Capture configuration **Step 10** Validate completeness **Step 11** Create challenge (Draft) **Step 12** Store challenge data **Step 13** Display confirmation
Alternate Flow	**Invalid Test Cases**	
	Step 5a.3 Return to Step 3	**Step 5a.1** Detect inconsistency/ies **Step 5a.2** Display validation error
	Invalid Execution Configuration	
	Step 8a.3 Return to Step 7	**Step 8a.1** Detect invalid configuration **Step 8a.2** Display error messages
	Cancel Creation	
	Step 9a.1 Cancel creation	**Step 9a.2** Discards input and aborts operation **Step 9a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed**	

	tab) If interruption occurs before submission → No challenge created If interruption occurs during processing → Challenge created only if process completes Upon return → System reflects last completed state
Includes	
Extends	
Assumptions	
Business Rules	• Challenge must include title, problem description and at least one test case • Execution constraints must be defined before submission • Challenge is not executable until approved (C06)
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-MNGT
Field Validations	DDT-Challenge-Information
Author	De Guzman Asher, Nathaniel E.
Date	March 23, 2026 - Created April 19, 2026 - Revised

4.1.c02. Edit Code Challenge

K:GPLPCMS - UC C02	Edit Code Challenge	
Description	This use case allows a contributor or instructor to modify a code challenge while preserving evaluation integrity and consistency.	
Actor	Contributor, Instructor	
Trigger	User selects "Edit Code Challenge"	
Complexity	Difficult	
Pre-Condition	• User has Contributor or Instructor role • Challenge exists and is not Deleted • Challenge is not Published or editing is restricted/versioned	
Post-Condition	• Challenge is updated • Changes are stored • Modification is recorded • Evaluation integrity is preserved	
Basic Flow	**Actor Action**	**System Response**

	Step 2 Modify challenge (problem, constraints, metadata) **Step 3** Update test cases and modify execution settings **Step 5** Submit updates	**Step 1** Retrieve challenge data and displays editable fields **Step 4** Capture changes **Step 6** Validate all inputs **Step 7** Save updates and record modification **Step 8** Display confirmation
Alternate Flow	**Validation Failure / Invalid Test Cases / Invalid Execution Configuration**	
	Step 6a.3 Return to Step 2	**Step 6a.1** Detects validation error, invalid test cases, or invalid configuration **Step 6a.2** Display field errors
	Concurrent Edit Conflict	
	Step 2a.3 Reloads or invoke Cancel Edit	**Step 2a.1** Detect multiple account editing **Step 2a.2** Prevent overwrite and prompts reload or merge **Step 2a.4** Return to Step 1
	Cancel Edit	
	Step 5a.1 Cancel edit	**Step 5a.2** Discard changes **Step 5a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No changes applied If interruption occurs during processing → Updates applied only if process completes Upon return → System reflects last completed state	
Includes		
Extends	UC C01 - Create Code Challenge	
Assumptions	• System enforces restriction or versioning for published challenges	
Business Rules	• Edits must not compromise evaluation fairness and reproducibility of results • If challenge is published, edits must be restricted or versioned • All modifications must be logged	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-MNGT	
Field Validations	DDT-Challenge-Information	

Author	De Guzman Asher, Nathaniel E.
Date	March 23, 2026 - Created April 19, 2026 - Revised

4.1.c03. Archive Code Challenge

K:GPLPCMS - UC C03	**Archive Code Challenge**	
Description	This use case allows a contributor or instructor to archive a code challenge, disabling new participation while preserving all historical data.	
Actor	Contributor, Instructor	
Trigger	User selects "Archive Code Challenge"	
Complexity	Easy	
Pre-Condition	• User has Contributor or Instructor role • Challenge exists and status is Active	
Post-Condition	• Challenge status is Archived • Challenge is not visible in listings (E03) • New submissions are disabled (E07)	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm action	**Step 1** Display confirmation **Step 3** Update status to Archived, remove from listings and disable submissions **Step 4** Record action **Step 5** Display confirmation
Alternate Flow	**Cancel Action**	
	Step 2a.1 Cancel archive	**Step 2a.2** Abort operation **Step 2a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No changes applied If interruption occurs during processing → Status updated only if process completes Upon return → System reflects last completed state	
Includes		
Extends	UC C01 - Create Code Challenge	
Assumptions	• Submissions (C07) and evaluations (C08) remain stored • Ranking data (D03) may reference archived challenges	
Business Rules	• Only Active challenges can be archived	

	• Archived challenges must not be visible in listings nor available for new attempts • Archiving must not delete submissions, evaluation results, and ranking data • Archiving actions must be logged (actor, timestamp)
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-MNGT
Field Validations	DDT-Challenge-Information
Author	De Guzman Asher, Nathaniel E.
Date	March 23, 2026 - Created April 19, 2026 - Revised

4.1.c04. Delete Code Challenge

K:GPLPCMS - UC C04	Delete Code Challenge	
Description	This use case allows a contributor or instructor to permanently delete a code challenge when no dependent data exists.	
Actor	Contributor, Instructor	
Trigger	User selects "Delete Challenge"	
Complexity	Easy	
Pre-Condition	• User has Contributor or Instructor role • Challenge exists and is not Deleted • Challenge has no active dependencies: submissions (C07), evaluation records (C08) and/or ranking data (D03)	
Post-Condition	• Challenge is permanently deleted • All allowable challenge data is removed • Referential integrity is preserved • Deletion is recorded	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm action	**Step 1** Display warning **Step 3** Check dependencies **Step 4** Delete challenge data and records deletion **Step 5** Display confirmation
Alternate Flow	**Dependency Exists**	
	Step 3a.4 Cancels action or Archive (C03)	**Step 3a.1** Detect dependency **Step 3a.2** Block deletion **Step 3a.3** Display reason and suggest archive (C03)
	Cancel Deletion	

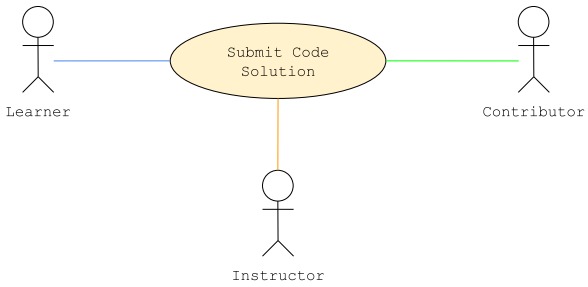
	Step 2a.1 Cancel action	**Step 2a.2** Abort operation **Step 2a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No changes applied If interruption occurs during processing → Deletion applied only if process completes Upon return → System reflects final state	
Includes		
Extends	UC C01 - Create Code Challenge	
Assumptions		
Business Rules	• Deletion allowed only when no dependencies exist • System must check submissions (C07), evaluation records (C08), and/or ranking data (D03) • Deletion must remove all challenge-level data • Deletion must be logged (actor, timestamp, challenge ID)	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-MNGT	
Field Validations	DDT-Challenge-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

4.1.c05. Approve Code Challenge

K:GPLPCMS - UC C05	Approve Code Challenge	
Description	This use case allows a moderator or administrator to approve a reviewed code challenge, making it visible and executable.	
Actor	Moderator, Administrator	
Trigger	Contributor or Instructor created a coding challenge	
Complexity	Average	
Pre-Condition	• Reviewer is authenticated with appropriate privileges • Challenge exists and is not Deleted	
Post-Condition	• Challenge status is set to Approved and Published • Challenge is visible in listings (E03), executable (E07) and is integrated into: evaluation pipeline (C07) and ranking system (D03) • Challenge creator is notified	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Selects unapproved	**Step 2** Display challenge

	coding challenge **Step 3** Verifies and approve challenge **Step 5** Re-confirms approval	details **Step 4** Prompts to re-confirm decision **Step 6** Updates status to Approved, enables visibility, and enables execution **Step 7** Record approval **Step 8** Display confirmation and notifies challenge creator
Alternate Flow	**Cancel Approval**	
	Step 3a.1 Cancel action **Step 3a.3** Return to Step 1	**Step 3a.2** Abort operation
	Concurrent State Conflict	
	Step 3b.4 Reloads or invoke Cancel Approval	**Step 3b.1** Detect state change made by another user **Step 3b.2** Block action **Step 3b.3** Prompt reload or cancel **Step 3b.5** Return to Step 2
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No approval applied If interruption occurs during processing → Approval applied only if process completes Upon return → System reflects final state	
Includes	UC C01 - Create Code Challenge	
Extends		
Assumptions	• Evaluation engine (C08) is available	
Business Rules	• Only Moderators/Admins can approve challenges • Challenge must be valid and complete • Approval must be logged	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-MNGT	
Field Validations	DDT-Challenge-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

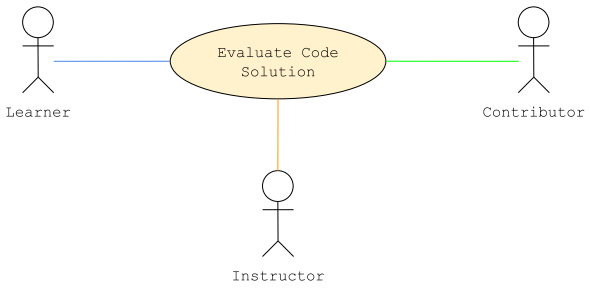
4.1.c06. Submit Code Solution



K:GPLPCMS - UC C06	Submit Code Solution	
Description	This use case allows a user to submit a code solution for a challenge (standard or weekly).	
Actor	Learner, Contributor, Instructor	
Trigger	User selects "Submit Code"	
Complexity	Easy	
Pre-Condition	• User is in challenge execution interface (E07) • Challenge is Approved and Published (C05) • User Submission count is less than 3	
Post-Condition	• Submission is forwarded to evaluation (C07) • Submission is recorded and metadata is stored	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm submission	**Step 1** Display confirmation **Step 3** Receive and validate submission **Step 4** Create submission record and store metadata **Step 5** Forward to evaluation (C07) **Step 6** Display submission status
Alternate Flow	**Invalid Submission / Unsupported Language**	
		Step 2a.1 Detect invalid input **Step 2a.2** Display validation error **Step 2a.3** End of use case
	Cancel Submission	
	Step 2c.1 Cancel submission	**Step 2c.2** Abort operation **Step 2c.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No submission recorded If interruption occurs during processing → Submission recorded only if process completes Upon return → user verifies submission status	
Includes		

Extends	UC C07 — Evaluate Code Solution
Assumptions	
Business Rules	• Submission must be recorded before evaluation • Submission limits are enforced (maximum of 3 submissions) • Weekly submissions must include valid weekly_id and occur within time window • All submissions must be timestamped
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-RESULT
Field Validations	DDT-Submission-Evaluation-Information
Author	De Guzman Asher, Nathaniel E.
Date	March 23, 2026 - Created April 19, 2026 - Revised

4.1.c07. Evaluate Code Solution



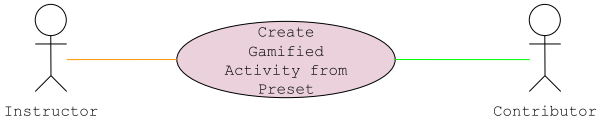
K:GPLPCMS - UC C07	Evaluate Code Solution	
Description	This use case evaluates a submitted code solution by executing it in a sandbox against predefined test cases and generating results and feedback.	
Actor	Learner, Contributor, Instructor, Event Trigger	
Trigger	User clicked "Evaluate Code" or a submission is received (C06)	
Complexity	Average	
Pre-Condition	• Submission exists and is valid (C06) • Test cases are defined and accessible • Execution environment is available	
Post-Condition	• Submission is evaluated against test cases • Results and score are generated • Feedback is stored and available (E08) • Evaluation data is stored for ranking (D03)	
Basic Flow	**Actor Action**	**System Response**
		Step 1 Retrieve submission data **Step 2** Initialize sandbox compile and execute against test cases **Step 3** Capture outputs and metrics and compare with

		expected results **Step 4** Determine results **Step 5** Compute score, store evaluation, and generate feedback
Alternate Flow	**User Have Submission Chance/s**	
	Step 1a.3 Confirm or Cancel evaluation	**Step 1a.1** Detect user has more submission chance **Step 1a.2** Display evaluation confirmation **Step 1a.4** Proceed to Step 2 or abort operation (end use case)
	Compilation/Runtime Error	
		Step 2a.1 Detect compilation/runtime failure **Step 2a.2** Record error and mark result as failed **Step 2a.3** Go to Step 5
	Memory Limit Exceeded	
		Step 2b.1 Detect memory breach **Step 2b.2** Terminate execution and record result **Step 2b.3** Proceed to Step 5
	Output Mismatch	
		Step 3a.1 Detect mismatch **Step 3a.2** Mark test case failed **Step 3a.3** Proceed to Step 5
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If user has remaining submission chance → Evaluation is not run, Otherwise evaluation continues independently Results are retrievable after completion	
Includes	UC - C06 Submit Code Solution	
Extends		
Assumptions	• User is satisfied with their submission	
Business Rules	• Evaluation must be deterministic • All test cases must be executed unless terminated • Time and memory limits must be enforced • Evaluation must be stored before feedback is shown • Only one submission can be evaluated.	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-RESULT	

Field Validations	DDT-Submission-Evaluation-Information
Author	De Guzman Asher, Nathaniel E.
Date	March 23, 2026 - Created April 19, 2026 - Revised

D. Gamification and Rewards System

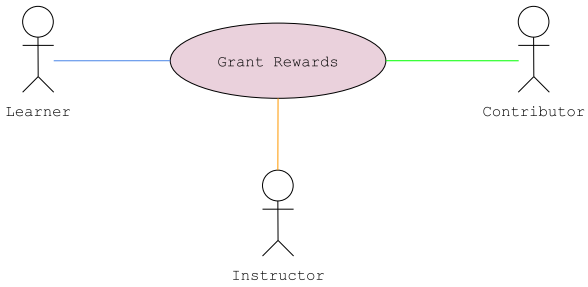
4.1.d01. Create Gamified Activity from Preset



K:GPLPCMS - UC D01	**Generate Game from Preset**	
Description	This use case allows a contributor or instructor to generate a game instance from a predefined preset by supplying required parameters.	
Actor	Contributor, Instructor	
Trigger	User selects "Generate Game"	
Complexity	Difficult	
Pre-Condition	• User has Contributor or Instructor role • At least one valid preset exists (G08 / G09) • Required content (modules/challenges) exists	
Post-Condition	• Game instance is created • Rules, objectives, and rewards are defined • Game is available for participation	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Select preset **Step 4** Provide parameters (scope, duration, content) **Step 6** Confirm generation	**Step 1** Retrieve and display available presets **Step 3** Display preset configuration **Step 5** Validate inputs **Step 7** Construct game structure and assign rules, objectives, rewards **Step 8** Create game instance and store data **Step 9** Display confirmation
Alternate Flow	**Invalid Input**	
	Step 5a.2 Return to Step 4	**Step 5a.1** Detect validation error **Step 5a.2** Prompt correction
	Cancel Generation	
	Step 6a.1 Cancel action	**Step 6a.2** Discard input **Step 6a.3** End of use case
	Memory Limit Exceeded	

	Step 7a.3 Return to Step 4	**Step 7a.1** Detect memory breach **Step 7a.2** Terminate execution, record result and prompt for reconfiguration
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No game created If interruption occurs during processing → Game created only if process completes Upon return → System reflects final state	
Includes		
Extends		
Assumptions	• Games can reference modules, challenges, or both	
Business Rules	• Preset must be valid and predefined • Game must define objectives, rules, reward conditions	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD	
Field Validations	DDT-Gamification-Rewards-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

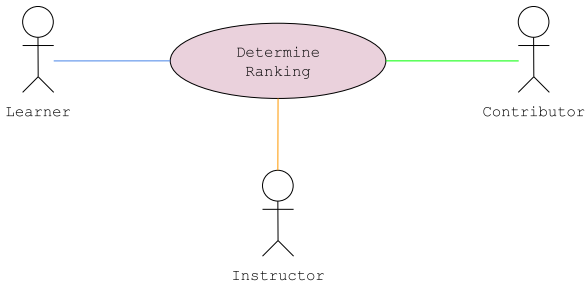
4.1.d02. Grant Rewards



K:GPLPCMS - UC D02	Grant Rewards
Description	This use case assigns rewards to users based on validated outcomes such as challenge evaluation or game completion.
Actor	Event Trigger, Learner, Contributor, Instructor
Trigger	A user evaluates their solution or weekly submission
Complexity	Easy
Pre-Condition	• Outcome is validated (C07 / D05) • User is eligible • Associated entity (challenge/game/module) is valid
Post-Condition	• Reward is assigned to user • User state is updated (points, tokens, badges)

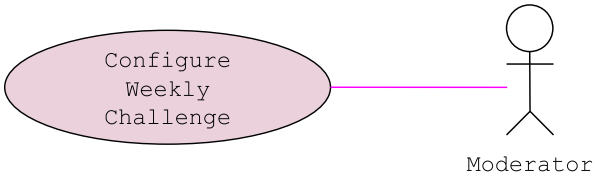
	• Reward transaction is recorded	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Evaluates solution or submission	**Step 2** Identify reward rule **Step 3** Compute reward **Step 4** Check duplicate condition **Step 5** Assign reward **Step 6** Update user metrics **Step 7** Record transaction **Step 8** Trigger notification (optional)
Alternate Flow	**Duplicate Reward**	
		Step 1a.1 Detect prior reward **Step 1a.2** Prevent assignment **Step 1a.2** Log duplicate
Exception Flow	**Processing Failure** • If the system fails to grant reward, no reward is granted. • Error is logged and ends use case. **Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → User may retry process If interruption occurs after confirmation → Process continues in the system	
Includes		
Extends		
Assumptions	• Reward rules are predefined	
Business Rules	• Rewards only granted after validated outcomes • Duplicate rewards must be prevented • Reward logic must follow predefined rules	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD	
Field Validations	DDT-Gamification-Rewards-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

4.1.d03. Determine Ranking



K:GPLPCMS - UC D03	Determine Ranking	
Description	This use case computes and updates user rankings based on validated performance data and predefined scoring rules.	
Actor	Event Trigger, Learner, Contributor, Instructor	
Trigger	Reward update (D02) or scheduled recalculation	
Complexity	Average	
Pre-Condition	• Performance data exists (C07, D05) • Eligible users exist	
Post-Condition	• Rankings are computed and updated • Results are available for viewing (E09)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Update ranking initiated or receives reward	**Step 2** Retrieve eligible users and metrics **Step 3** Apply scoring rules and compute scores **Step 4** Sort users and resolve ties **Step 5** Assign ranks and update leaderboard **Step 6** Store results
Alternate Flow		
Exception Flow	**Computation Failure** • If system fails to compute rankings, no update is applied and previous leaderboard is retained • Error is logged and ends use case.	
Includes		
Extends		
Assumptions	• Rankings are based on quantifiable metrics	
Business Rules	• Tie-breaking rules favors time and efficiency • Ranking must occur after reward assignment (D02) • Leaderboard must reflect latest committed state	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD	
Field Validations	DDT-Gamification-Rewards-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

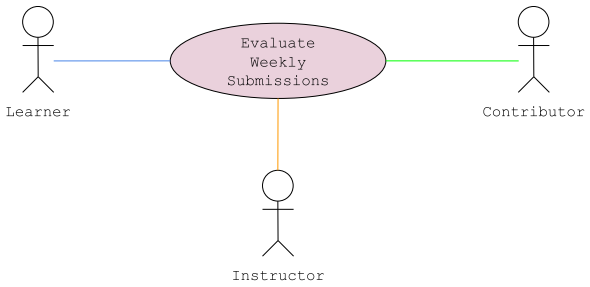
4.1.d04. Configure Weekly Challenge



K:GPLPCMS - UC D04	Configure Weekly Challenge	
Description	This use case allows the system (scheduled event) or a moderator to configure and activate a weekly challenge instance by selecting an existing approved code challenge and defining its schedule, rules, and reward configuration.	
Actor	Moderator, Event Trigger (Scheduled)	
Trigger	Scheduled weekly cycle initiates (system time) or Moderator manually configures weekly challenge	
Complexity	Difficult	
Pre-Condition	• Code challenge exists and is Approved and Published (C06) • No active weekly challenge instance exists	
Post-Condition	• Weekly challenge instance is created with a unique Weekly ID • Selected challenge is linked to the weekly instance • Schedule (start/end time) is registered • All submissions during the active window are tagged with Weekly ID	
Basic Flow	**Actor Action**	**System Response**
	Step 3 Select challenge **Step 5** Define/confirm schedule **Step 7** Define rules and reward configuration **Step 9** Confirm setup	**Step 1** Check for existing weekly challenge instance **Step 2** Retrieve eligible challenges (Approved only) **Step 4** Validate challenge eligibility **Step 6** Validate time window (weekly constraint) **Step 8** Validate configuration **Step 10** Create weekly challenge instance (generate Weekly ID) **Step 11** Link challenge to weekly instance **Step 12** Register schedule using system time and activate weekly challenge **Step 13** Enforce submission tagging rule (Weekly ID binding) **Step 14** Store configuration and audit

		log
Alternate Flow	**Auto-Selection (Event Trigger)**	
		Step 3a.1 selects challenge at random **Step 3a.2** Go to Step 6
	Manual Override by Moderator	
	Step 3b.1 Select specific challenge	Step 3b.2 Override auto-selection Step 3b.3 Continue to Step 4
Exception Flow	**Creation / Activation Failure** System fails during instance creation or activation → No weekly challenge is created → All changes are rolled back → Error is logged → Use case ends **Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No weekly challenge is created If interruption occurs during processing: → Atomic guarantee applies → Either fully created OR not created Upon return → System reflects last committed state	
Includes		
Extends		
Assumptions		
Business Rules	• Weekly challenge must reference an Approved challenge • Only one active weekly challenge may exist at a time • Weekly schedule must follow system-defined cycle (Sundays, midnight or 00:00 AM) • Prior completion must not block participation • Weekly evaluation must be isolated from historical submissions • All configurations must be logged and auditable	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD	
Field Validations	DDT-Gamification-Rewards-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

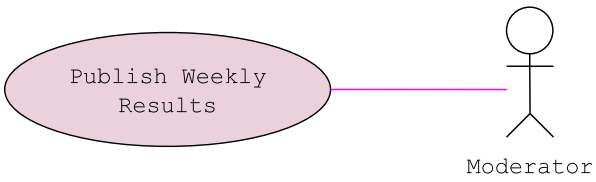
4.1.d05. Evaluate Weekly Submissions



K:GPLPCMS - UC D05	Evaluate Weekly Submissions	
Description	This use case processes and aggregates previously evaluated submissions associated with a completed weekly challenge instance to compute final user scores for reward distribution and ranking.	
Actor	Event Trigger, Learner, Contributor, Instructor	
Trigger	End of weekly challenge (D04), scheduled evaluation, or a user selects "Evaluate Submission"	
Complexity	Average	
Pre-Condition	• Weekly challenge instance exists and has ended (D04) • Submissions exist and are tagged with weekly_id (C06) • Each submission has completed evaluation (C07)	
Post-Condition	• Exactly one final score per user is determined • Aggregated results are stored (scoped to weekly instance) • Results are forwarded to rewards (D02) and ranking (D03)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Trigger evaluation	**Step 2** Retrieve weekly challenge instance **Step 3** Retrieve corresponding evaluation results **Step 4** Evaluate submissions **Step 5** Aggregate per user **Step 6** Apply attempt selection rule **Step 7** Compute final score per user and store results **Step 9** Dispatch to D02 and D03
Alternate Flow		
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → User may retry process If interruption occurs after confirmation → Process continues in the system	

Includes	
Extends	
Assumptions	• Evaluation is deterministic • Batch processing is supported
Business Rules	• Only submissions within the challenge window are evaluated • Each user must have one final score • Multiple attempts follow a predefined selection rule • Invalid submissions are excluded
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD
Field Validations	DDT-Gamification-Rewards-Information
Author	De Guzman Asher, Nathaniel E.
Date	March 23, 2026 - Created April 19, 2026 - Revised

4.1.d06. Publish Weekly Results

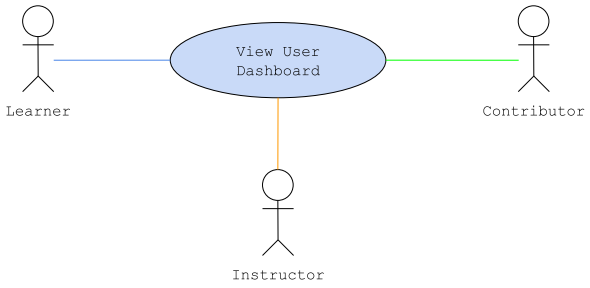


K:GPLPCMS - UC D06	Publish Weekly Results	
Description	This use case publishes finalized weekly challenge results, making rankings visible and triggering downstream actions such as rewards and notifications.	
Actor	Event Trigger, Moderator	
Trigger	Completion of evaluation (D05) and ranking (D03) or invoked by a Moderator	
Complexity	Average	
Pre-Condition	• Evaluation is complete (D05) • Rankings are finalized (D03) • Notification service is available (F05)	
Post-Condition	• Results are published • Leaderboard is visible (E09) • Rewards are distributed (D02)	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Confirm publication of result	**Step 1** Retrieve finalized results and prompt confirmation **Step 3** Publish results and update leaderboard visibility **Step 4** Distribute rewards (D02) **Step 5** Record publication

Alternate Flow	**No Rewards Defined**	
	Step 2a.2 Configures or skip rewards	**Step 2a.1** Detect no reward rules and prompt for configuration **Step 2a.3** Proceed to Step 3
Exception Flow	**Publication Failure** If system fails to publish results: → Error is logged → Notifies Moderator for Manual configuration → Use case ends **Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs during configuration → User must retry from the beginning of the process	
Includes		
Extends		
Assumptions		
Business Rules	• Leaderboard must reflect finalized rankings only • Reward distribution must follow finalized results (D02) • Results must be stored for audit	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD	
Field Validations	DDT-Gamification-Rewards-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

E. User Interaction and Engagement

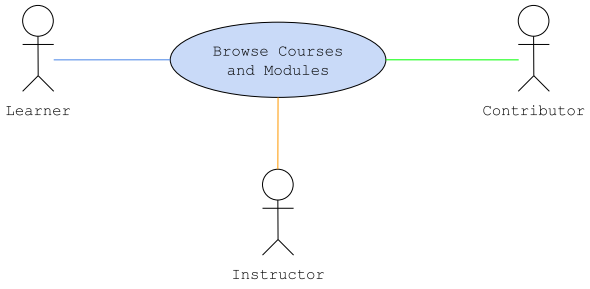
4.1.e01. View User Dashboard



K:GPLPCMS - UC E01	View User Dashboard
Description	This use case displays a consolidated dashboard with user-specific data such as courses, progress, activities, and notifications.
Actor	Learner, Contributor, Instructor
Trigger	User access dashboard (post-login or direct navigation)
Complexity	Easy

Pre-Condition	• User account is Active • Relevant user data exists or is retrievable	
Post-Condition	• Dashboard is displayed	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Open dashboard	**Step 2** Validate session **Step 3** Retrieve profile data **Step 4** Retrieve courses and progress **Step 5** Retrieve recent activities **Step 6** Retrieve notifications **Step 7** Display dashboard
Alternate Flow		
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before render → No data shown During render: → Partial rendering possible → No data modification	
Includes		
Extends		
Assumptions		
Business Rules	• Only user-specific data must be displayed • Dashboard must be read-only	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-DASHBOARD	
Field Validations	DDT-Learning-Progress-Information	
Author	dela Cruz, Rupert C.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

4.1.e02. Browse Courses and Modules

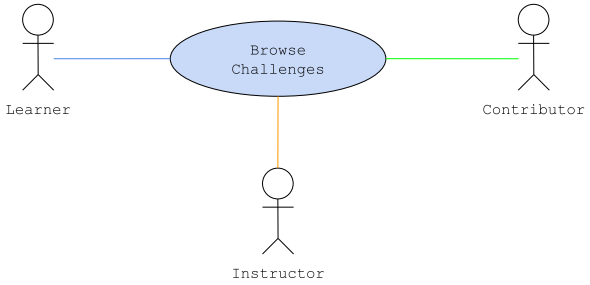


K:GPLPCMS - UC E02	**Browse Courses and Modules**
Description	This use case allows users to browse and discover available courses and standalone modules using search and filters.
Actor	Learner, Contributor, Instructor

Trigger	User selects Courses and Modules	
Complexity	Easy	
Pre-Condition	• Content exists (courses/modules) • Content visibility and status are defined	
Post-Condition	• Filtered content list is displayed • No data is modified	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Load interface and retrieve visible content **Step 2** Apply default filters (Published / Active) and display content list	
	Step 3 Apply search or filters	**Step 4** Validate parameters **Step 5** Retrieve filtered results display updated results
	Step 6 Select content	**Step 7** Navigate to preview/details
Alternate Flow	**No Available Content**	
		Step 1a.1 Detect empty datasetcourses/modules **Step 1a.2** Display "No content available"
	No Matching Results	
	Step 4a.3 Return to Step 3	**Step 4a.1** Detect no matches **Step 4a.2** Display "No results found"
	Restricted Content	
	Step 6a.1 Select restricted content	**Step 6a.2** Display preview or prompt enrollment (E04)
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before display → No data shown During display: → Partial rendering possible → No data modification	
Includes		
Extends		
Assumptions	• There are courses published	
Business Rules	• Only Published and Active content must be visible • Browsing does not grant access to full content • Visibility must respect user permissions	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-COURSE-BROWSE	
Field Validations	DDT-Course-Information DDT-Module-Information	

Author	dela Cruz, Rupert C.
Date	March 22, 2026 - Created April 18, 2026 - Revised

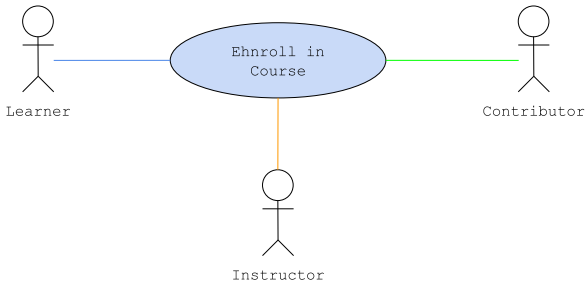
4.1.e03. Browse Challenges



K:GPLPCMS - UC E03	**Browse Challenges**	
Description	This use case allows users to browse approved and published code challenges using search and filtering.	
Actor	Learner, Contributor, Instructor	
Trigger	User selects "Challenges"	
Complexity	Easy	
Pre-Condition	• Challenges exist • Approval workflow enforced (C05 → C06)	
Post-Condition	• Filtered challenge list is displayed	
Basic Flow	**Actor Action**	
		Step 1 Load interface and retrieve eligible challenges **Step 2** Apply default filters (Approved + Published) and display challenge list
	Step 3 Apply search or filters **Step 5** Select challenge	**Step 4** Retrieve and display filtered results **Step 6** Display challenge to preview/details
Alternate Flow	**No Available Challenges**	
		Step 1a.1 Detect empty dataset **Step 1a.2** Display "No challenges available"
	No Matching Results	
	Step 4a.3 Return to Step 3	**Step 4a.1** Detect no search matches **Step 4a.2** Display "No results found"
	Restricted or Locked Challenge	
	Step 3a.1 Select locked challenge	**Step 3a.2** Display preview and requirements

		Step 3a.3 Suggest participation (E07) if eligible
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before display → No data shown During display: → Partial rendering possible → No data modification	
Includes		
Extends		
Assumptions	• There are challenges published	
Business Rules	• Draft, Under Review, Archived, or Deleted challenges must not be displayed • Browsing does not grant execution access • Access requirements (tokens, prerequisites, rank) must be validated only upon selection • Filters must use predefined attributes (difficulty, category, tags)	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-BROWSE	
Field Validations	DDT-Challenge-Information	
Author	dela Cruz, Rupert C.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

4.1.e04. Enroll in Course

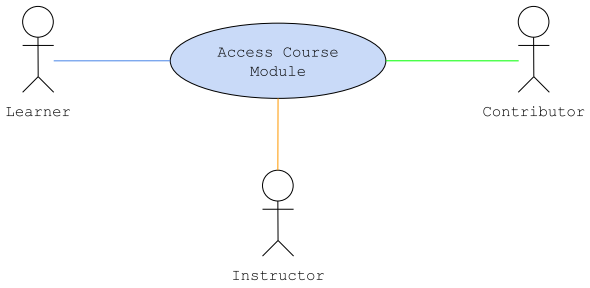


K:GPLPCMS - UC E04	Enroll in Course
Description	This use case allows a user to enroll in a course, granting access to its modules. Enrollment may be free or require tokens and must satisfy eligibility conditions.
Actor	Learner, Contributor, Instructor
Trigger	User is interested in a course
Complexity	Average
Pre-Condition	• Course exists and is Published and Active • User is not already enrolled
Post-Condition	• Enrollment is created • Access to course modules is granted

	• Enrollment is reflected in dashboard (E01) • Transaction is recorded (paid course)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select course **Step 3** Select "Enroll"	**Step 2** Display course details **Step 4** Validate eligibility (status, prerequisites, existing enrollment) **Step 5** Determine enrollment type (free or token-based) **Step 6** Create enrollment record **Step 7** Grant access to modules **Step 8** Update user dashboard data **Step 9** Confirm enrollment
Alternate Flow	**Enrollment Restricted**	
	Step 4a.3 Return to Step 1 or E02	**Step 4a.1** Detect unmet prerequisites/constraints **Step 4a.2** Display requirements
	Paid Enrollment (Token-Based)	
	Step 5a.2 User confirms purchase	**Step 5a.1** Detect token requirement and prompts user **Step 5a.3** Validate token balance and deduct tokens (F02) **Step 5a.4** Continue to Step 6
	Insufficient Tokens	
	Step 5b.3 Cancel enrollment or proceeds to F01	**Step 5b.1** Detect insufficient balance **Step 5b.2** Deny enrollment show token requirements
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before confirmation → No enrollment created During processing: → Either: enrollment and payment succeed OR no changes applied After return → System reflects committed state only	
Includes		
Extends		
Assumptions	• Courses may be free or token-based	
Business Rules	• A user can enroll only once per course • Only Published and Active courses are enrollable	

	• Enrollment must grant module access immediately • Token deduction must occur before enrollment is finalized
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ENROLL
Field Validations	DDT-Course-Information DDT-Learning-Progress-Information
Author	dela Cruz, Rupert C.
Date	March 22, 2026 - Created April 18, 2026 - Revised

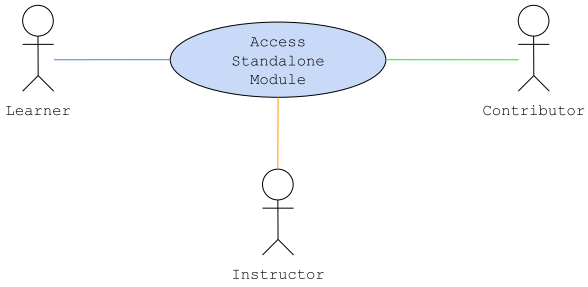
4.1.e05. Access Course Module



K:GPLPCMS - UC E05	**Access Course Module**	
Description	This use case allows an enrolled user to access a module within a course after validating enrollment, module association, and availability.	
Actor	Learner, Contributor, Instructor	
Trigger	User selects a module from course view or dashboard	
Complexity	Easy	
Pre-Condition	• User is enrolled in the course (E04) • Module exists and is linked to the course • Module is Published and Active	
Post-Condition	• Module content is displayed • User session/progress context is initialized	
Basic Flow	**Actor Action**	
	Step 5 navigates through course/module	**Step 1** Receive module request **Step 2** Retrieve module content **Step 3** Initialize session/progress context **Step 4** Display module content
Alternate Flow	**Resume Progress**	
	Step 1a.1 Reopens module **Step 1a.4** Proceed to Step 5	**Step 1a.2** Detect existing progress **Step 1a.3** Load last state
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed**	

	tab) Before display → No content shown During interaction: → Session may be lost → Progress persistence depends on the most recent state user was in
Includes	
Extends	UC E04 - Enroll in Course
Assumptions	• Progress tracking is handled by a separate subsystem
Business Rules	• Only enrolled users can access modules • Only Published and Active modules are accessible • Access must always re-validate enrollment (no cached bypass)
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-MODULE-ACCESS
Field Validations	DDT-Module-Information DDT-Learning-Progress-Information
Author	dela Cruz, Rupert C.
Date	March 22, 2026 - Created April 18, 2026 - Revised

4.1.e06. Access Standalone Module

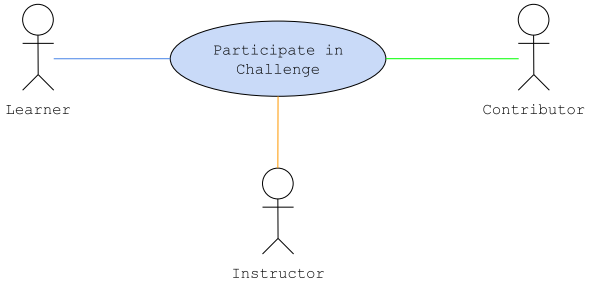


K:GPLPCMS - UC E06	Access Standalone Module	
Description	This use case allows a user to access a standalone module after validating module type, availability, and access conditions.	
Actor	Learner, Contributor, Instructor	
Trigger	Selects standalone module	
Complexity	Easy	
Pre-Condition	• Module is Published and Active	
Post-Condition	• Module content is displayed • Session/progress context is initialized	
Basic Flow	**Actor Action**	**System Response**
		Step 1 Receive request **Step 2** Validate module status (Published / Active) and access conditions (tokens /

	Step 7 Navigates through module	prerequisites) **Step 3** Process payment if required (F02) **Step 4** Retrieve module content **Step 5** Initialize session/progress context **Step 6** Display module content
Alternate Flow	**Paid Access(Token-Based)**	
	Step 3a.2 User confirms purchase	**Step 3a.1** Detect token requirement and prompts user **Step 3a.3** Validate token balance and deduct tokens (F02) **Step 3a.4** Continue to Step 4
	Insufficient Tokens	
	Step 3b.3 Cancels enrollment or proceeds to F01	**Step 3b.1** Detect insufficient balance **Step 3b.2** Deny enrollment show token requirements
	Resume Progress	
	Step 1a.1 Reopen module	**Step 1a.2** Detect existing progress **Step 1a.3** Load last state **Step 1a.4** Display module at resume point
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before validation → No access granted During processing: → Operation must be atomic → Either payment + access succeed OR no changes applied During viewing: → Session may reset → Progress persistence handled separately	
Includes		
Extends		
Assumptions	• Standalone modules are independent of courses	
Business Rules	• Course-linked modules must be accessed via E05 • Only Published and Active modules are accessible • Token deduction must occur before content access	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-MODULE-ACCESS	
Field Validations	DDT-Module-Information DDT-Learning-Progress-Information	

Author	dela Cruz, Rupert C.
Date	March 22, 2026 - Created April 18, 2026 - Revised

4.1.e07. Participate in Challenge



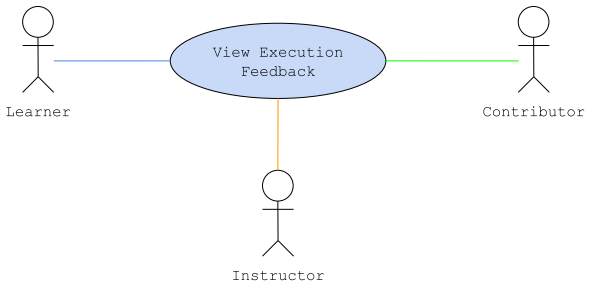
K:GPLPCMS - UC E07	**Participate in Challenge**	
Description	This use case allows a user to attempt a code challenge by interacting with the coding interface and submitting solutions for evaluation.	
Actor	Learner, Contributor, Instructor	
Trigger	User selects "Start Challenge"	
Complexity	Difficult	
Pre-Condition	• Challenge is Approved and Published (C06) • Access conditions are satisfied (tokens, prerequisites, rank) • Weekly challenge instance may exist (D04)	

Post-Condition	• Submission is processed via C07
	• Evaluation is completed via C08
	• Feedback is available (E08)
	• Submission is recorded with proper context:
	  ○ Standard (normal challenge)
	  ○ OR Weekly (linked to weekly_id)
	• Attempt is available for ranking (D03)	

Basic Flow	**Actor Action**	
	Step 1 Open challenge	**Step 2** Retrieve challenge details **Step 3** Detect active weekly context (if exists)
	Step 4 Select "Start"	**Step 5** Validate access conditions **Step 6** Initialize challenge session **Step 7** Assign context (STANDARD or WEEKLY) **Step 8** Load coding interface
	Step 9 Write code	**Step 10** Capture editor input
	Step 11 Submit solution	**Step 12** Validate submission request **Step 13** Attach context

		(include weekly_id if applicable) **Step 14** Invoke C07 — Submit Code Solution **Step 15** Invoke C08 — Evaluate Code Solution **Step 16** Retrieve evaluation result **Step 17** Display feedback (E08)
Alternate Flow	**Invalid Submission / Unsupported Language**	
	Step 12a.3 Return to Step 9	**Step 12a.1** Detect invalid format **Step 12a.2** Display validation errors
	Execution Errors (From C08)	
		Step 4a.1 Receive execution result **Step 4a.2** Display outcome (timeout/runtime/error)
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before submission → no submission recorded During submission → atomic via C07 During evaluation → continues server-side	
Includes		
Extends		
Assumptions	• Execution environment is available	
Business Rules	• E07 must not implement submission or evaluation logic • All submissions must go through C07 and C08 • Execution must enforce time and memory limits • Weekly participation must attach weekly_id to submission • Standard and weekly submissions must not be mixed	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-BROWSE	
Field Validations	DDT-Challenge-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

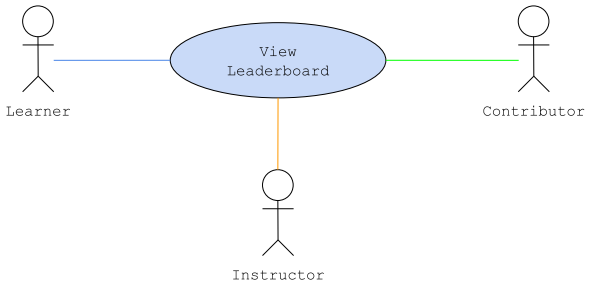
4.1.e08. View Execution Feedback



K:GPLPCMS - UC E08	View Execution Feedback	
Description	This use case allows users to view execution results and feedback for submitted code solutions.	
Actor	Learner, Contributor, Instructor	
Trigger	User access submissions interface	
Complexity	Easy	
Pre-Condition	• Submission exists (C07) • Evaluation is completed (C08)	
Post-Condition	• Execution feedback is displayed	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select submission **Step 5** Navigate feedback interface	**Step 2** Retrieve feedback data **Step 3** Process evaluation results **Step 4** Display feedback
Alternate Flow	**Select Different Submission**	
	Step 5a.1 Closes interface **Step 5a.3** Return to Step 1	**Step 5a.2** Displays submissions interface
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before display → No data shown If interruption occurs during display: → Partial rendering possible → No data modification	
Includes		
Extends		
Assumptions	• Feedback is stored after evaluation • Multiple submissions may exist per user	
Business Rules	• Feedback must reflect evaluated results only • Users may only access their own submissions • Feedback must be consistent with stored data	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-CHALLENGE-RESULT	
Field Validations	DDT-Submission-Evaluation-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created	

	April 19, 2026 - Revised

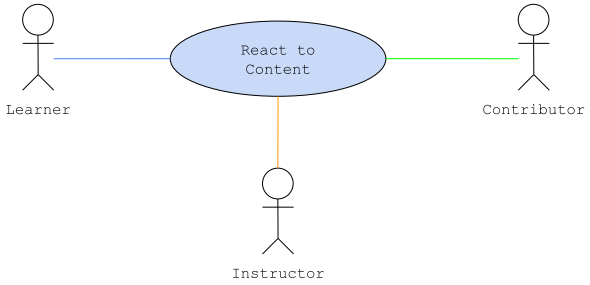
4.1.e09. View Leaderboard



K:GPLPCMS - UC E09	View Leaderboard	
Description	This use case allows users to view leaderboard rankings based on finalized scoring and ranking data.	
Actor	Learner, Contributor, Instructor	
Trigger	Select leaderboard view	
Complexity	Easy	
Pre-Condition	• Ranking data exists (D03) • Leaderboard data is available and consistent	
Post-Condition	• Leaderboard is displayed	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Apply filters or select a scope	**Step 1** Retrieve ranking data and apply default scope/filter **Step 3** Display leaderboard
Alternate Flow		
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before display → No data shown If interruption occurs during display: → Partial rendering possible → No data modification	
Includes		
Extends		
Assumptions	• Rankings are precomputed (D03) • Leaderboard reflects latest committed state (D06 if event-based)	
Business Rules	• Leaderboard must reflect validated ranking data only • Rankings must remain consistent with D03 • Only committed (published) results should be visible (D06 for events)	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD	
Field Validations	DDT-Gamification-Rewards-Information	

Author	De Guzman Asher, Nathaniel E.
Date	March 23, 2026 - Created April 19, 2026 - Revised

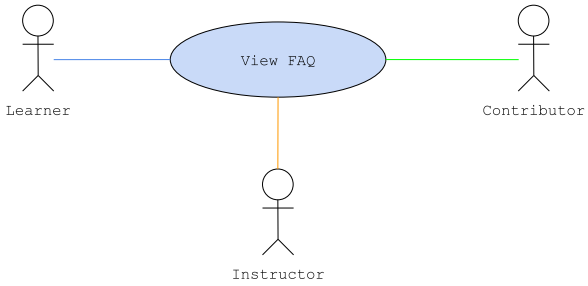
4.1.e10. React to Content



K:GPLPCMS - UC E10	Rate Content	
Description	This use case allows users to add or remove a reaction to content they have accessed.	
Actor	Learner, Contributor, Instructor	
Trigger	User is satisfied with the content	
Complexity	Easy	
Pre-Condition	• Target content exists (course / module / challenge) • User has access to the content	
Post-Condition	• Reaction is added or removed • Reaction state is updated for the user • Aggregated reaction count is updated	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select content	**Step 2** Retrieve content and current reaction state
	Step 3 Select "Like" reaction	**Step 4** Validate user eligibility **Step 5** Check existing reaction record **Step 6** Create reaction record (if none exists) **Step 7** Update aggregated reaction count (+1)
	Step 9 Use case ends	**Step 8** Confirm reaction applied
Alternate Flow	**Remove Existing Reaction**	
	Step 5a.1 Select reaction again	**Step 5a.2** Detect existing reaction **Step 5a.3** Remove reaction record **Step 5a.4** Update aggregated reaction count (-1) **Step 5a.5** Confirm reaction removed

	Unauthorized Reaction	
	Step 4a.1 Attempt reaction	**Step 4a.2** Detect lack of access **Step 4a.3** Block action **Step 4a.4** Display "Action not allowed"
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before action → No reaction applied If interruption occurs during processing → Reaction applied only if transaction completes Upon return → UI reflects last committed state	
Includes		
Extends		
Assumptions	• User is truthful on their review	
Business Rules	• User must have accessed or completed content • Each user may have one rating per content • Subsequent ratings overwrite previous entries • Rating values must follow defined scale • Aggregated metrics must reflect all valid ratings	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-GAME-LEADERBOARD	
Field Validations	DDT-User-Interaction-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 23, 2026 - Created April 19, 2026 - Revised	

4.1.e11. View FAQ

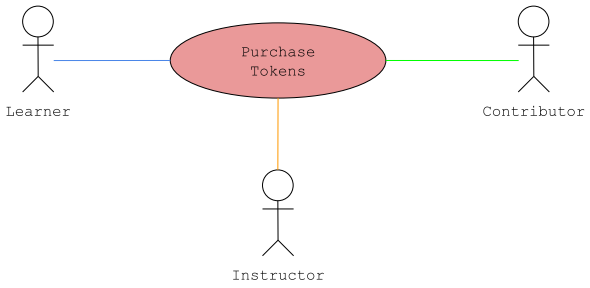


K:GPLPCMS - UC 011	View FAQ
Description	This use case allows users to browse and view published FAQ entries to obtain answers to common concerns.
Actor	Learner, Contributor, Instructor (optionally Guest — must be consistently defined system-wide)
Trigger	Open FAQ interface
Complexity	Easy
Pre-Condition	• FAQ entries are Published/Active
Post-Condition	• FAQ list or selected entry is displayed

Basic Flow	**Actor Action**	
	Step 1 Open FAQ page **Step 6** Select FAQ entry	**Step 2** Load interface **Step 3** Retrieve published FAQ entries **Step 4** Organize FAQs (category/topic) **Step 5** Display FAQ list **Step 7** Retrieve full content **Step 8** Display FAQ details
Alternate Flow	**Search or Filter FAQ**	
	Step 1a.1 Apply keyword/category filter	**Step 1a.2** Validate parameters **Step 1a.3** Retrieve filtered entries **Step 1a.4** Display updated list
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before display: No data shown During display: Partial rendering possible or no data modification	
Includes		
Extends		
Assumptions	• Entries follow structured Q&A format • Categories/tags are predefined	
Business Rules	• Only Published/Active FAQ entries must be visible • FAQ entries should be categorized for navigation • Search must support keyword-based matching • Visibility rules must be consistently enforced (including guest access, if enabled)	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-FAQ	
Field Validations	DDT-User-Interaction-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 22, 2026 - Created April 18, 2026 - Revised	

F. Transaction and Earnings Management

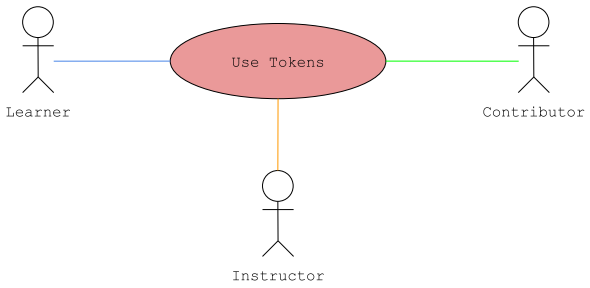
4.1.f01. Purchase Tokens



K:GPLPCMS - UC F01	Purchase Tokens	
Description	This use case allows users to purchase tokens via an external payment service. Tokens are credited only after verified payment.	
Actor	Learner, Contributor, Instructor	
Trigger	Initiate token purchase	
Complexity	Average	
Pre-Condition	• User account is Active • Payment service is available	
Post-Condition	• Payment is verified • Tokens are credited • Transaction is recorded • Confirmation is sent (F05)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select token package **Step 4** Provide payment details and confirms	**Step 2** Retrieve package details **Step 3** Create payment session **Step 5** Process payment via gateway **Step 6** Verify transaction (gateway callback/webhook) **Step 7** Credit tokens and record transaction **Step 88** Send confirmation (F05)
Alternate Flow	**Payment Failed**	
	Step 4a.1 Submit payment **Step 4a.4** Return Step 1	**Step 4a.2** Detect failure **Step 4a.3** Display "Payment failed"
	Payment Cancelled	
	Step 4b.1 Cancel payment	**Step 4b.2** Abort process **Step 4b.3** No tokens credited **Step 4b.4** End of use case
	Verification Failed	
	Step 6b.3 Return to Step 1	**Step 6a.1** Detect invalid/unverified transaction **Step 6a.2** Reject transaction
	Duplicate Transaction (Idempotency)	
		Step 6b.1 Detect repeated transaction ID **Step 6b.2** Display error and ends use case

Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before payment → No transaction created If interruption occurs during payment → Final state depends on gateway verification Upon return → System reflects verified transaction only
Includes	
Extends	
Assumptions	• The user has balance on their e-wallet / bank account
Business Rules	• Tokens must be credited only after verified payment • Duplicate transactions must not result in double credit • Payment status must be verified server-side
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-PURCHASE-AND-NOTICE
Field Validations	DDT-Token-And-Earnings-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 24, 2026 - Created April 20, 2026 - Revised

4.1.f02. Use Tokens



K:GPLPCMS - UC F02	**Use Tokens**	
Description	This use case allows users to spend tokens to access gated content or features.	
Actor	Learner, Contributor, Instructor	
Trigger	Request access to token-gated content or feature	
Complexity	Average	
Pre-Condition	• Token balance is available • Target content/feature defines token cost • Content/feature exists	
Post-Condition	• Tokens are deducted • Access is granted • Transaction is recorded	
Basic Flow	**Actor Action**	**System Response**

	Step 1 Request access	**Step 2** Retrieve token cost **Step 3** Validate gating requirement **Step 4** Check token balance validate sufficient balance **Step 5** Deduct tokens (atomic) **Step 6** Record transaction **Step 7** Grant access
Alternate Flow	**Insufficient Tokens**	
	Step 4a.2 Cancel action or proceed to F01	**Step 4a.1** Detect insufficient balance **Step 4a.2** Deny access and display error
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before validation → No deduction During processing → Operation completes only if fully committed Upon return → Balance reflects committed state only	
Includes	UC E04 - Enroll in Course UC E07 - Participate in Challenge	
Extends		
Assumptions	• The User has enough KodeBits	
Business Rules	• Tokens must not be deducted without successful validation • Token balance must never be negative • Each deduction must be recorded • Access must only be granted after successful deduction	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-PURCHASE-AND-NOTICE	
Field Validations	DDT-Token-And-Earnings-Information	
Author	Macaraeg, Mark Cyrus P.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

4.1.f03. View Publisher Earnings

K:GPLPCMS - UC F03	View Publisher Earnings
Description	This use case allows contributors and instructors to view earnings derived from validated transactions.
Actor	Contributor, Instructor
Trigger	Open earnings view
Complexity	Easy

Pre-Condition	• User has Contributor or Instructor role • Earnings data (transactions/ledger) exists • Financial records are accessible and consistent	
Post-Condition	• Earnings data is displayed	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Open earnings dashboard	**Step 2** Validate access **Step 3** Retrieve earnings data (transactions/ledger) **Step 4** Apply default scope (if any) **Step 5** Aggregate earnings **Step 6** Display earnings
Alternate Flow	**Apply Filters**	
	Step 1a.1 Apply filters (date range, content, source)	**Step 1a.2** Validate parameters and retrieve scoped data **Step 1a.3** Aggregate and display earnings
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before display → No data shown During display → Partial rendering possible or no data modification	
Includes		
Extends		
Assumptions	• The user has pre-existing uploaded paid content	
Business Rules	• Users may access only their own earnings • Earnings must reflect validated transactions only • Display must be read-only	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-PURCHASE-AND-NOTICE	
Field Validations	DDT-Token-And-Earnings-Information	
Author	Macaraeg, Mark Cyrus P.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

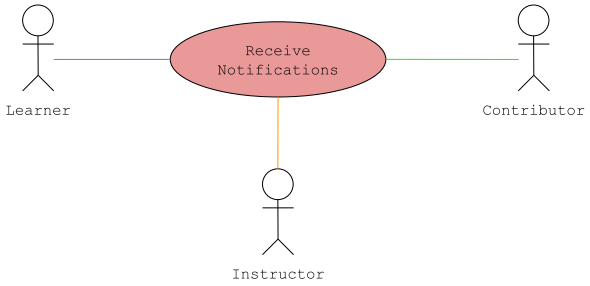
4.1.f04. Receive Earnings

K:GPLPCMS - UC F04	**Receive Earnings**
Description	This use case allows instructors to withdraw earnings via an external payout service after validation and verification; while allowing contributors to only receive the gained KodeBits to their balance

Actor	Contributor, Instructor	
Trigger	Submit payout request	
Complexity	Average	
Pre-Condition	• User has Contributor or Instructor role • Earnings balance is available • Minimum payout threshold is met • Payout method is configured • Payment service is available	
Post-Condition	• Payout is verified • Earnings are deducted • Transaction is recorded • Confirmation is sent (F05)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Request payout **Step 3** Specify payout details **Step 5** Confirm payout	**Step 2** Retrieve earnings balance **Step 4** Validate request (balance, threshold, method) **Step 6** Create payout request (idempotent) **Step 7** Initiate payout via service **Step 8** Verify payout result (callback/webhook) **Step 9** Deduct earnings and record transaction **Step 10** Send confirmation (F05)
Alternate Flow	**Contributor Path**	
		Step 2a.1 Detect user is contributor **Step 2a.2** Accredit KodeBits to user wallet **Step 2a.3** Go to Step 10
	Payout Cancelled	
	Step 5b.1 Cancel operation	**Step 5b.2** Abort process **Step 5b.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before confirmation → No payout initiated During processing → Final state depends on verified payout result Upon return → System reflects committed transaction only	
Includes		
Extends		
Assumptions	• The user has earnings in the platform	
Business Rules	• Earnings must be deducted only after verified payout	

	• Payout requests must meet minimum threshold • Payout verification must be server-side • Balance must never become negative
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-PURCHASE-AND-NOTICE
Field Validations	DDT-Token-And-Earnings-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 24, 2026 - Created April 20, 2026 - Revised

4.1.f05. Receive Notifications

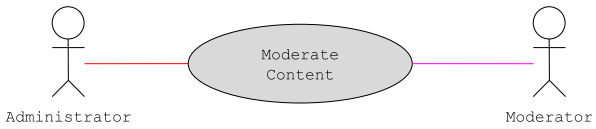


K:GPLPCMS - UC F05	Receive Notifications	
Description	This use case delivers system-generated notifications to users via in-app or email channels based on event type and priority.	
Actor	Learner, Contributor, Instructor	
Trigger	Notification event raised by another use case	
Complexity	Average	
Pre-Condition	• User account exists and is Active	
Post-Condition	• Notification is delivered (or queued for retry) • Delivery status is recorded	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Event occurs	**Step 2** Resolve recipient(s) **Step 3** Determine channel (in-app/email) **Step 4** Generate message content **Step 5** Deliver notification **Step 6** Record delivery status
Alternate Flow		
Exception Flow	**Delivery Failure** If notification failed to deliver → retries delivery	
Includes		
Extends		
Assumptions	• Notification types and priorities are predefined	

	• Email service is available (for email channel)
Business Rules	• Notifications must originate from valid system events • Critical events must use email delivery (with retry): token purchase (F01), payouts/earnings (F04), approvals/rejections (G03, G07) • Non-critical events may use in-app delivery only • User preferences may modify non-critical delivery channels
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-PURCHASE-AND-NOTICE
Field Validations	DDT-Notification-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 24, 2026 - Created April 20, 2026 - Revised

G. Administration and Governance

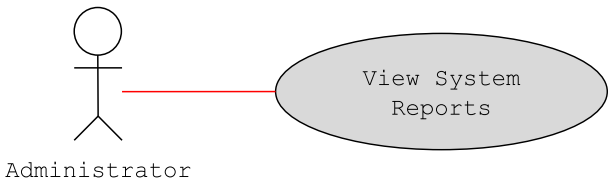
4.1.g01. Moderate Content



K:GPLPCMS - UC G01	Moderate Content	
Description	This use case allows administrators and moderators to review flagged or queued content and apply moderation decisions.	
Actor	Administrator, Moderator	
Trigger	Select content for moderation	
Complexity	Average	
Pre-Condition	• Content exists (courses/modules/challenges) • Content is flagged or queued for review	
Post-Condition	• Moderation decision is applied • Content status is updated • Notification is sent (F05, if applicable)	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select content **Step 3** Review content **Step 5** Select action (approve/reject/remove) **Step 7** Confirm decision	**Step 2** Retrieve content and reports **Step 4** Display details and context **Step 6** Validate action against policy **Step 8** Apply decision (atomic) **Step 9** Update content status **Step 10** Record moderation action (audit)

		Step 11 Send notification (F05)
Alternate Flow	**Cancel Operation**	
	Step 7a.1 Cancel operation	**Step 7a.2** Aborts operation **Step 7a.3** End of use case
	Concurrent Moderation Conflict	
	Step 2a.3 Reloads or invoke Cancel Operation	**Step 2a.1** Detect multiple account editing **Step 2a.2** Prevent actions and prompts reload or merge **Step 2a.4** Return to Step 1
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** Before confirmation → No action applied During processing → Operation completes only if fully committed Upon return → Content reflects committed state	
Includes		
Extends		
Assumptions	• Moderation policies define allowed actions and transitions • Content may originate from multiple modules (B, C)	
Business Rules	• All actions must follow predefined policies • Moderation must result in valid state transitions only • All actions must be logged and auditable • Duplicate moderation must be prevented	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations	DDT-Moderation-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

4.1.g02. View System Reports



K:GPLPCMS - UC G02	View System Reports
Description	This use case allows administrators to access and analyze system reports related to user activity, content performance, gamification outcomes, and financial data.
Actor	Administrator

Trigger	User selects "View System Reports"	
Complexity	Easy	
Pre-Condition	• Administrator is authenticated • Reporting data exists and is accessible • Data sources are up-to-date and validated	
Post-Condition	• System reports are displayed	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select report type **Step 3** Apply filters/parameters	**Step 2** Retrieve report data **Step 4** Validate parameters **Step 5** Aggregate data **Step 6** Display report
Alternate Flow	**Invalid Parameters**	
	Step 4a.3 Return to Step 3	**Step 4a.1** Detect validation error **Step 4a.2** Display error message
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before display → No data shown If interruption occurs during display: → Partial rendering may occur → No data modification	
Includes		
Extends		
Assumptions	• Reports are generated from validated system data • Filtering and aggregation are supported	
Business Rules	• Only administrators can access system reports • Reports must be read-only • Filtering must not alter underlying data • Reports must reflect latest available state • Aggregation must be consistent and reproducible	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations	DDT-Moderation-Information	
Author	De Guzman Asher, Nathaniel E.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

4.1.g03. Approve Contributor Requests

K:GPLPCMS - UC G03	**Approve Contributor Requests**
Description	This use case allows administrators to review and

	decide on contributor applications submitted by users.	
Actor	Administrator, Moderator	
Trigger	Select contributor application for review	
Complexity	Average	
Pre-Condition	• Administrator/Moderator is authenticated • Contributor application exists (A07) • Application data is complete and accessible	
Post-Condition	• Application decision is applied • User role is updated if approved • Notification is sent (F05)	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Review application **Step 4** Select decision (approve/reject) **Step 6** Confirm decision	**Step 1** Retrieve application data **Step 3** Display applicant details **Step 5** Validate decision **Step 7** Apply decision **Step 8** Update user role (if approved) **Step 9** Record decision **Step 10** Send notification
Alternate Flow	**Invalid Parameters**	
	Step 1a.1 Select invalid option	**Step 1a.2** Detect invalid action **Step 1a.3** Display error
	No Data Available	
	Step 2a.1 Select application	**Step 2a.2** Detect missing record **Step 2a.3** Display error
	Large Dataset Handling	
	Step 3a.1 Select processed application	**Step 3a.2** Detect final state **Step 3a.3** Display "Already processed"
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation: → No decision applied If interruption occurs during processing: → Decision must be atomic → Either decision + role update applied, OR no changes occur Upon return: → Application reflects last committed state	
Includes		
Extends		
Assumptions	• Role assignment rules are defined	

Business Rules	• Each application must be processed once • Approval must assign contributor role • Rejection must not alter user role • Notifications must be sent for all outcomes
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ACC-ROLE
Field Validations	DDT-Contributor-Request-Information
Author	De Guzman Asher, Nathaniel E.
Date	March 24, 2026 - Created April 20, 2026 - Revised

4.1.g04. Suspend User Account

K:GPLPCMS - UC G04	Suspend User Account	
Description	This use case allows administrators and moderators to suspend a user account due to policy violations or other enforcement actions.	
Actor	Administrator, Moderator	
Trigger	Select user account for suspension	
Complexity	Average	
Pre-Condition	• Administrator or Moderator is authenticated • Target user account exists (G06) • Account is active (not already suspended)	
Post-Condition	• User account status is set to "Suspended" • Access to system functionalities is restricted • Notification is sent (F05)	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Initiate suspension **Step 4** Confirm suspension	**Step 1** Retrieve account details **Step 3** Validate action **Step 5** Apply suspension **Step 6** Update account status **Step 7** Record action **Step 8** Send notification
Alternate Flow	**Already Suspended**	
	Step 1a.1 Select suspended account	**Step 1a.2** Detect current status **Step 1a.3** Display "Already suspended"
	Invalid Action	
	Step 2a.1 Initiate restricted action	**Step 2a.2** Detect violation **Step 2a.3** Display error
	User Not Found	

	Step 3a.1 Select account	**Step 3a.2** Step A3.2 Detect missing record **Step 3a.3** Display error
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No suspension applied If interruption occurs during processing: → Suspension must be atomic → Either account is suspended and recorded, OR no changes occur Upon return → Account reflects last committed state	
Includes		
Extends		
Assumptions	• A user behaves suspiciously within the system	
Business Rules	• Suspension must apply immediately after confirmation • Suspended accounts must not access restricted features • Notification must be sent for enforcement actions	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

4.1.g05. Reinstate User Account

K:GPLPCMS - UC G05	Reinstate User Account	
Description	This use case allows administrators and moderators to reinstate a suspended user account.	
Actor	Administrator, Moderator	
Trigger	Select suspended user account for reinstatement	
Complexity	Average	
Pre-Condition	• Administrator or Moderator is authenticated • Target user account exists (G06) • Account is currently suspended	
Post-Condition	• User account status is set to "Active" • Access to system functionalities is restored • Notification is sent (F05)	
Basic Flow	**Actor Action**	**System Response**
		Step 1 Retrieve and display account details

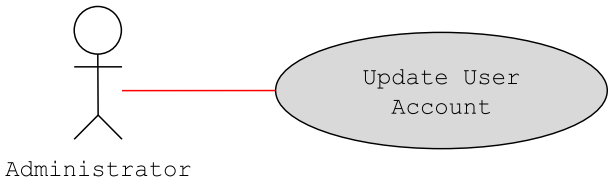
	Step 2 Initiate reinstatement **Step 4** Confirm reinstatement	**Step 3** Validate action **Step 5** Apply reinstatement **Step 6** Update account status and record action **Step 7** Send notification to user
Alternate Flow		
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No reinstatement applied If interruption occurs during processing: → Reinstatement must be atomic → Either account is reinstated and recorded, OR no changes occur Upon return → Account reflects last committed state	
Includes		
Extends		
Assumptions	• User appealed for reinstatement	
Business Rules	• Reinstatement must apply immediately after confirmation • Reinstated accounts must regain access to permitted features • Notification must be sent for enforcement reversal	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

4.1.g06. View User Accounts

K:GPLPCMS - UC G06	View User Accounts
Description	This use case allows administrators and moderators to view user accounts, including roles, status, and relevant account details.
Actor	Administrator, Moderator
Trigger	Select "View User Accounts"
Complexity	Easy
Pre-Condition	• Administrator or Moderator is authenticated • User account data exists
Post-Condition	• User account information is displayed

Basic Flow	Actor Action	System Response
	Step 2 Apply filters/search	**Step 1** Retrieve and display user data **Step 3** Validate parameters **Step 4** Process user data **Step 5** Display user accounts
Alternate Flow	**Invalid Filter Parameters**	
	Step 3a.3 Return to Step 2	**Step 3a.1** Detect validation error **Step 3a.2** Display error
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before display → No data shown If interruption occurs during display: → Partial rendering may occur → No data modification	
Includes		
Extends		
Assumptions	• There are other registered users in the system	
Business Rules	• Only administrators and moderators can view user accounts • Display must be read-only • Sensitive data must be restricted based on access level • Displayed data must be consistent and up-to-date	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

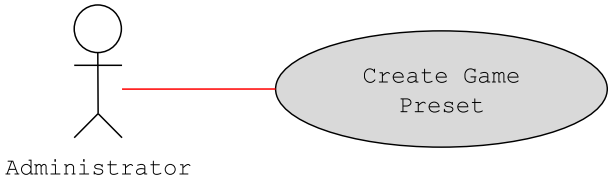
4.1.g07. Update User Account



K:GPLPCMS - UC G07	Edit User Account
Description	This use case allows administrators to update user account attributes such as roles, permissions, and profile-related administrative fields.
Actor	Administrator
Trigger	User selects "Edit User Account"
Complexity	Difficult
Pre-Condition	• Administrator is authenticated

	• Target user account exists (G06) • Update does not conflict with enforcement actions (G04, G05)	
Post-Condition	• User account is updated • Notification is sent if applicable (F05)	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Initiate update **Step 3** Modify account attributes **Step 4** Submit updates	**Step 1** Retrieve account details and display editable fields **Step 5** Validate changes **Step 6** Apply updates and store updated data **Step 7** Record action **Step 8** Send notification
Alternate Flow	**Invalid Input**	
	Step 5a.3 Return to Step 3	**Step 5a.1** Detect validation error **Step 5a.2** Display error
	Unauthorized Modification	
	Step 2a.1 Attempt restricted update	**Step 2a.2** Detect violation **Step 2a.3** Reject update
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No updates applied If interruption occurs during processing: → Update must be atomic → Either all changes applied and stored, OR no changes occur Upon return → Account reflects last committed state	
Includes		
Extends		
Assumptions	• There are error/s that needs attention in a user's account • The user requested for help in configuring basic account details	
Business Rules	• Updates must not conflict with enforcement states • Role changes must follow defined hierarchy rules • Updates must be validated before application • Notifications must be sent for significant changes	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations	KODY DDT-User-Account-Information	
Author	dela Cruz, Rupert C.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

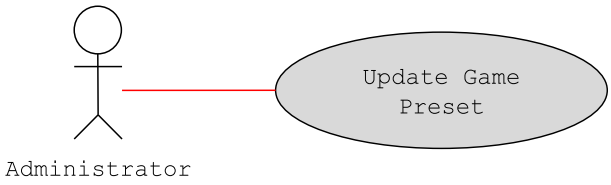
4.1.g08. Create Game Preset



K:GPLPCMS - UC G08	Create Game Preset	
Description	This use case allows administrators to create a new game preset by defining rules, scoring logic, reward structures, and participation conditions.	
Actor	Administrator	
Trigger	Select "Create Game Preset"	
Complexity	Difficult	
Pre-Condition	• Administrator is authenticated • Preset configuration interface is accessible	
Post-Condition	• Game preset is created, stored and available for use	
Basic Flow	**Actor Action**	**System Response**
	Step 2 Input preset details **Step 4** Submit preset	**Step 1** Display creation form **Step 3** Capture input **Step 5** Validate configuration **Step 6** Check rule consistency **Step 7** Create preset and store preset **Step 8** Record action
Alternate Flow	**Invalid Input / Conflicting Rules or Preset Name**	
	Step 5a.3 Return to Step 2	**Step 5a.1** Detect validation error / conflicts **Step 5a.2** Display error
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No preset created If interruption occurs during processing → No partial preset stored Upon return → Creation must be restarted	
Includes		
Extends		
Assumptions	• The administrator has access in the platform's backend	
Business Rules	• Reward structures must be defined • Presets must comply with system constraints	

	• All creations must be logged and traceable
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN
Field Validations	
Author	Macaraeg, Mark Cyrus P.
Date	March 24, 2026 - Created April 20, 2026 - Revised

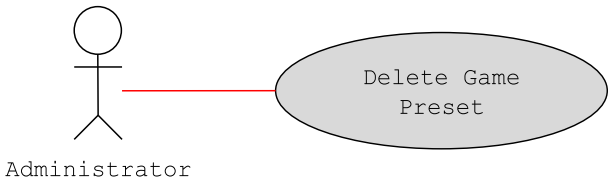
4.1.g09. Update Game Preset



K:GPLPCMS - UC G09	Update Game Preset (Refactored)	
Description	This use case allows administrators to update an existing game preset.	
Actor	Administrator	
Trigger	Select "Update Game Preset"	
Complexity	Highly Complex	
Pre-Condition	• Administrator is authenticated • Target preset exists • Preset usage status is determinable	
Post-Condition	• Updated configuration is stored • Existing game instances remain unaffected • Action is recorded	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select preset **Step 3** Initiate update **Step 5** Modify configuration **Step 7** Submit update	**Step 2** Retrieve preset data **Step 4** Display editable fields **Step 6** Capture changes **Step 8** Validate changes **Step 9** Check preset usage **Step 10** Apply update or create version store configuration **Step 11** Record action
Alternate Flow	**Invalid Input / Conflicting Rules**	
	Step 8a.1 Return to Step 5	**Step 8a.1** Detect validation error **Step 8a.2** Display error
	Preset In Use (Versioning)	
		Step 9a.1 Detect active usage **Step 9a.2** Create new

		version and preserve original preset **Step 9a.3** Proceed to Step 10
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No changes applied If interruption occurs during processing: → Update must be atomic → Either update/version applied and stored, OR no changes occur Upon return → Preset reflects last committed state	
Includes		
Extends		
Assumptions	• The administrator has access in the platform's backend	
Business Rules	• Updates must be validated before application • In-use presets must be versioned, not overwritten • Updates must not affect existing game instances	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations		
Author	Macaraeg, Mark Cyrus P.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

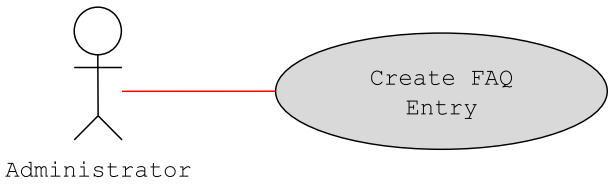
4.1.g10. Delete Game Preset



K:GPLPCMS - UC G10	Delete Game Preset	
Description	This use case allows administrators to remove an existing game preset.	
Actor	Administrator	
Trigger	Select "Delete Game Preset"	
Complexity	Difficult	
Pre-Condition	• Administrator is authenticated • Target preset exists	
Post-Condition	• Preset is deleted or marked inactive • Preset is unavailable for future use	
Basic Flow	**Actor Action**	**System Response**

	Step 1 Select preset **Step 3** Initiate deletion **Step 5** Confirm deletion	**Step 2** Retrieve preset data **Step 4** Validate request **Step 6** Check preset usage **Step 7** Apply deletion or inactivation **Step 8** Update preset status and record action
Alternate Flow	**Preset In Use (Soft Delete)**	
	Step 6a.3 End of use case	**Step 6a.1** Detect active usage **Step 6a.2** Mark preset as inactive and preserve existing references
	Deletion Cancelled	
	Step 5a.1 Cancel deletion	**Step 5a.2** Abort operation **Step 5a.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No deletion applied If interruption occurs during processing: → Deletion must be atomic → Either deletion/inactivation applied and recorded, OR no changes occur Upon return → Preset reflects last committed state	
Includes		
Extends		
Assumptions	• Presets may be referenced by active or historical game instances • System supports soft delete mechanism	
Business Rules	• Presets in use must not be permanently deleted • Deleted presets must not be available for new game generation • All deletion actions must be logged and traceable	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations		
Author	Macaraeg, Mark Cyrus P.	
Date	March 24, 2026 - Created April 20, 2026 - Revised	

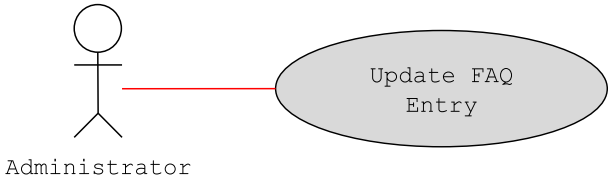
4.1.g11. Create FAQ Entry



K:GPLPCMS - UC G11	Create FAQ Entry	
Description	This use case allows administrators to create a new FAQ entry consisting of a question and answer.	
Actor	Administrator	
Trigger	Select "Create FAQ Entry"	
Complexity	Easy	
Pre-Condition	• Administrator is authenticated • FAQ management interface is accessible	
Post-Condition	• FAQ entry is created • Entry is stored and visible to users	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Open FAQ creation **Step 3** Input question and answer **Step 5** Submit entry	**Step 2** Display entry form **Step 4** Capture input **Step 6** Validate fields **Step 7** Create and store entry **Step 8** Record action
Alternate Flow	**Invalid Input**	
		Step 5a.1 Detect validation error **Step 5a.2** Display error
	Step 5a.3 Return to Step 3	
	Duplicate Question	
		Step 5b.1 Detect duplication **Step 5b.2** Display warning
	Step 5b.3 Submit similar question	
	Creation Cancelled	
	Step 5c.1 Cancel creation	**Step 5c.2** Abort operation **Step 5c.3** End of use case
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No entry created If interruption occurs during processing → No partial entry stored Upon return → Creation must be restarted	
Includes		
Extends		
Assumptions	• FAQ entry will help regular users in navigating through the platform	
Business Rules	• Question and answer fields must be non-empty • Entries should avoid duplication • All creations must be logged and traceable	
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN	
Field Validations	DDT-User-Interaction-Information	

Author	Macaraeg, Mark Cyrus P.
Date	March 24, 2026 - Created April 20, 2026 - Revised

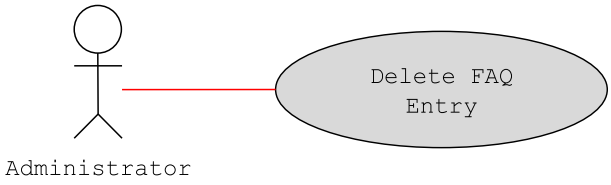
4.1.g12. Update FAQ Entry



K:GPLPCMS - UC G12	**Update FAQ Entry (Refactored)**	
Description	This use case allows administrators to update an existing FAQ entry by modifying its question and/or answer.	
Actor	Administrator	
Trigger	Select "Update FAQ Entry"	
Complexity	Easy	
Pre-Condition	• Administrator is authenticated • Target FAQ entry exists	
Post-Condition	• FAQ entry is updated • Updated content is visible to users	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select FAQ entry **Step 3** Initiate update **Step 5** Modify content **Step 7** Submit update	**Step 2** Retrieve entry data **Step 4** Display editable fields **Step 6** Capture changes **Step 8** Validate changes **Step 9** Apply updates store updated entry **Step 10** Record action
Alternate Flow	**Duplicate Content**	
	Step 7a.3 Return to Step 5	**Step 8a.1** Detect duplication **Step 8a.2** Display warning
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before submission → No changes applied If interruption occurs during processing: → Update must be atomic → Either all changes applied and stored, OR no changes occur Upon return: → Entry reflects last committed state	
Includes		
Extends		
Assumptions	• FAQ entry will help regular users in navigating	

	through the platform
Business Rules	• Updated fields must not be empty • Changes must be validated before application
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN
Field Validations	DDT-User-Interaction-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 24, 2026 - Created April 20, 2026 - Revised

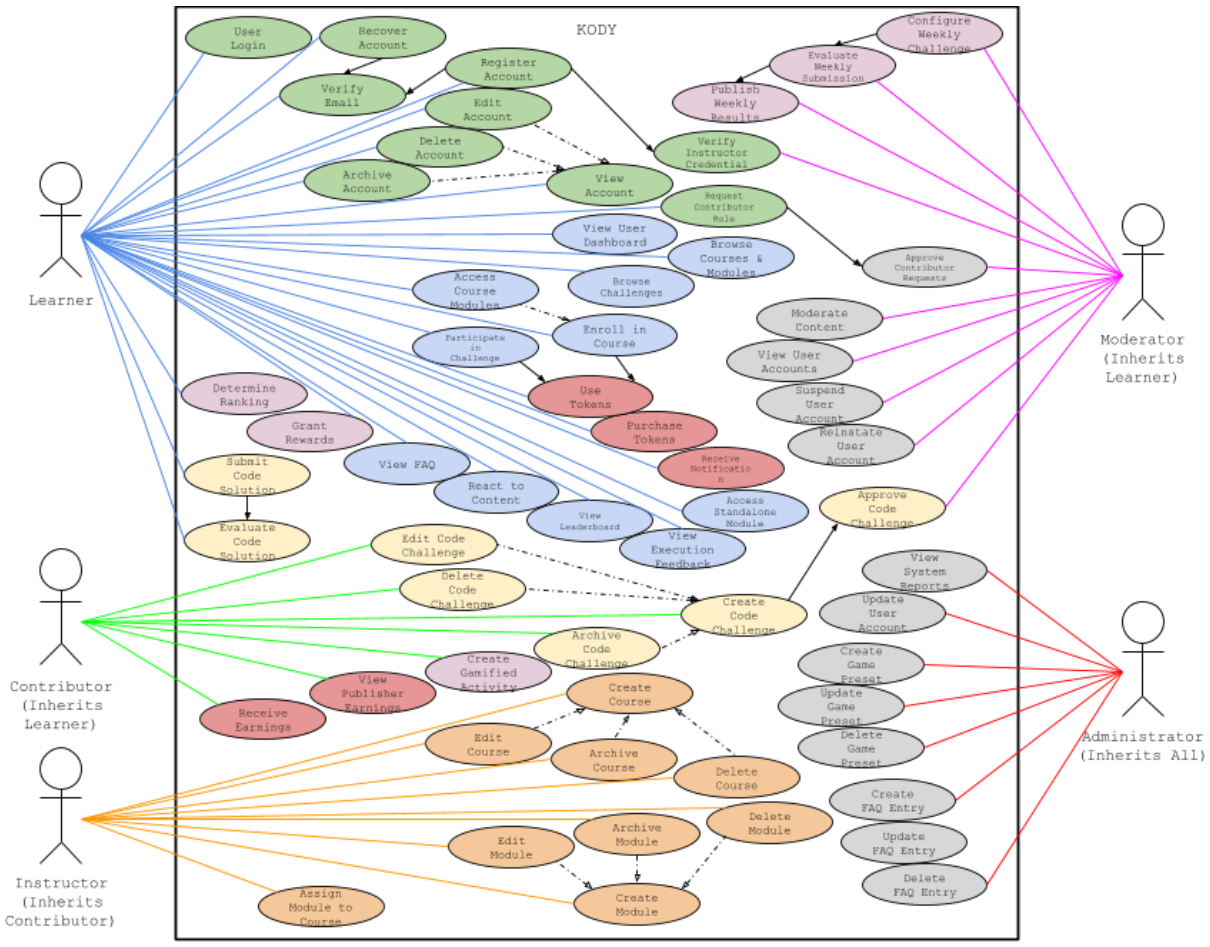
4.1.g13. Delete FAQ Entry



K:GPLPCMS - UC G13	Delete FAQ Entry (Refactored)	
Description	This use case allows administrators to delete an existing FAQ entry. The entry is removed from the system and is no longer visible to users.	
Actor	Administrator	
Trigger	Select "Delete FAQ Entry"	
Complexity	Easy	
Pre-Condition	• Administrator is authenticated • Target FAQ entry exists • FAQ management interface is accessible	
Post-Condition	• FAQ entry is deleted • Entry is no longer visible to users	
Basic Flow	**Actor Action**	**System Response**
	Step 1 Select FAQ entry **Step 3** Initiate deletion **Step 5** Confirm deletion	**Step 2** Retrieve entry data **Step 4** Validate request **Step 6** Delete entry **Step 7** Update FAQ list **Step 8** Record action
Alternate Flow	**Deletion Cancelled**	
	Step 1a.1 Cancel deletion	**Step 1a.2** Abort operation **Step 1a.3** No changes applied
Exception Flow	**Client-Side Interruptions (Lost connection/ Refreshed tab)** If interruption occurs before confirmation → No deletion applied If interruption occurs during processing: → Deletion must be atomic → Either entry deleted and recorded, OR no changes occur Upon return → System reflects last committed state	

Includes	
Extends	
Assumptions	• FAQ entry is outdated and/or has major mistakes
Business Rules	• Deletion must require explicit confirmation • Deleted entries must not be accessible
Related Models / Diagrams	KODY: GPLPCMS - USE CASE DIAGRAM KODY-ADMIN
Field Validations	DDT-User-Interaction-Information
Author	Macaraeg, Mark Cyrus P.
Date	March 24, 2026 - Created April 20, 2026 - Revised

4.2 Use Case Diagram



Legend:

- Learner
- Contributor
- Instructor
- Moderator
- Administrator
- Account & Auth. Mngt.
- Content Mngt.
- Challenge Mngt.
- Gamification & Reward
- Interaction & Engage.
- Transaction &
- Administ. & Governance
- <<include>>
- <<>extend>

4.3 Non-Functional Requirements

4.3.a System Interfaces

Code Execution Response Time

The system shall return code execution results, including output, errors, and evaluation metrics, within 3–5 seconds under normal operating conditions.

Page Load Performance

Core system interfaces (Dashboard, Course Listings, Challenges, Leaderboards) shall fully render within ≤3 seconds on a standard broadband connection.

Concurrent Users

The system shall support at least 500 concurrent users while maintaining response time degradation within ≤2 seconds of baseline performance.

Asynchronous Processing

Non-critical operations (e.g., notifications, leaderboard updates) shall be processed asynchronously to prevent blocking of user interactions.

4.3.b Security

Sandboxed Code Execution

All user-submitted code shall execute in an isolated sandbox environment with no access to system memory, database, file system, or internal services.

Data Protection and Encryption

All sensitive data shall be:

- Encrypted at rest using AES-256
- Encrypted in transit using TLS 1.3

Authentication and Session Management

The system shall validate session tokens on all authenticated requests. Sessions shall expire after a defined period of inactivity.

Role-Based Access Control (RBAC)

Access to system functionalities shall be restricted based on user roles, preventing unauthorized execution of privileged actions.

Rate Limiting and Abuse Prevention

The system shall enforce request limits on critical endpoints (logins and submissions) to mitigate brute-force and abuse attempts.

4.3.c Reliability and Availability

System Availability

The system shall maintain at least 99.9% uptime, excluding scheduled maintenance.

Fault Tolerance

The system shall handle failures in external services (code execution, payment, and email) by:

- returning appropriate error responses
- retrying operations when applicable

Data Persistence and Backup

Critical user data (XP, rank, KodeBits balance, course progress) shall be:

- stored persistently
- backed up at least once every 24 hours

Maintenance Constraints

Scheduled maintenance shall:

- occur during off-peak hours (e.g., 1:00–5:00 AM PHT)
- not exceed 4 hours per month

4.3.d Usability

Responsive Interface

The system shall provide a responsive interface supporting desktop and mobile browsers. Core learning content shall remain readable and navigable across device sizes.

Browser Compatibility

The system shall support the latest stable versions of:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Learnability and Consistency

Navigation, content structure, and interaction patterns shall be consistent across modules to reduce user learning effort.

4.3.e Scalability and Maintainability

Modular System Design

The system shall be composed of modular components (e.g., authentication, content management, gamification), enabling isolated updates without affecting the entire system.

Horizontal Scalability

The system shall support scaling of application and database resources to accommodate increasing user load.

External Service Integration

The system shall integrate external services (authentication, code execution, email, payment) through well-defined APIs to allow replacement or upgrades without major system redesign.

Logging and Monitoring

The system shall maintain logs for:

- user activity
- system errors
- transaction events

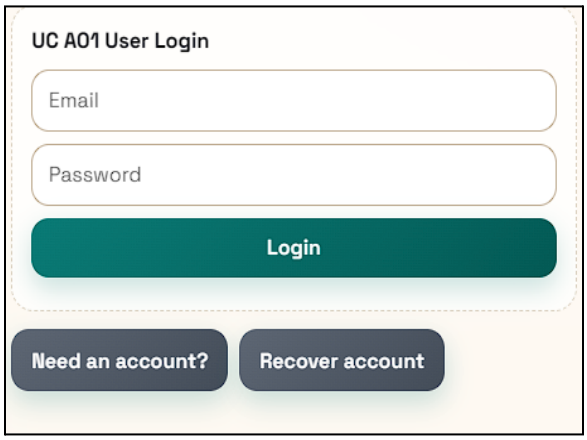
Logs shall support debugging, auditing, and system monitoring.

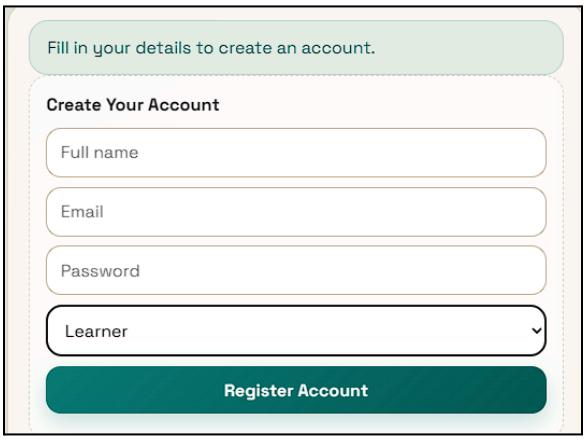
5 Other Requirements

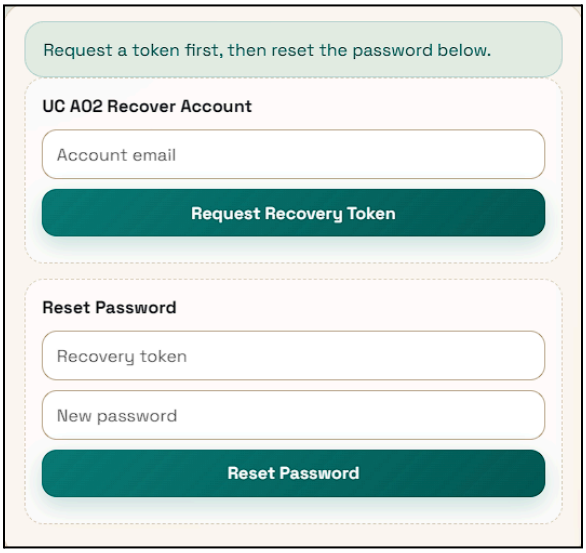
5.1 Interfaces

5.1.a Authentication Interfaces

KODY-ACC-AUTH



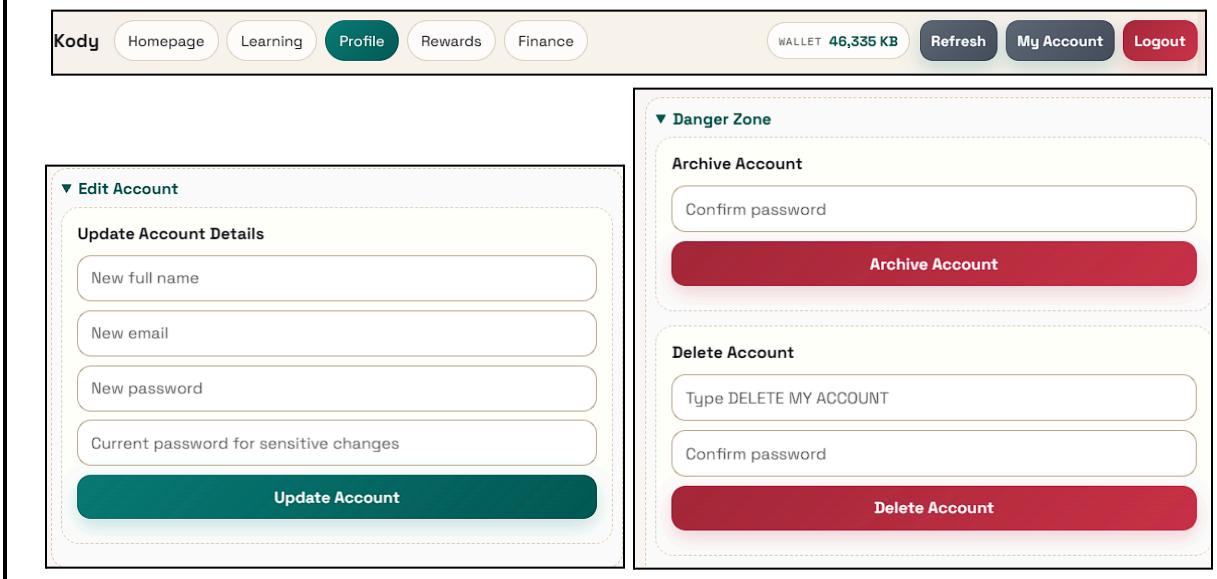




Purpose	Allows users to log in, register, verify email, and recover accounts.
Navigation and user	To access the system, the user enters valid login credentials or registers a new account. Email verification is required for activation. If credentials are forgotten, the user may initiate account recovery. Successful authentication redirects the user to the dashboard.

5.1.b Account Management Interfaces

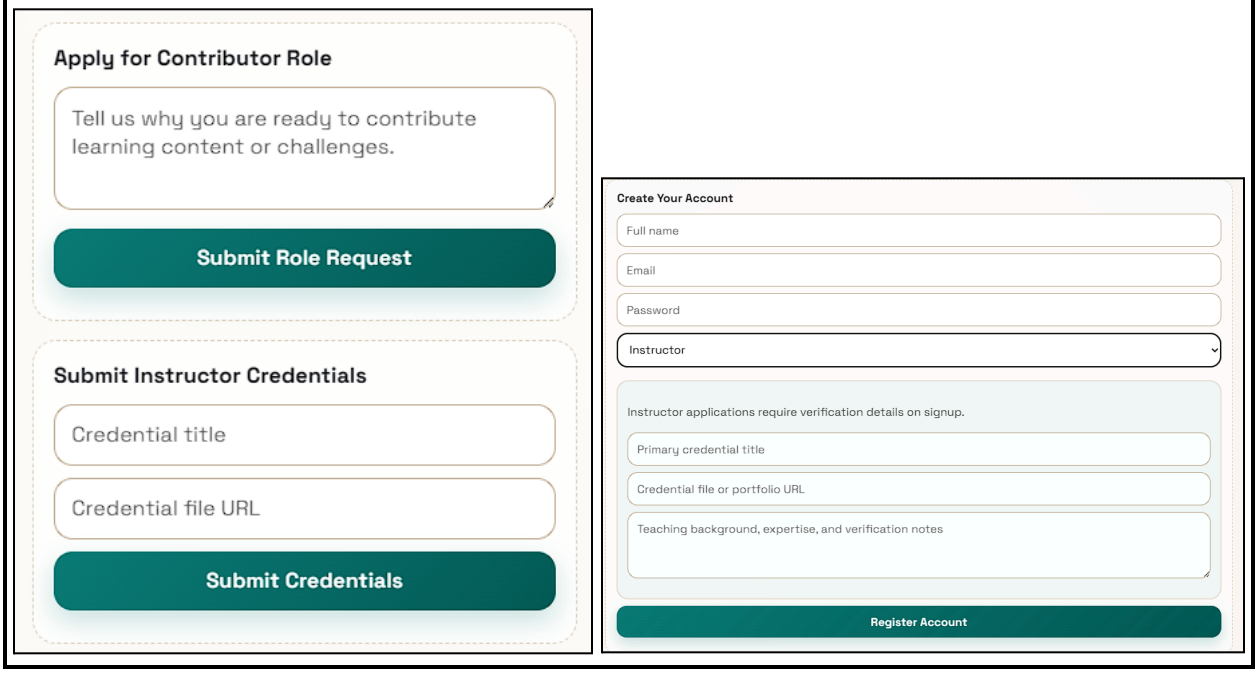
KODY-ACC-MNGT

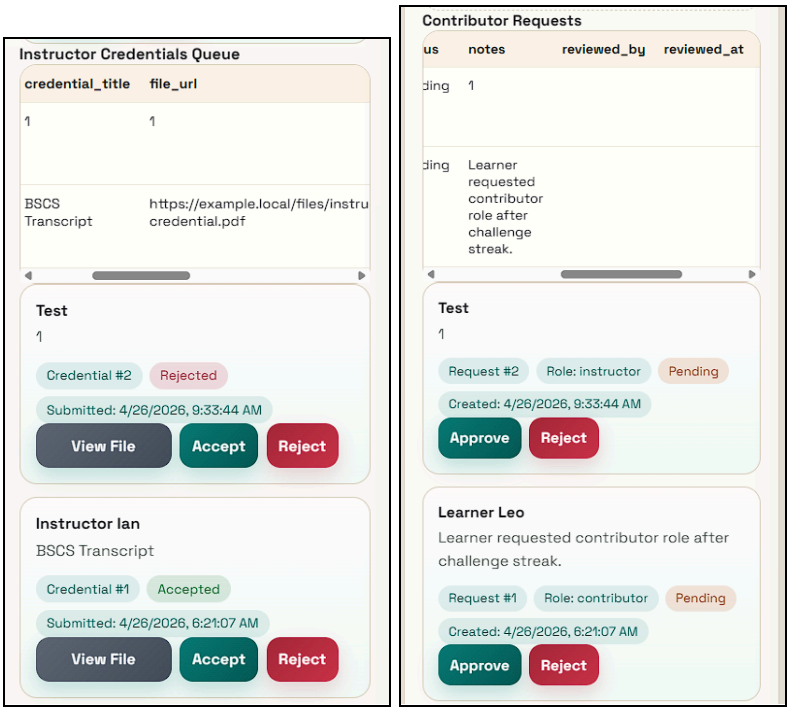


Purpose	Allows users to view, update, archive, or delete their account.
Navigation and user	The user accesses account settings from the navigation bar to view or edit profile details. The user may archive or permanently delete the account, which updates account status and restricts future access.

5.1.c Role Request & Verification Interface Interfaces

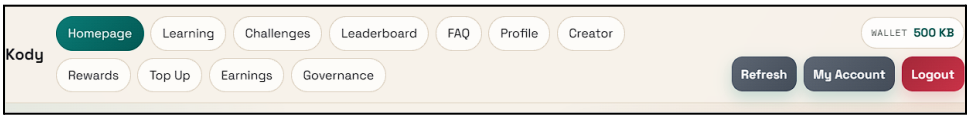
KODY-ACC-ROLE

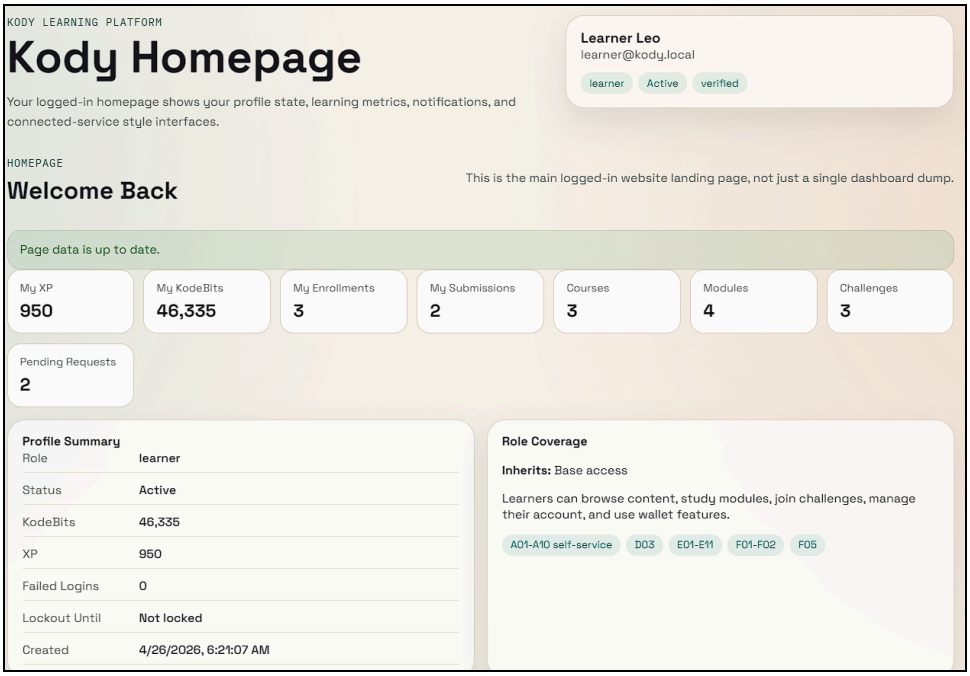


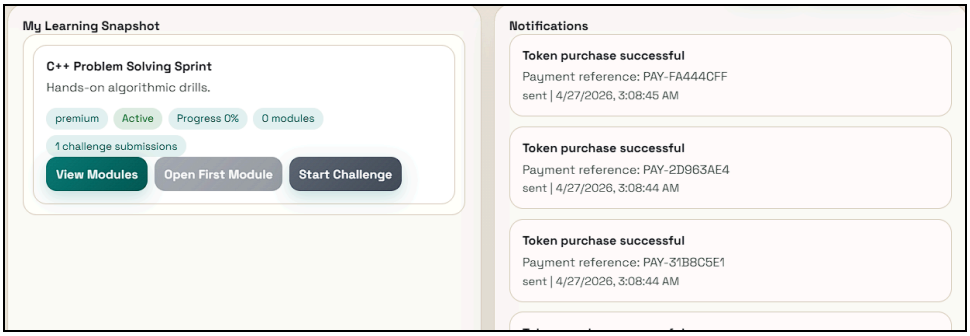


Purpose	Allows users to request elevated roles and submit credentials for verification.
Navigation and user	The user submits a role request with justification or uploads credentials for instructor verification. Admin reviews and approves or rejects the request, updating the user's role accordingly.

5.1.d Dashboard Interface

KODY-DASHBOARD


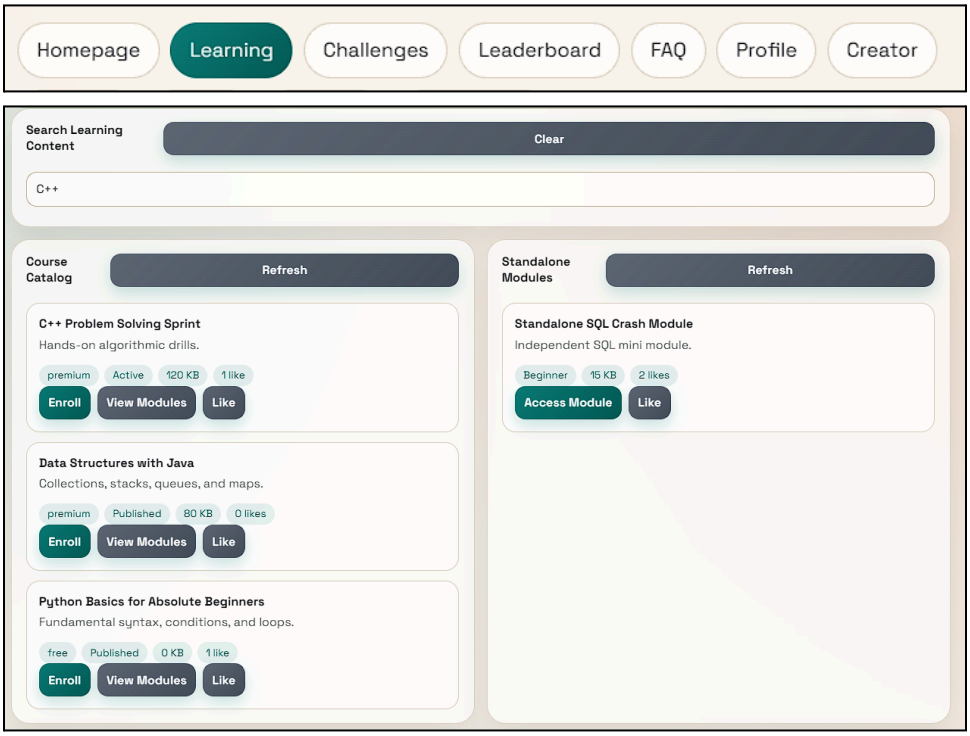




Purpose	Displays user progress, activities, and system notifications.
Navigation and user	Upon login, the user is redirected to the dashboard, where progress, rankings, and recent activities are displayed. The user navigates to courses, challenges, or other features from this interface.

5.1.e Course Browsing Interface

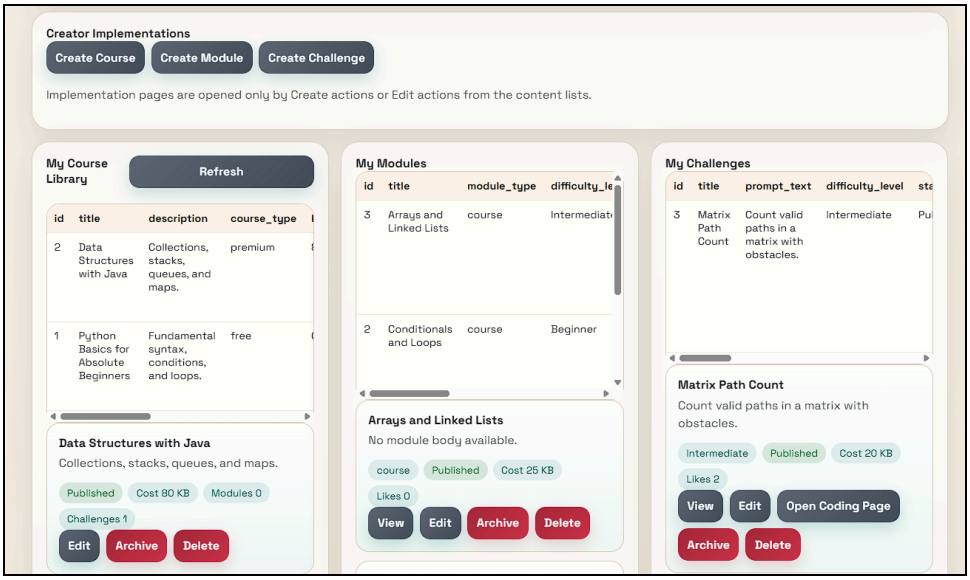
KODY-COURSE-BROWSE

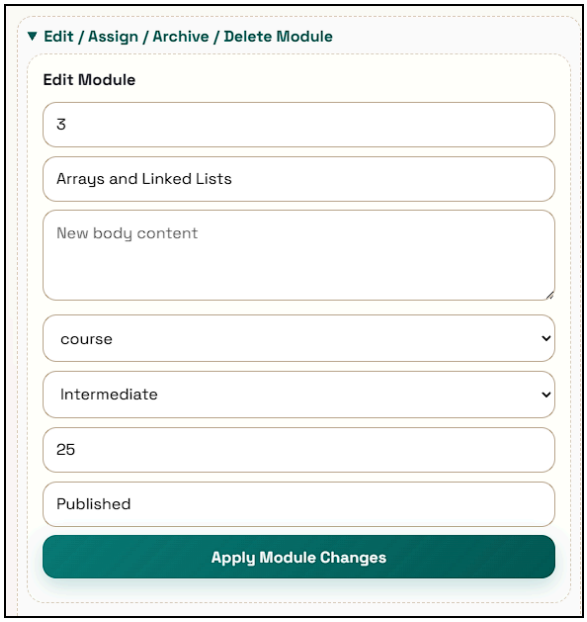


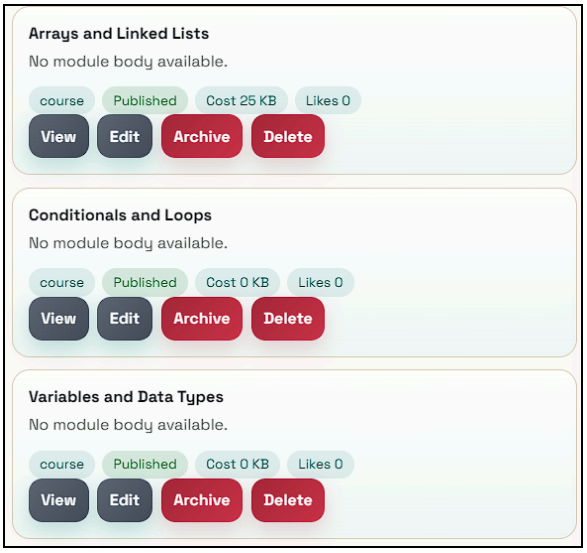
Purpose	Allows users to browse available courses and modules.
Navigation and user	The user navigates from the dashboard to browse courses and modules. Selecting a course displays its details and available modules.

5.1.f Course and Module Management Interface

KODY-COURSE-MODULE-MNGT

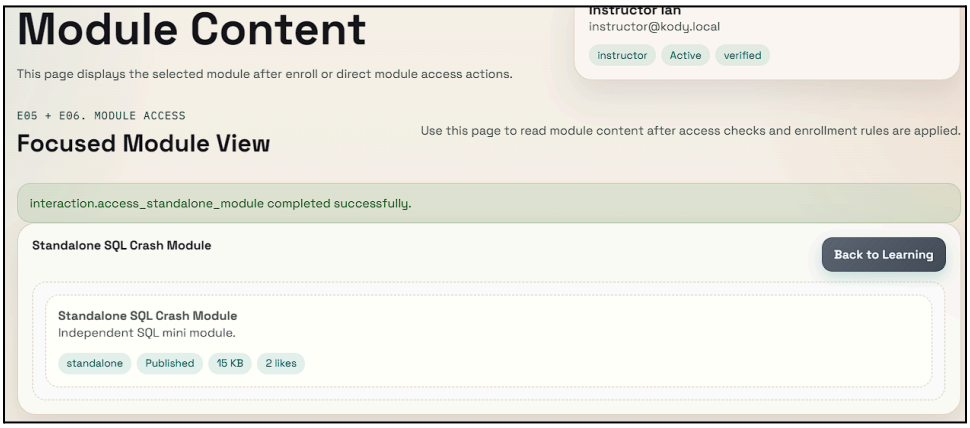




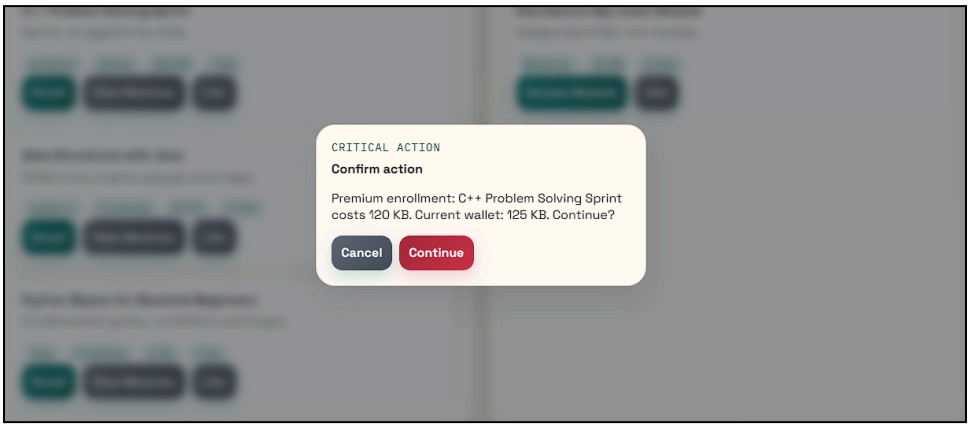


Purpose	Allows authorized users to create, edit, archive, or delete courses and modules
Navigation and user	Authorized users access course management tools to create or modify course and module details. Courses/modules may be archived or deleted, updating their availability in the system.

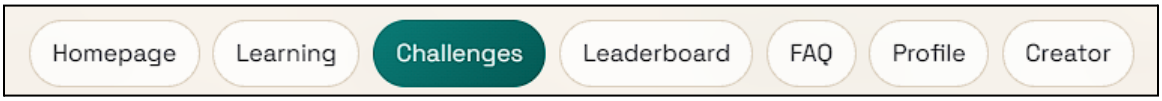
5.1.g Module Access Interface

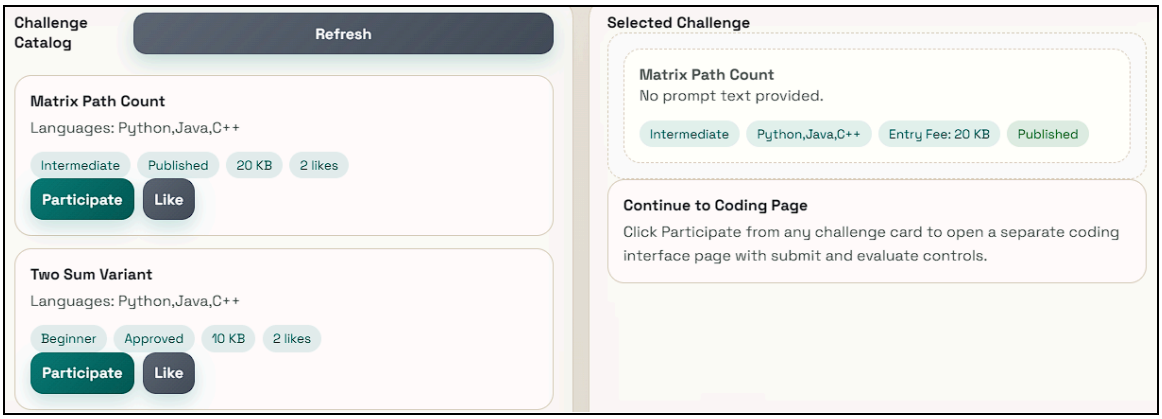
KODY-MODULE-ACCESS	
	
Purpose	Allows users to access course modules or standalone modules.
Navigation and user	The user selects a module from a course or standalone listing. The system displays module content for learning and progression.

5.1.h Enrollment Interface

KODY-ENROLL	
	
Purpose	Allows users to enroll in courses using tokens (or for free).
Navigation and user	The user selects a course and confirms enrollment. If required, tokens are deducted. Successful enrollment grants access to course modules.

5.1.i Challenge Browsing & Participation Interface

KODY-CHALLENGE-BROWSE




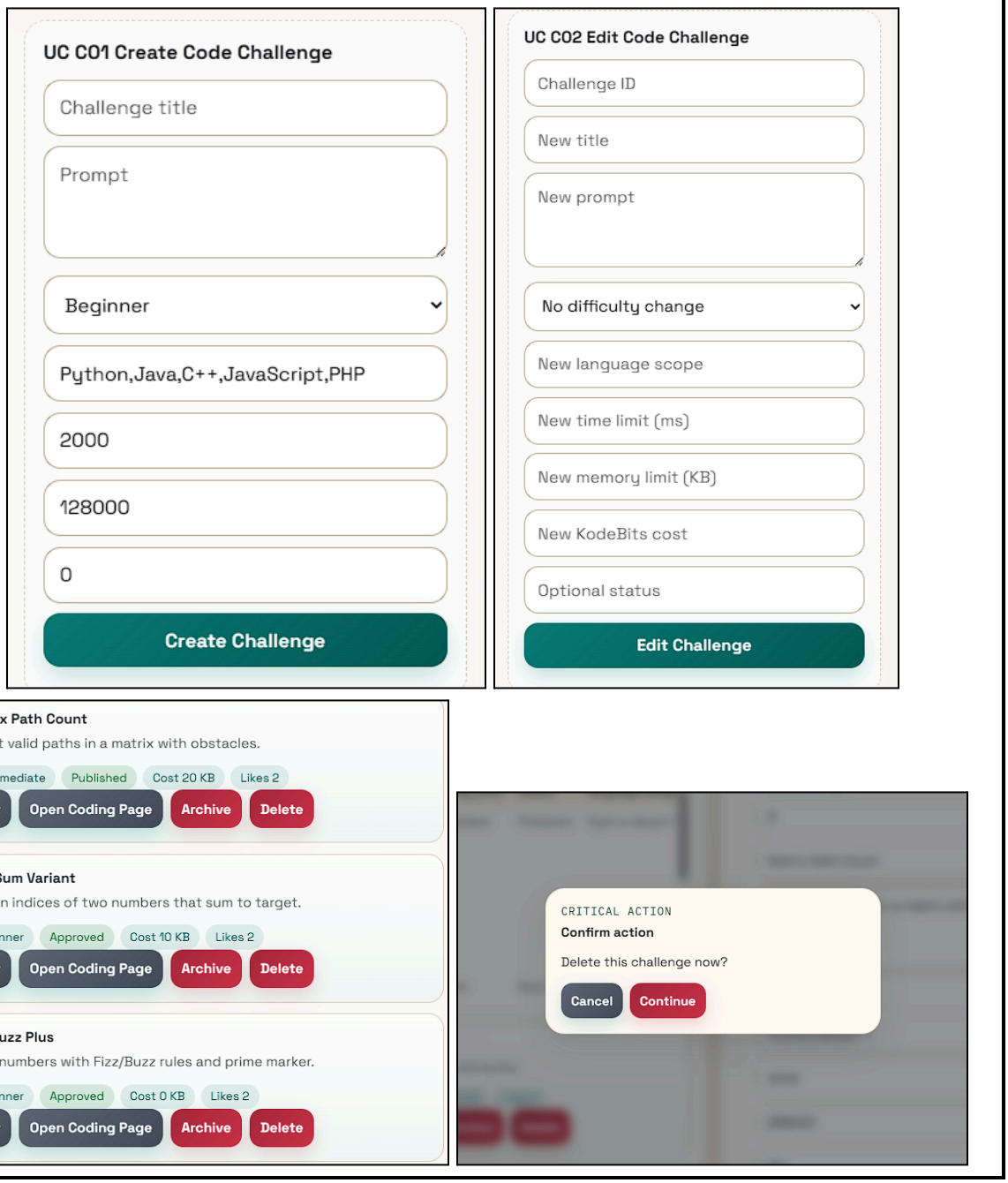
Purpose	Allows users to browse and participate in coding challenges.
Navigation and user	The user browses available challenges and selects one to participate in. The system provides problem details and submission options.

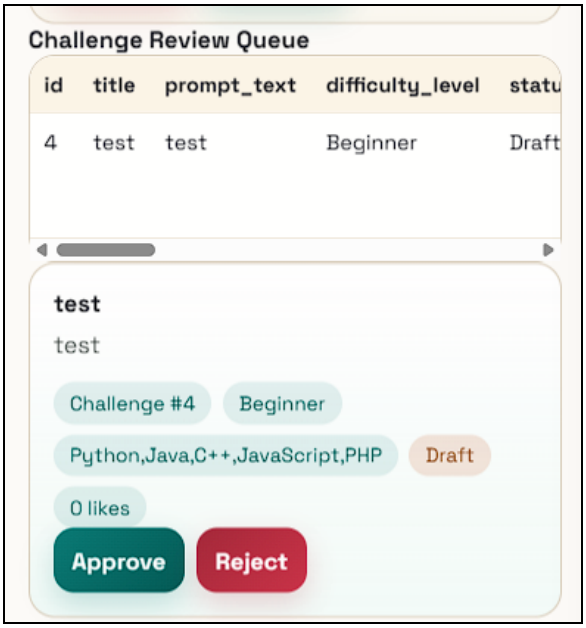
5.1.j Code Submission & Evaluation Interface

KODY-CHALLENGE-RESULT	
	
Purpose	Allows users to submit code and view execution feedback.
Navigation and user	The user submits code solutions to a challenge. The system evaluates the submission and displays execution results and feedback.

5.1.k Challenge Management Interfaces

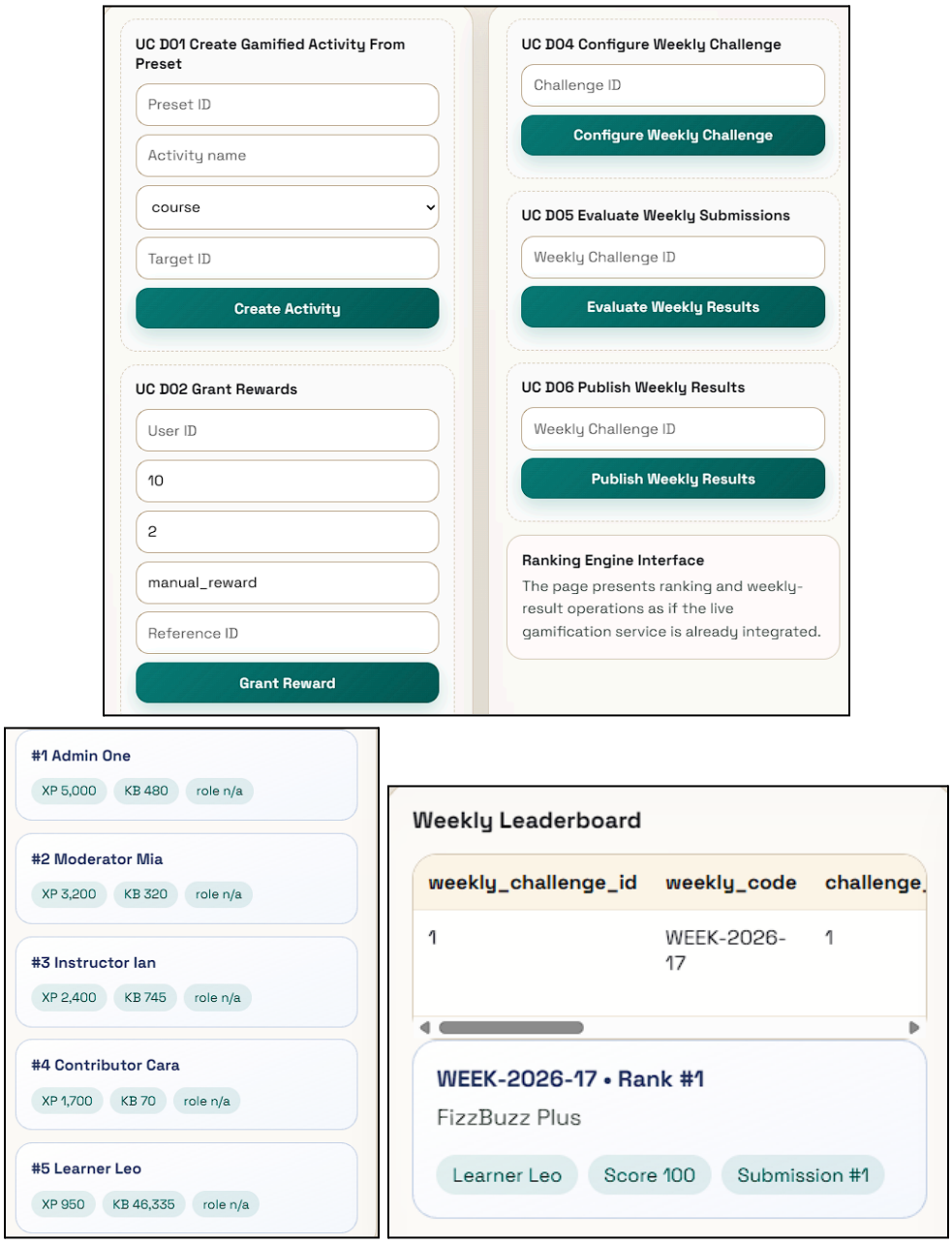
KODY-CHALLENGE-MNGT





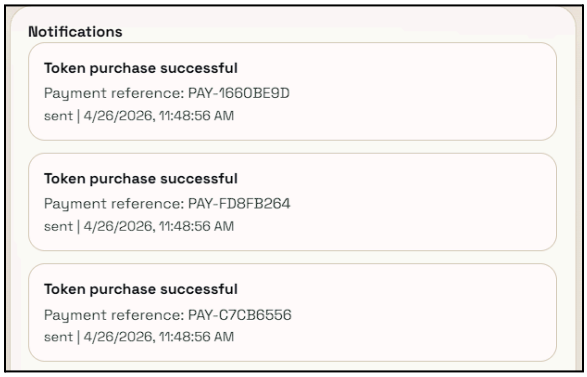
Purpose	Allows creation, editing, archiving, deletion, approval of challenges.
Navigation and user	Submitted challenges require admin approval before becoming available to users.

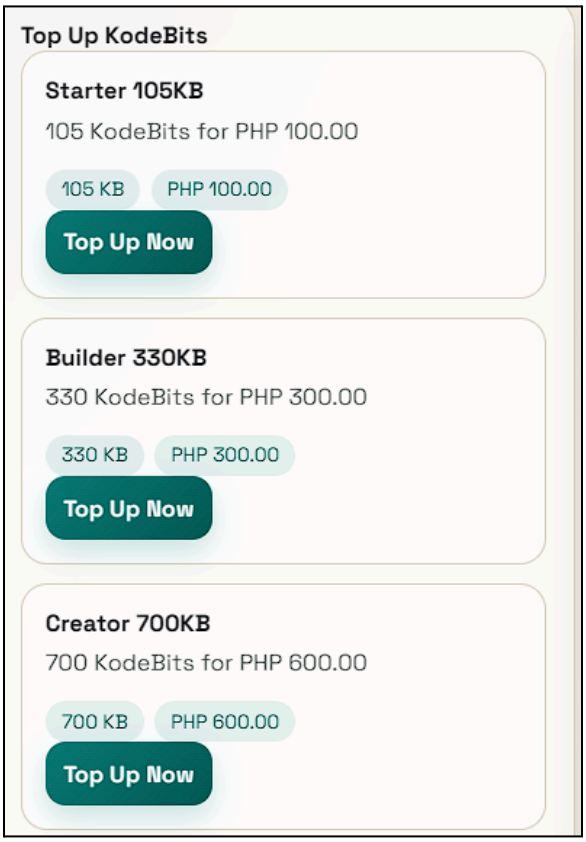
5.1.1 Gamification & Leaderboard Interfaces

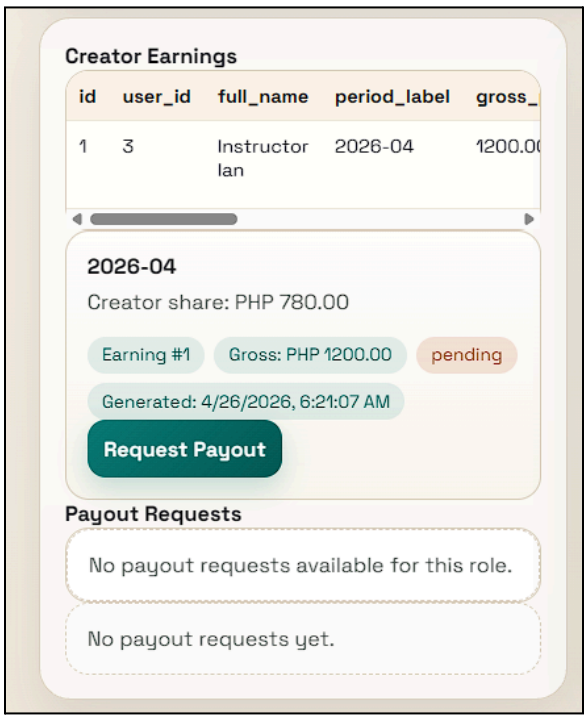
KODY-GAME-LEADERBOARD	
 	
Purpose	Displays rankings, manages gamified activities, and weekly challenges.
Navigation and user	The user views rankings and participates in gamified activities. Administrator configures challenges, evaluates submissions, and publishes results.

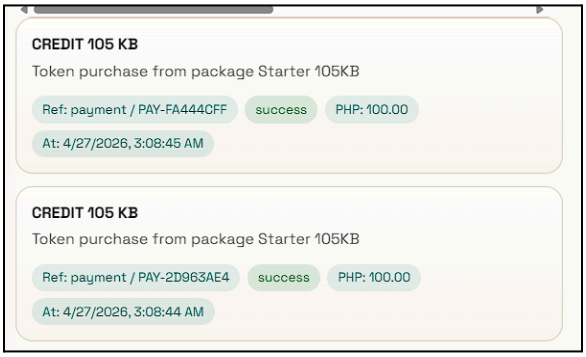
5.1.m Transactions & Notifications Interfaces

KODY-PURCHASE-AND-NOTICE







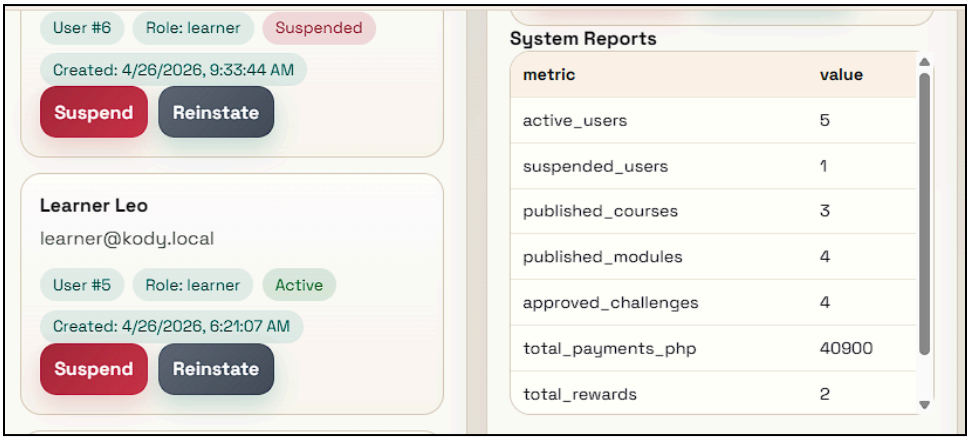


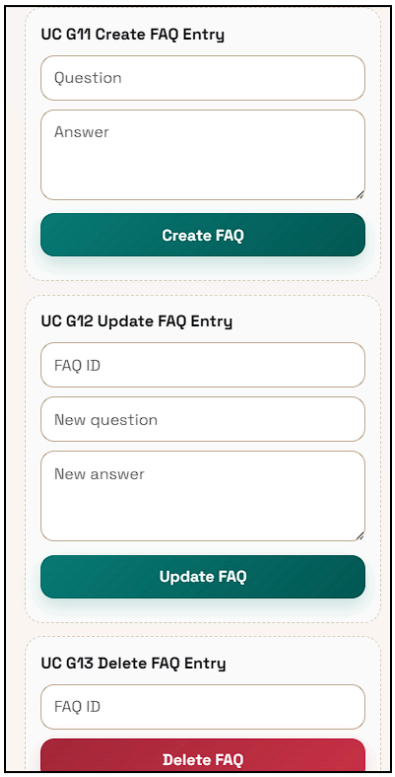
Purpose	Handles token purchases, usage, earnings, and system notifications.
Navigation and user	The user purchases or spends tokens for access to content. Earnings are tracked and received.

	Notifications inform the user of system updates and activities.

5.1.n Administration & Governance Interfaces

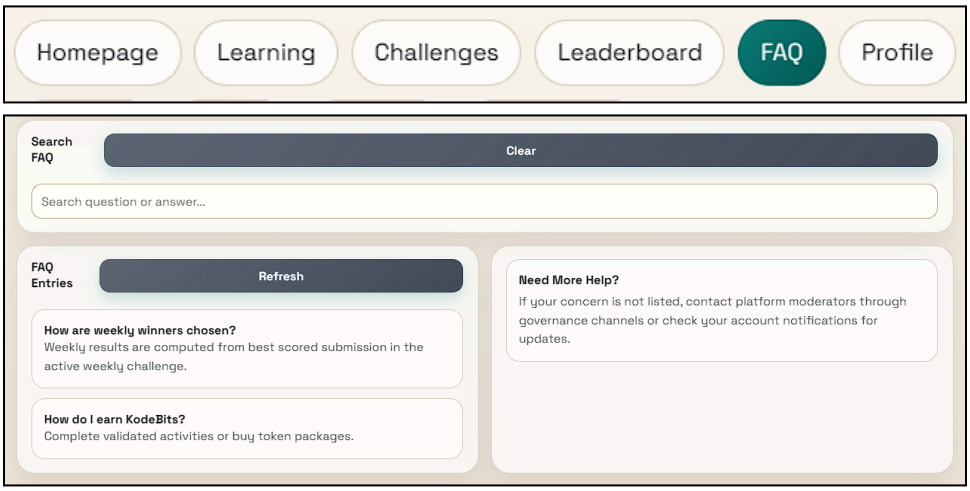
KODY-ADMIN





Purpose	Enables system monitoring, moderation, and configuration.
Navigation and user	Administrators access system controls to moderate content, manage users, configure game presets, generate reports, and maintain FAQ entries.

5.1.o FAQ Interface

KODY-FAQ	
	
Purpose	Contains guides and answers to assist users through navigating the platform
Navigation and user	Users can click on the FAQ from the navigation bar or they may see the FAQ carousel in the homepage.

5.2 Logical Data Design

5.2.a DDT-User-Account-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
user_id	int	Unique identifier of the user account	Admin, Moderator	System Administrator	Required, unique, auto-generated
username	varchar(30)	Unique username used for login	Learner, Contributor, Instructor, Moderator, Admin	Account Owner	Required, unique, minimum 6 characters, maximum 30 characters
email_address	varchar(100)	Registered email address of the user			Required, unique, valid email format
password_hash	varchar(255)	Encrypted account password			Required, 2-32 characters; mix of uppercase/lowercase letters, numbers, and symbols before encryption
first_name	varchar(50)	User's first name			Required, alphabetic

					characters only
last_name		User's last name			Required, alphabetic characters only
profile_picture	varchar(255)	Profile image reference or file path			Optional, accepted image formats only
account_role	varchar(20)	Role assigned to the user account	Admin	Admin	Required, values limited to Learner, Contributor, Instructor, Moderator, Admin
account_status		Current status of the account	Admin, Moderator	Admin, Moderator	Required, values limited to Active, Suspended, Archived, Deleted
email_verified	boolean	Indicates whether email has been verified	Admin	Account Owner	Required, default value is False
last_login_at	datetime	Timestamp of latest successful login		System Generated	Auto-generated upon successful login
recovery_email	varchar(100)	Email used for account recovery	Learner, Contributor, Instructor, Moderator, Admin	Account Owner	Must follow valid email format
archive_reason	varchar(255)	Reason for archiving account	Admin, Moderator	Admin, Moderator	Required when account is archived
suspension_reason		Reason for account suspension			Required when account is suspended

5.2.b DDT-Contributor-Request-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
request_id	int	Unique identifier for contributor request	Admin, Moderator	System Administrator	Required, unique, auto generated
applicant_user_id		User requesting		Learner	Required, must

		contributor role			reference an existing user
request_message	varchar(255)	Explanation for contributor request			Required, maximum 500 characters
portfolio_link		Link to portfolio or supporting work			Optional, must follow valid URL format
completed_modules_count	int	Number of completed learning modules		System Generated	Must be greater than or equal to 25
completed_challenges_count		Number of completed coding challenges			Must be greater than or equal to 50
account_age_days		Number of days since account registration			Must be greater than or equal to 30
approval_status	enum('pending', 'approved','rejected')	Status of contributor request		Moderator, Admin	Values limited to Pending, Approved, Rejected
moderator_feedback	varchar(255)	Moderator comments regarding request decision			Optional, maximum 500 characters
reviewed_at	datetime	Date and time request was reviewed			Auto-generated upon review

5.2.c DDT-Instructor-Verification-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
verification_id	int	Unique verification request identifier	Admin	Admin	Required, unique, auto-generated
instructor_user_id		User requesting instructor verification		Instructor Instructor Instructor Instructor	Required, must reference existing user
institution_name	varchar(100)	Name of educational institution			Required
credential_document	varchar(255)	Uploaded proof of teaching credentials			Required, PDF or image format only

specialization	varchar(100)	Instructor specialization or expertise			Required
verification_status	varchar(20)	Verification outcome		Admin	Values limited to Pending, Approved, Rejected
verification_notes	varchar(255)	Remarks regarding verification review			Optional
verified_at	datetime	Date and time verification was completed			Auto-generated upon verification

5.2.d DDT-Course-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
course_id	int	Unique identifier of the course	Instructor, Admin	Instructor	Required, unique, auto-generated
course_title	varchar(150)	Name of the course			Required, unique within category
course_description	text	Overview of course content			Required
course_category	varchar(50)	Category classification of the course			
difficulty_level	varchar(20)	Difficulty level of the course			Values limited to Beginner, Intermediate, Advanced
estimated_duration	int	Estimated completion duration in hours			Must be greater than 0
course_thumbnail	varchar(255)	Course image or thumbnail reference			Optional, image format only
course_status	varchar(20)	Publication status of the course		Instructor, Admin	Values limited to Draft, Published, Archived, Deleted
created_by	int	User who created the course	Admin	Instructor	Required, must reference existing user

created_at	datetime	Date and time course was created		System Generated	Auto-generated
updated_at		Date and time course was last updated			Auto-generated upon modification

5.2.e DDT-Module-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
module_id	int	Unique identifier of the module	Instructor, Admin	Instructor	Required, unique, auto-generated
module_title	varchar(150)	Title of learning module			Required
module_description	text	Description of learning module			
module_content		Educational content of the module			
module_type	enum('video', 'article', 'interactive')	Classification of module			Values limited to Video, Article, Interactive
module_status	enum('draft', 'published', 'archived', 'deleted')	Status of module		Instructor, Admin	Values limited to Draft, Published, Archived, Deleted
course_id	int	Associated course identifier		Instructor	Optional for standalone modules
module_order		Sequence order within a course			Must be greater than 0
created_by		User who created the module	Admin		Required
created_at	datetime	Date and time module was created		System Generated	Auto-generated

5.2.f DDT-Challenge-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
challenge_id	int	Unique identifier of	Contributor, Instructor,	Contributor, Instructor	Required, unique,

		code challenge	Moderator, Admin		auto-generated
challenge_title	varchar(150)	Title of code challenge			Required
challenge_description	text	Problem statement of challenge			
programming_language	varchar(50)	Required programming language			
difficulty_level	enum('easy','medium','hard')	Difficulty classification			Values limited to Easy, Medium, Hard
challenge_rules	text	Rules and constraints of challenge			Required
expected_output	text	Expected output format			
challenge_status	enum('pending', 'approved', 'archived', 'deleted')	Status of challenge		Moderator, Admin	Values limited to Pending, Approved, Archived, Deleted
approval_feedback	varchar(255)	Moderator remarks regarding approval	Moderator, Admin	Moderator	Optional
created_by	int	Creator of challenge	Admin	Contributor, Instructor	Required

5.2.g DDT-Submission-Evaluation-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
submission_id	int	Unique identifier for code submission	Learner, Moderator, Admin	Learner	Required, unique, auto-generated
challenge_id		Associated challenge identifier			Required, must reference existing challenge
submitted_code	text	Source code submitted by learner			Required

programming_language	varchar(50)	Language used in submission			
execution_result	text	Output from code execution		System Generated	Auto-generated after evaluation
test_cases_passed	int	Number of successful test cases			Must be greater than or equal to 0
total_test_cases	int	Total number of executed test cases			Must be greater than 0
evaluation_status	enum('passed', 'failed', 'pending')	Result of evaluation			Values limited to Passed, Failed, Pending
feedback_message	varchar(500)	Evaluation feedback details			Optional
submitted_at	datetime	Date and time of submission	Admin	Learner	Auto-generated

5.2.h DDT-Gamification-Rewards-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
activity_id	int	Unique identifier for gamified activity	Admin	Admin	Required, unique, auto-generated
preset_name	varchar(100)	Name of selected game preset			Required
reward_points	int	Number of points awarded			Must be greater than or equal to 0
badge_name	varchar(100)	Badge awarded to user			Optional
leaderboard_rank	int	User ranking position	Learner, Admin	System Generated	Must be greater than 0
weekly_challenge_title	varchar(150)	Title of weekly challenge	Admin, Moderator, System Generated	Admin, System Generated	Required
submission_score	decimal(5,2)	Score achieved in weekly challenge		Moderator, Admin	Range 0 to 100

result_status	enum('draft', 'published')	Publication status of weekly result		Admin	Values limited to Draft, Published
published_at	datetime	Date and time results were published			Auto-generated upon publication

5.2.i DDT-Learning-Progress-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
enrollment_id	int	Unique identifier for enrollment	Learner, Admin	Learner	Required, unique, auto-generated
learner_user_id		Learner enrolled in course			Required
course_id		Enrolled course identifier			
enrollment_date	datetime	Date of course enrollment	Admin	System Generated	Auto-generated
completion_percentage	decimal(5,2)	Percentage of completed progress	Learner, Admin		Range 0 to 100
completed_modules	int	Number of completed modules			Must be greater than or equal to 0
last_accessed_module		Most recently accessed module			Must reference existing module
dashboard_summary	text	Personalized progress overview	Learner		Auto-generated summary

5.2.j DDT-User-Interaction-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
reaction_id	int	Unique identifier of content reaction	Learner, Contributor, Instructor, Admin	Learner, Contributor, Instructor	Required, unique, auto-generated
content_id		Referenced content identifier			Required

reaction_type	enum('like','helpful','favorite')	Type of reaction given			Values limited to Like, Helpful, Favorite
faq_id	int	Unique FAQ entry identifier	Admin	Admin	Required, unique, auto-generated
faq_question	varchar(255)	FAQ question text			Required
faq_answer	text	FAQ answer content			Required
faq_status	enum('active', 'archived', 'deleted')	Current status of FAQ entry			Values limited to Active, Archived, Deleted
updated_at	datetime	Date and time FAQ entry was updated		System Generated	Auto-generated

5.2.k DDT-Token-And-Earnings-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
transaction_id	int	Unique identifier for financial transaction	Admin	Learner, Contributor	Required, unique, auto-generated
user_id	int	User associated with transaction			Required
token_amount	int	Number of tokens purchased or used		Learner	Must be greater than 0
transaction_type	enum('purchase', 'usage', 'earnings')	Type of transaction		Learner, System Generated	Values limited to Purchase, Usage, Earnings
payment_method	varchar(50)	Method used for token purchase		Learner	Required for purchases
earnings_amount	decimal(10,2)	Amount earned by publisher	Contributor, Admin	System Generated	Must be greater than or equal to 0

payout_status	enum('pending', 'paid')	Status of earnings payout	Contributor, Admin	Admin	Values limited to Pending, Paid
transaction_date	datetime	Date and time transaction occurred	Admin	System Generated	Auto-generated

5.2.l DDT-Notification-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
notification_id	int	Unique notification identifier	Learner, Contributor, Instructor, Moderator, Admin	System Generated	Required, unique, auto-generated
recipient_user_id		User receiving the notification			Required
notification_title	varchar(150)	Title of notification			
notification_message	varchar(500)	Notification message content			
notification_type	varchar(50)	Category of notification			Values limited to Account, Course, Challenge, Rewards, Transaction
created_at	datetime	Date and time notification was generated	Admin		Auto-generated

5.2.m DDT-Moderation-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
moderation_id	int	Unique moderation record identifier	Moderator, Admin	Moderator	Required, unique, auto-generated
moderated_content_id		Identifier of moderated content	Moderator, Admin		Required
content_type	varchar(50)	Type of moderated content	Moderator, Admin		Values limited to Course, Module, Challenge, FAQ

moderation_action	varchar(50)	Action performed during moderation	Moderator, Admin		Values limited to Approved, Rejected, Archived, Flagged
moderation_reason	varchar(255)	Reason for moderation action	Moderator, Admin		Required
report_type	varchar(50)	Type of generated report	Admin	Admin	Values limited to User Activity, Revenue, Content Performance
generated_at	datetime	Date and time report was generated		System Generated	Auto-generated

5.2.n DDT-Game-Preset-Information

Data Item	Type and Length	Description	Security (Control)	Responsible (User)	Domain Validity Rules
preset_id	int	Unique identifier of game preset	Admin	Admin	Required, unique, auto-generated
preset_name	varchar(100)	Name of game preset			Required, unique
preset_description	text	Description of game preset			Required
reward_configuration		Reward mechanics and configuration			
preset_status	enum('active', 'archived', 'deleted')	Status of game preset			Values limited to Active, Archived, Deleted
updated_at	datetime	Date and time preset was updated		System Generated	Auto-generated